

Contents

1	Introduction	3
2	System Architecture and Requirements	4
2.1	Requirements	4
2.2	Digital Architectures	5
2.3	Notation and Default Parameters	5
3	FPGA Programming	5
3.1	Why Would You Use an FPGA?	7
3.2	How Do You Use an FPGA?	7
3.2.1	The Issue of Floating Point Numbers	8
3.2.2	Types and Optimizations	9
3.2.3	Valid Signals	10
3.2.4	Testing and Verification Strategy	10
3.3	The DAC	11
4	Digital Design and Implementations	12
4.1	Symbol Shaping	13
4.1.1	Comparing Candidate Modulation Schemes	13
4.1.2	Implementation of MQAM	15
4.2	Single Carrier	15
4.2.1	Pulse Shaping	15
4.2.2	Frequency Shifting	17
4.2.3	Testing and Verification	17
4.3	Frequency Multiplexing - OFDM & SC-FDMA	20
4.3.1	FFT Implementation on the FPGA	20
4.3.2	Orthogonal Frequency-Division Multiplexing	23
4.3.3	OFDM Module - FPGA Implementation and Testing	24
4.3.4	OFDM SDR - Implementation and Testing	25
4.3.5	Single Carrier Frequency-Division Multiple Access	29
4.3.6	SC-FDMA Module - FPGA Implementation and Testing	30
4.3.7	SC-FDMA SDR - Implementation and Testing	31
5	Analog Circuitry - From FPGA to Antenna	33
5.1	Signal Chain Design	33
5.1.1	The Low-Pass Reconstruction Filter	34
5.1.2	Fully Differential Amplifier - THS4552	35
5.1.3	Quadrature Modulator - ADL5375	36
5.1.4	Local Oscillator - ADF4351	36
5.1.5	Proposed Output Stage	37
5.2	PCB Design	37
6	Results and Discussion	38
7	Conclusion	41
8	References	43
	Appendices	45

Abstract - English

A Software Defined Radio allows some or all physical-layer functions to be implemented in configurable hardware or software, enabling flexible radio-based systems.

This project designs, implements, and evaluates the digital baseband transmission chain for an FPGA based SDR transmitter, and also includes the design, but not verification, of a proposed RF front-end.

The SDR architecture is based on a modular design built on blocks using a common interface, allowing easy interchangeability. Three different baseband architectures were implemented in VHDL; a single-carrier 16QAM transmitter with Root Raised Cosine based pulse shaping, an OFDM transmitter, and an SC-FDMA transmitter. Both the OFDM and SC-FDMA transmitters are built on a custom configurable FFT/IFFT core.

The three baseband architectures were tested under similar circumstances, allowing for direct comparison between them. They all met the 50 MHz timing requirement and fit comfortably within the DE25 FPGA. The transmitted deterministic payload was decoded without errors for all architectures. The measured EVM values were -23.8 dB, -36.2 dB, and -42.8 dB for single-carrier, OFDM, and SC-FDMA respectively. The RF front-end was not tested, but the design fit the given requirements.

The results verify that the implemented FPGA baseband transmitter is suitable as a prototype for an SDR transmitter, while full wireless verification would require experimental testing of the RF front-end.

Abstrakt - Dansk

En Software Defineret Radio har nogen eller alle funktioner i det fysiske lag implementeret i konfigurerbar hardware eller software, hvilket tillader fleksible radio-baserede systemer.

Dette projekt designer, implementerer og evaluerer den digitale baseband transmissions-kæde for en FPGA baseret SDR transmitter, og det inkluderer også en foreslået, men ikke verificeret, RF front-end.

SDR-arkitekturen er baseret på et modulært design opbygget af blokke med en fælles grænseflade, hvilket muliggør nem udskiftning og uafhængig test af de enkelte moduler. Tre forskellige baseband-arkitekturer blev implementeret i VHDL: en single-carrier 16QAM-sender med Root-Raised Cosine-pulsformning, en OFDM-sender og en SC-FDMA-sender. Både OFDM- og SC-FDMA-senderne er baseret på en specialudviklet konfigurerbar FFT/IFFT-kerne.

De tre baseband-arkitekturer blev testet under lignende forhold, hvilket muliggjorde en direkte sammenligning mellem dem. Alle tre opfyldte timingkravet på 50 MHz og passede komfortabelt inden for ressourcerne på DE25-FPGA'en. Den transmitterede deterministiske payload blev dekoderet uden fejl for alle arkitekturer. De målte EVM-værdier var henholdsvis -23.8 dB, -36.2 dB og -42.8 dB for single-carrier, OFDM og SC-FDMA.

Resultaterne verificerer den implementerede FPGA-baseband-sender som en funktionel digital prototype for en SDR-sender, mens fuld trådløs verificering kræver eksperimentel test af RF-front-end'en.

1 Introduction

The Wireless Innovation Forum has, in collaboration with the IEEE P1900.1 working group, defined what a Software Defined Radio (SDR) is [1]:

"A radio in which some or all of the physical layer functions are software defined."

The main benefit of an SDR is the large amount of flexibility offered from a physical layer defined by software. It allows modifications of the software defined components to be made on the fly, such as the modulation scheme, data rate or even carrier frequency. This is a significant advantage in many situations, for example during product development, by allowing much faster prototyping, or educational settings, where different radio architectures can be implemented, tested and compared.

An SDR can be based on different types of configurable hardware, from normal CPUs to FPGAs. The benefit of an FPGA based SDR over a CPU based one is its suitability for the highly parallelized workflow and deterministic timing requirements involved in the signal processing for a real-time SDR, such as FFT-based modulations like Orthogonal Frequency Division Multiplexing (OFDM) and Single-Carrier Frequency Division Multiple Access (SC-FDMA), or high-speed real-time DAC interfacing, which can be a challenge in many CPU based systems. Although FPGA development is generally more complex than software development on a CPU, an FPGA allows the signal chain to be implemented as parallel, clocked hardware with deterministic timing. This makes it well suited for real-time digital baseband processing, where several operations may need to be performed every clock cycle. A central engineering problem is therefore how to design a digital baseband transmitter that is flexible enough to support different waveforms, while still meeting the resource and timing constraints of an FPGA implementation.

This project investigates the following questions:

1. How can a custom streaming FFT/IFFT core be implemented within the constraints of the Altera DE25 FPGA, while being able to maintain a 50 MHz timing requirement?
2. Can single-carrier, Orthogonal Frequency Division Multiplexing (OFDM), and Single-Carrier Frequency Division Multiple Access (SC-FDMA) transmission chains be implemented on the DE25 FPGA while meeting a 50 MHz timing requirement?
3. How do the implemented architectures compare in terms of EVM, PAPR, spectral efficiency, resource utilization, and suitability for wireless transmission?
4. How does the FPGA based digital transmission chain interface with an RF front-end, and what challenges arise in the design of one?

The goal of this project is therefore to design, implement and evaluate the digital baseband transmission chain for an SDR with a focus on modern technologies, such as OFDM and SC-FDMA, implemented with a configurable custom real-time streaming FFT/IFFT module. They are evaluated experimentally under comparable conditions using timing closure, FPGA resource utilization, payload decoding, Error Vector Magnitude (EVM), Peak-To-Average-Power Ratio (PAPR), bandwidth, and spectral efficiency. The theoretical background is used to justify architectural choices such as pulse shaping, cyclic prefixes, pilot-based channel estimation, and DFT precoding, while experimental measurements from the DAC output are used to verify the implemented designs. This combination of theoretical analysis, FPGA implementation, simulation, and measurement forms the basis for evaluating whether the proposed SDR transmitter architecture satisfies the project requirements. Additionally, the remaining RF front-end is designed and discussed, but not experimentally verified due to issues with shipping.

The architecture for the project is based on a modular approach, where each component of the signal chain is a separate module with a clear and consistent interface based on valid signals, allowing easy testing and verification of individual blocks, as well as simple extension of the system by implementing new modules or modifying existing blocks. Additionally, all the blocks

Category	Requirement	Reason
Digital Timing	Must meet 50 MHz clock	Matches FPGA clock [2]
Digital Performance	Must decode 16QAM test payload without errors	Functional verification
Resource utilization	Must fit within DE25 resources	Hardware feasibility
System Architecture	Must support complex baseband signals	Allows full utilization of bandwidth
Modulation Scheme	Must be suitable for wireless transmission	Suitability for use as a transmitter
RF front-end	Must be able to upconvert to 2.4 GHz	Commonly available ISM band [3]
Modularity	System must be built of modular components with clearly separated responsibilities	Simplifies testing, verification, and extension
Evaluation	Implementations must be evaluated using comparable metrics	Allows fair comparisons

Table 1: System Requirements

are designed to be configurable, with many of the parameters being changeable, such as the size of the IFFT for OFDM, or the number of samples in an SC-FDMA symbol.

The modular architecture of the digital signal chain allows verification to be performed on several levels. Individual VHDL modules can be simulated, with the output written to a file and analyzed externally, and the larger transmitter chains are verified by transmitting a deterministic payload through the DAC, which is then sampled, decoded and analyzed externally. This made it possible to test things like fixed-point quantization, DAC behaviour and signal quality of the full transmitter.

First, the FPGA development flow, fixed-point representation, valid-signal interface and DAC behavior are examined and discussed. Next, a single-carrier transmitter is implemented as a reference, using MQAM symbol mapping, Root-Raised Cosine pulse shaping filter, and a frequency shifter. Following this, a custom configurable streaming FFT/IFFT module is developed and verified. Then an OFDM based transmitter is implemented and verified using the aforementioned FFT/IFFT module. Lastly an SC-FDMA based transmitter is implemented and verified using the OFDM module and FFT core for DFT precoding. All three implementations use the same testing framework, allowing for direct comparison between them.

2 System Architecture and Requirements

The goal of the project is to design, implement, and evaluate an FPGA-based SDR transmitter. The main focus is on the digital baseband signal chain, including FPGA implementation, DAC interfacing, and verification of the analog output. Additionally, an RF front-end is designed to support future wireless transmission, however the experimental verification of this front-end is outside the scope of the project, as the manufactured PCBs were not delivered on time.

2.1 Requirements

The requirements of the system are based on a mixture of the constraints of the available hardware, relevant regulations, and the need for functional correctness and reliable transmissions. The requirements are given by Table 1. The implemented architectures will be compared to each other and the requirements in chapter 2.1.

2.2 Digital Architectures

Several digital architectures are implemented, but they are all based on the same shared architecture due to the modularity requirement. The common architecture is shown in Figure 1.

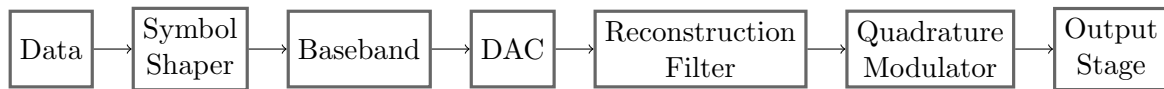


Figure 1: Block diagram of the Broader System Architecture

Where the Baseband block is the interchangeable digital baseband implementation.

The common architecture is worth discussing, starting with the data source and symbol interface. In an effort to simplify the verification and comparison of each baseband implementation, the same data source and symbol interface are used, as stated in the requirements. In the current project, the data source is a deterministic set of ASCII characters, specifically 1024 bytes of Lorem Ipsum text. While this does mean that the first bit is always zero, leading to uneven symbol utilization, the deterministic and readable nature of the payload was prioritized because it simplifies verification. However, in the future, an interface with an outside data source, allowing for transmission of arbitrary data, would be a priority to implement. The symbol interface takes in a series of bits, determined by the specific modulation, and outputs the corresponding complex symbol.

The advantage of a modular system design is that every block is agnostic to the global state of the system, allowing easy extension, interchanging, and testing and debugging of modules. However, to achieve this, it is necessary to develop a strategy for inter-module communication, which, as will be discussed in chapter 3.2.3, is based on valid signals, such that every module has inputs and outputs for both data and valid signals. As such, each block is designed to operate at the maximum throughput possible, with a global controller deciding when to request new symbols, limited by the lowest throughput module.

With this modular design philosophy, it is easy to change the baseband processing modules, so long as it adheres to the valid signal interface. For this project three different baseband modules were implemented;

- Single-carrier, to be used as a simple reference implementation.
- OFDM, which is Robust to frequency-selective channels, and widely used in modern systems [4].
- SC-FDMA, which solves one of the main issues with OFDM, its high PAPR, but is more complicated. It is commonly associated with uplink cellular systems [5].

Each of these baseband modules are implemented in the same modular fashion, allowing testing and verification of the individual blocks, as well as the wider digital SDR by inserting the block in the chain, and measuring the output of the DAC for decoding and verification via an external tool, which in this project is Julia. While the Error Vector Magnitude (EVM) is not listed in the requirements, it is used to compare the implementations.

2.3 Notation and Default Parameters

This project utilizes many different parameters, each with their own symbol, and, when relevant, a default value in tests. The most important of these can be seen in Table 2.

3 FPGA Programming

An FPGA, Field Programmable Gate Array, is a type of configurable hardware, consisting of blocks of logic, configurable interconnectors, and even memory in some cases. They are programmed using a Hardware Description Language (HDL), such as VHDL [6], which will be used

Symbol	Meaning	Default value in tests
F_s	FPGA & DAC sample rate	50 MHz
R_s	Single-carrier symbol rate	5 MSym/s
N	FFT/IFFT size	1024
M	Number of active subcarriers	256
M_{QAM}	QAM constellation order	16
N_{CP}	Cyclic prefix length	8
N_{pilot}	Number of internal pilots	8
k_0	First data-bin	22
β	RRC roll-off factor	0.35
f_{IF}	Intermediate frequency	architecture-dependent
f_{RF}	Desired carrier frequency	2.4 GHz
f_{LO}	Local Oscillator frequency	$f_{\text{RF}} - f_{\text{IF}}$
B	Bandwidth	Context dependent
N_{LUT}	Number of samples in the sine/cos LUT	4096

Table 2: Overview of notation and default values in tests.

in this project. FPGAs are often placed somewhere in between normal CPUs and Application Specific Integrated Circuits (ASIC), which are application specific ICs that cannot be changed later, unlike FPGAs which are reconfigurable. The Terasic DE25 Development and Education Board, which will be used in this project, is based on an Altera Agilex 5 SoC, with 46,800 Adaptive Logic Modules (ALMs), which are Lookup Table (LUT) based combinational logic blocks combined with optional output registers for synchronous operations, roughly comparable to a 6-input LUT, 358 RAM blocks, containing a total of 7.3 million bits of storage, and 188 Digital Signal Processing blocks of various types, mostly 18 bit multiplications [2]. A photo of the DE25 is shown in Figure 2.

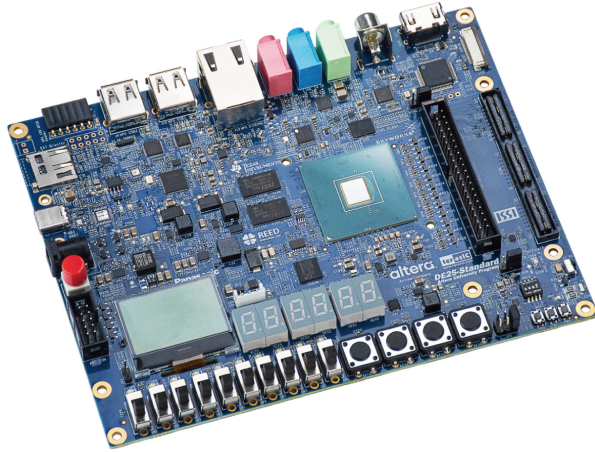


Figure 2: Terasic DE25-Standard Board. Source: [7]

It also contains a large amount of configurable interconnectors that connect both the I/O pins to the logic blocks, as well as connecting the blocks to each other. Some modern FPGA SoCs even contain a CPU, usually referred to as a Hard Processor System (HPS), which can be used to complement the FPGA. The DE25 contains such an HPS. This is naturally an oversimplification, since modern FPGAs are very complicated, especially ones as powerful as the Agilex 5 series, but this is sufficient knowledge for this project.

3.1 Why Would You Use an FPGA?

While FPGAs can be used in place of microcontrollers, they are generally a lot more specialized than normal CPUs, and as such are mostly used in highly parallelizable systems where high throughput and deterministic behaviour is desired. Before discussing when the use of an FPGA is appropriate, it is pertinent to consider the strengths and weaknesses of both systems.

CPUs are inherently sequential, they execute one instruction per clock-cycle.¹ This allows them to run at very high frequencies, with many modern CPUs running at several gigahertz, however the instructions they can execute are generally relatively simple, and so most programs must be decomposed into many smaller instructions, increasing the time it takes to execute a program [8]. Additionally, if your program is running on a system with an operating system, such as most normal computers, it will be competing for compute time with all the other processes that are running, which is usually decided by the scheduler, that is, outside of very specialized applications, almost always an indeterminate process, meaning the program cannot be certain as to when it will be allowed processing time [9]. This isn't true on many modern microcontrollers, such as Arduinos, since they do generally not have operating systems [10].

This is very different on an FPGA, which can, depending on its size and how you configure it, execute any number of operations at once [11]. Generally there are two main ways of configuring an FPGA, synchronously and asynchronously (or a mix of both). In asynchronous operations, also called combinational logic, the output of an instruction will execute instantly² upon a change to any of the inputs, whereas synchronous instructions execute upon a clock, not unlike in a CPU. The desire for synchronous operation introduces the need for a clock, and unlike modern CPUs that generally run in the low gigahertz, most FPGAs run at megahertz speed. This project will aim for an operating frequency of 50MHz, as the DE25 includes several 50MHz oscillators for this exact purpose [2].

FPGAs are also often used for prototyping, especially for projects that will later require the use of ASICs, as an FPGA can be used to test the functionality of them before committing to large orders.

The need for an FPGA in this project was partially determined due to the highly parallelized nature of the Fast Fourier Transform, which means many digital signal processing projects lend themselves naturally to FPGAs. Secondly, this project is partially about learning about FPGA development in the context of communication systems and signal processing.

3.2 How Do You Use an FPGA?

Having discussed what an FPGA is, and why you would choose to use one, it is appropriate to consider how FPGA development is done. There are two major companies that produce them, Altera and Xilinx. For this project an Altera board is used, specifically the Altera DE25. This Development and Education board was chosen due to its large amount of RAM and DSP blocks, which are needed for the custom real-time FFT/IFFT core, and because it has an integrated HPS. The software that is used is Quartus Prime Pro alongside the free Agilinx 5 license. Depending on the specific Altera FPGA chip, other versions of Quartus may be necessary [12].

There are two main HDLs that are used for FPGA development³, VHDL and (System)Verilog. This project uses VHDL, but both languages are viable and widely used in industry [13].

Programming in an HDL is very different from any other modern programming language, since the code does not execute sequentially line by line, but rather executes all relevant instructions at once. There are a few major programming paradigms in HDLs, combinational/dataflow and behavioural [14]. The former is generally asynchronous, with outputs updating as inputs

¹Modern CPUs employ pipelining, allowing them to partially execute several instructions at once, increasing throughput, but they're still inherently sequential.

²In practice it will not be instantaneous, as the signal still need to propagate, which can create glitches due to race conditions, but it is a good mental model.

³There are a lot more up-and-coming languages, but these will not be considered here.

change, and the latter being synchronous, with executions occurring on the rising edge of the clock.

If the purpose of a written HDL program is to be run and executed on an FPGA, it is generally necessary to declare what pins each I/O signal corresponds to, as well as their voltages, current limits and signaling types. Additionally, flashing a program onto an FPGA can also be an involved process, requiring several steps. A small VHDL example consisting of a combinational and a behavioural program is shown in Appendix A, including details on pin assignments and flashing.

3.2.1 The Issue of Floating Point Numbers

The previous section touched on how to use an FPGA in practice, but one big issue of FPGA development is how to represent numbers.

Modern CPUs generally have hardware specifically for doing floating point math, which means they are very efficient at doing so [15]. Contrast this with FPGAs, where many actually do have specialized hardware for integer based mathematics, including the DE25, but none for floating points [2]. And this raises a problem; how do you do mathematics that require more than just integers, when floating points are not available? It is possible to implement floating point arithmetic units for an FPGA, but that would be severely inefficient. Instead, we use fractional binary numbers. Fractional binary numbers are conceptually simple. They're interpreted the same as binary integers, as a sum of powers of two, with the difference being that the exponents can be positive and negative. Normal signed numbers use something called two's complement, and are calculated as:

$$\text{Number} = -2^{N-1}b_{N-1} + \sum_{n=0}^{N-2} 2^n b_n, \quad b_n \in \{0, 1\} \quad (1)$$

Where b_n is the n th digit from the right (starting at zero) [16]. Fixed point numbers are essentially the same, but the exponent does not start at zero, but rather some negative number [15]. For this project, the Q number format was used. It is a form of signed integer, written on the form "QN.M", for example "Q1.15". The 1 represents how many integer bits plus sign bits are used⁴, and the 15 how many fractional bits. In this case, there is no integer bit, meaning only values in the range $-1 \leq x < 1$ can be represented. Written as a binary expansion:

$$\text{Number} = -2^{N-1}b_{N-1} + \sum_{n=-M}^{N-2} 2^n b_n, \quad b_n \in \{0, 1\} \quad (2)$$

Q1.31 will be used extensively in this project, so it's worth examining a bit closer. The smallest step size that can be represented with this format is $\Delta = \frac{1}{2^{31}} \approx 4.65 \cdot 10^{-10}$, which is the resolution, so all numbers between -1 and 1 will be rounded to the nearest whole multiple of Δ .

This project utilizes a lot of multiplication between Q format numbers. Multiplying a QM.N number with a QK.J number results in a Q(M+K).(N+J) number, which has (M+K+N+J) bits. This is fine if this is the desired format of the output, but usually the desired format is different. Consider an example of multiplication between a Q1.15 and Q1.31 number. The inputs are 32 and 16 bit respectively, and so the output is 48 bits in Q2.46 format. If the desired output format is Q1.31, divide by 2 to the power of the difference in fractional, so 2^{15} , and resize the binary number to 32 bits⁵. Care must also be taken when adding Q numbers, because they can, like integers, overflow and underflow. As such, a strategy for dealing with this must be developed, for example by clamping the output, rather than allowing it to wrap.

⁴There are at least two different ways to write Q format, this one is the ARM notation.

⁵When resizing, you have to be extremely careful with truncation vs saturation, especially when losing integer bits. However, multiplication between a Q1.N and Q1.M number stays in the range $-1 \leq x < 1$, so this is not an issue here.

3.2.2 Types and Optimizations

Another issue of FPGA development is resource utilization and optimization, especially regarding storing data in logic units versus RAM. The available types, and when to utilize custom types, is also a necessary discussion, especially in larger, more complex projects.

Base VHDL does not include a lot of types that are useful for mathematical operations, and so a set of packages from the IEEE library are usually used, called `numeric_std` and `std_logic`. With this, the usable mathematical types are signed, unsigned and integer (and other integer related types like `natural`), as well as the broader logic type `std_logic`, which includes several more states than just true and false that are useful for real hardware, especially in simulations [17]. The difference between signed and integer is that signed is simply an array of `std_logic` that is interpreted as a number. While these types are generally enough to do most basic mathematical operations, some algorithms, such as the Fast Fourier Transform, require the use of complex numbers [18]. As such, it can make sense to implement a set of custom complex number types, as opposed to a couple of independent signed numbers to represent the real and imaginary part, which can quickly clutter code, and reduce readability. When defining custom types like this, it is usually also necessary to define and implement the necessary mathematical operations, such as multiplication and addition, that will be performed on the numbers. This project will define a whole range of custom types, but the most important are three custom complex number types, containing two signed integers of 16, 32 and 48 bits respectively. This is done using a record, which is generally called a struct in most other languages.

```
type complex_S16 is record
    re : S16;
    im : S16;
end record;
```

Where S16 is also a custom type:

```
subtype S16 is signed(15 downto 0);
```

Another notable subtype that will be used in the project is the LUT type, used for the look-up tables for sine and cosine:

```
constant LUT_SIZE : integer := 4096;
type lut_t is array(0 to LUT_SIZE-1) of std_logic_vector(15 downto 0);
constant COS_LUT : lut_t := (...);
constant SIN_LUT : lut_t := (...);
```

The above example leads into very a important point. Note that in the type definition of `lut_t`, the array is of `std_logic_vectors`, and not integers. This is because we want the arrays to be stored in the embedded RAM blocks, as there are far more of them, and not just live in the logic units themselves. To accomplish this, these lines can be added below the declaration:

```
attribute romstyle : string;
attribute romstyle of COS_LUT : constant is "M20K";
attribute romstyle of SIN_LUT : constant is "M20K";
```

This lets the compiler know that we would like the cos and sine look-up tables to be stored in "M20K", which is the name of the embedded RAM. This small piece of optimization will become crucial later, as large amounts of data need to be stored for later computation in the chosen FFT implementation. This topic will be covered a lot more in later chapters, where this optimization issue naturally arises. It is important to note that not everything should be stored in RAM, just large arrays of data that need to be accessed at later points in time.

However, just telling the compiler that we would like this data stored in RAM is not always enough to force it to comply, even when using the correct data types, such as `std_logic_vector`. The compiler requires that the RAM always be used synchronously, which can cause a lot of issues if it is not used consistently. This is also an optimization issue that arises in a later chapter, and which forced some odd algorithmic choices.

3.2.3 Valid Signals

Having discussed what an FPGA is and why you would use one, there is one more practical consideration that must also be dealt with, as well as discussing testing and verification in the next section.

In an FPGA, it is not viable to turn the actual signal wires on and off, which raises an important question; how do you deal with inactive signals? For example, if the SDR is not ready to send another symbol yet, how do you disable the input? This issue is a major challenge in synchronous FPGA design, and as such one that will have to be dealt with in this project. One approach is to use valid signals. Any time there is an input, there is an associated signal that says if it's valid or not. The same goes for outputs, each module must ensure that the valid signal is always high when the data is valid, such that the next block in the chain knows it can be used, and it must also be drawn low if there is no data to output, so an invalid signal is not propagated. This project is generally based on a modular approach, where individual modules are responsible for only one thing, and the systems are then built from several modules chained together. Any time there is a connection between modules, a valid signal is used.

3.2.4 Testing and Verification Strategy

When developing for an FPGA, there are a couple of different requirements outside of "does it work as intended" that must be fulfilled. Two that will be discussed here are timing requirements, here meaning if the compiled project can run at 50 MHz, and resource requirements which are set by the specific FPGA. Once these are fulfilled it becomes natural to discuss how to test designs in VHDL, as well as how the individual modules and the overall systems will be tested.

When doing a full compilation in Quartus Prime Pro, it will automatically check if a constraints file exists. Such a file is used to inform the compiler of physical constraints of the system, such as the intended clock, as well as the response of external pins. If one is not provided, Quartus will automatically make certain assumptions, such as the clock being 1GHz. Using the Timing Analyzer tool in Quartus, it is possible to see if the compiled design will fulfill the timing requirements given by the defined clock on the specific chip in the worst case scenarios⁶. The slack for the design is also shown, which means how much time is left before the next clock cycle in the worst case scenario. Using this, it is possible to know how fast the design can be run. For example, if the clock period is 20 ns, with 7 ns of slack, that means the worst-case propagation time is 13ns, leading to a maximum frequency, referred to as F_{max} , of $\frac{1}{13ns} \approx 77MHz$.

It is also necessary to know if the compiled design can fit within the resources of the specific board. For example, the DE25 has 46,800 ALMs, as discussed earlier, which means the design must use less than that. For example, when not being careful to ensure that all large arrays are stored in RAM, rather than in logic blocks, the FFT design can easily use over 3 times as many logic blocks as are available, meaning that the design will simply not work on the DE25. As such, it is critical to implement the optimizations mentioned in section 3.2.2. This is where the requirements for resource utilization and timing closures in section 2.1 arise from.

The last type of verification that is necessary is to verify the actual design. Does it work as intended? There are two ways of doing this; simulation and executing it on an FPGA. In this project, both methods will be done. For individual modules it often makes sense to simulate, whereas for larger systems utilizing multiple modules it is advantageous to run the code on the FPGA, and analyze the result.

To simulate an HDL design in Quartus, a testbench is utilized, which is essentially an entity with no inputs or outputs [19]. In the testbench the Device Under Test (DUT) is instantiated, and a set test signals are applied to it sequentially. The output can be used to throw errors if it doesn't match the expected, or it can be written to a file for analysis using an external tool, as will be done in this project using Julia. Once the testbench has been implemented, to actually execute it, a tool called Questa, which comes bundled with Quartus Prime Pro, is utilized. The

⁶It simulates the design in various conditions, including at 0C and 100C

structure of the testbenches used in this project will be discussed in later chapters, when the modules are tested.

In summary, verification in this project includes three parts; Timing verification using the Timing Analyzer tool, resource verification using the compilation reports, and functionality verification using testbenches and external tools to ensure that the code works as intended.

3.3 The DAC

To actually generate an analog signal that can be modulated onto an RF carrier and sent to an antenna, a Digital-to-Analog-Converter (DAC) is needed. For this project, the high-speed THDB-ADA from Terasic was used. It is a 14-bit resolution dual DAC with data rates up to 125 MSPS, and a built in 100MHz oscillator. It also contains a dual 14-bit resolution ADC with data rates up to 65 MSPS [20], but this functionality will not be utilized in this project.

This DAC was chosen for several reasons:

- A data rate of at least 50MHz is required, as that is the desired frequency of the FPGA.
- Native support with Terasic (Altera) Development and Education Boards. There exists two version of the THDB-ADA, one with the proprietary HSMC connector, which the Altera DE25 utilizes, and one with 2x40 standard GPIO pins, which other older Altera Development and Education boards use..
- It has a pin that is used for an external clock, enabling the use of the FPGA oscillator to control the DAC, meaning there is no issues with synchronicity between the two.
- High resolution. The 14-bits of the DAC lead to $2^{14} = 16384$ levels, which allows for the use of more complicated digital modulations schemes, such as high levels of QAM.
- The DAC is 2V peak-to-peak, so natively ranging from +1V to -1V. Having the output centered around zero volts simplifies the analog chain.

One issue that presented itself regarding the DAC is listed in the features in the manual "7. DA and AD converters do not support DC signaling" [20]. This means the DAC does not allow low-frequencies to pass. How the lack of DC signalling is overcome will be discussed in later sections, but firstly it is important to actually characterize what exactly that phrase means. It can be seen in the schematic that the outputs of the DAC are connected via transformers [21], and so a hypothesis was made: "The outputs of the DAC behave as a high-pass filter."

The hypothesis was tested by doing a small experiment; generate a pure cosine wave on the FPGA, transmit it through the DAC, sample the output using an Analog Discovery 2 to calculate an RMS voltage, and repeat for many different frequencies. It is not realistic to actually calculate sine and cosine on an FPGA, and so a precomputed look-up table was used. The look-up table consists of 4096 samples ($N = 4096$), all stored as 16-bit Q1.15 values. The LUT is generated as such:

$$\cos[n] = \text{round}(\cos(2\pi n/N) \cdot (2^{15} - 1)) \quad (3)$$

A sampling frequency of 50MHz leads to a frequency resolution of $50\text{MHz}/4096 \approx 12\text{kHz}$.

To test the hypothesis, a DAC driver was written in VHDL. A general overview of the driver is shown below:

- The DAC allows dual-channel and interleaved mode. For the rest of this project, dual-channel will be used, as it severely simplifies the analog chain. In this experiment, we will just be testing channel A, but they should be identical.
- The DAC has both a clock input, and a write strobe input. From experimentation it is clear that the clock input must match the FPGA clock, and the write strobe must be an inverted version of the clock.

- The DAC is binary-offset, meaning 0 corresponds to -1V, and 16383 corresponds to +1V⁷, with 0V being 8192. This means we need to add 8192 to any signed number to convert it to our binary-offset format.
- The 10 switches on the DE25 will be used to set the step size, with $F_{out} = F_s/\text{size}(LUT) \cdot \text{step} \approx 12\text{kHz} \cdot \text{step}$.

The measurements are shown in Table 3. As can be seen in the table, the output remains approximately flat above around 1MHz. If it can be modeled as a first-order highpass filter, then all that is needed to characterize it is the gain, which is just the value in the passband (which is around 0.72V), and the critical frequency, defined as the frequency at which there is -3dB, or $\frac{1}{\sqrt{2}}$ gain. That means an RMS voltage of $\frac{0.72\text{V}}{\sqrt{2}} \approx 0.51\text{V}$. This happens between 122kHz and 109kHz, so estimating a critical frequency of 115kHz leads to this fit:

$$|H(f)| = G \frac{f}{\sqrt{f^2 + f_c^2}} = 0.72\text{V} \cdot \frac{f}{\sqrt{f^2 + (115\text{kHz})^2}} \quad (4)$$

The measurements and the fit plotted together, both in linear units and dB, and is shown in Figure 3. As can be seen, this is a very good fit, and, using the coefficient of determination to judge the goodness of fit yields $R^2 = 0.997$. Having now verified that the DAC behaves as a first-order highpass filter with critical frequency of 115kHz, it is now possible to use the DAC with the confidence that it won't meaningfully contribute to errors (outside of quantization noise), so long as we ensure the signal stays above a certain threshold. Using $f_{min} = 1\text{MHz}$ gives a gain of -0.06dB, which is very acceptable.

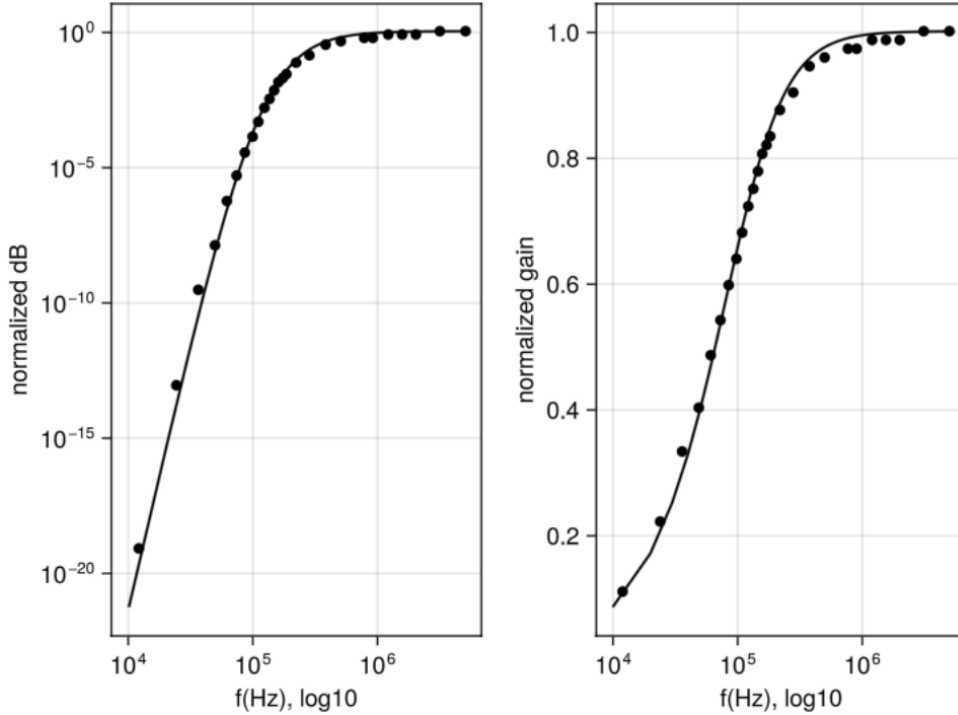


Figure 3: DAC measurements vs first-order highpass fit

4 Digital Design and Implementations

This chapter presents the digital baseband design and implementations, building on the design foundations established in Chapter 3, starting with a simple single-carrier architecture using

⁷Technically 16383 corresponds to 0.99988V

Frequency	RMS Voltage	Frequency	RMS Voltage
5.00MHz	0.72V	158kHz	0.58V
3.11MHz	0.72V	146kHz	0.56V
2.00MHz	0.71V	134kHz	0.54V
1.55MHz	0.71V	122kHz	0.52V
1.20MHz	0.71V	109kHz	0.49V
903kHz	0.70V	98kHz	0.46V
769kHz	0.70V	85kHz	0.43V
500kHz	0.69V	73kHz	0.39V
378kHz	0.68V	61kHz	0.35V
280kHz	0.65V	49kHz	0.29V
219kHz	0.63V	36kHz	0.24V
183kHz	0.60V	24kHz	0.16V
171kHz	0.59V	12kHz	0.08V

Table 3: Frequency vs RMS voltage

pulse shaping, and progresses to more complicated multiplexing schemes, specifically OFDM and SC-FDMA, which are generally more suitable for wireless channels.

To meaningfully compare the different implementations, the same test data set will be transmitted through each, then sent through the DAC, sampled by an Analog Discovery 2, and decoded in the Julia programming language. The test data set consists of 1024 bytes of fixed lorem ipsum data, represented as ASCII. While this has its own challenges, specifically the first bit of every character being zero, leading to certain empty spots in the constellation diagrams, it provides a deterministic and readable payload that simplifies verification.

The chapter starts by going over the general symbol shaping scheme used, then moves on to the single-carrier implementation, which uses pulse shaping and frequency shifting. It then goes over the FPGA implementation of the FFT/IFFT core used for OFDM and SC-FDMA, followed by a discussion of these schemes and their testing and verification. For testing and verification, the measures of goodness for the schemes are resource utilization and timing requirements, complexity, error rate, Error Vector Magnitude (EVM), and Peak-to-Average-Power Ratio (PAPR).

4.1 Symbol Shaping

Modern digital communication systems require a mapping from binary data to a set of symbols that can actually be transmitted. There exist many such modulation schemes, most of which utilize a complex symbol stream, as it allows for full utilization of the bandwidth compared to a real signal. This is due to the symmetric nature of the spectrum of real-valued signals, which inherently require double the bandwidth. There are ways to avoid this bandwidth increase, for example by utilizing single-sideband modulation, which suppresses one of the symmetric halves, but it is more complicated than simply using a complex signal.

The different complex modulation schemes all have their trade-offs between data-rates, spectral efficiency and bit error rate, and which one is best depends on the specific signal chain and channel characteristics. For this reason, the choice of modulation scheme is an important design consideration in any modern communication system.

4.1.1 Comparing Candidate Modulation Schemes

In general, the information of a signal is embedded onto a carrier via either the amplitude, phase or frequency of the carrier. Only amplitude and phase modulations will be considered. When utilizing complex signals, the carrier will have the general form of [22]:

$$s(t) = \text{Re}\{x(t)e^{2i\pi(f_c t + \phi)}\} \quad (5)$$

Where f_c is the carrier frequency, $x(t) = I(t) + iQ(t)$, where I is the In-phase, or real, component, and Q is the Quadrature, or imaginary, component of the signal. Since any complex number can be written in polar coordinates, the above equation can be rewritten to more explicitly show where the amplitude and phase modulation parts each come in:

$$s(t) = \text{Re}\{|x(t)|e^{i\angle x(t)}e^{2i\pi(f_c t + \phi)}\} = \text{Re}\{|x(t)|e^{2i\pi(f_c t + \phi + \angle x(t))}\} \quad (6)$$

When using amplitude or phase modulation, it is possible to visually represent the modulation schemes with a constellation diagram, where each dot represents a complex symbol [23]. The constellation diagram for 16QAM, 16PSK and 4PAM are shown in Figure 4. In an Additive

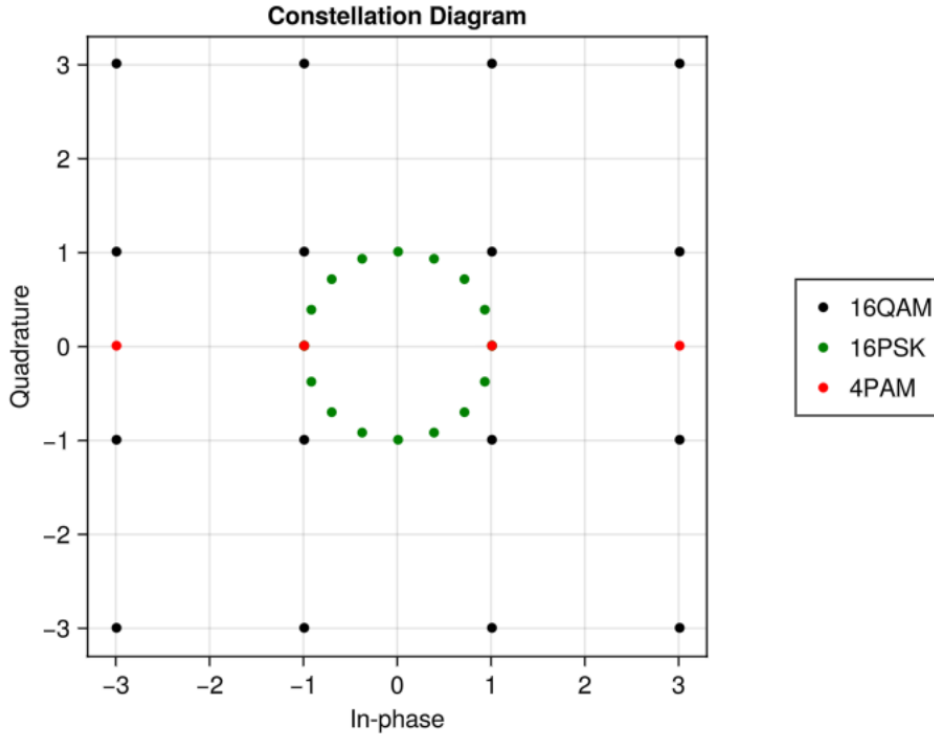


Figure 4: Constellation Diagram of 16QAM vs 16PSK vs 4PAM

White Gaussian Noise (AWGN) channel, the probability of decoding a symbol wrong is related to the distance between a symbol and its nearest neighbors. For coherent modulation in an AWGN channel, the probability of symbol error for various modulations can be seen in Table 4, where $Q(\cdot)$ is the tail distribution function for the normal distribution, meaning larger inputs corresponds to smaller outputs. As can be seen in Table 4, MQAM performs very well for

Modulation	Probability of error
MPSK	$2Q(\sqrt{2\gamma} \sin(\pi/M))$
MPAM	$2^{\frac{(M-1)}{M}} \cdot Q\left(\sqrt{\frac{6\gamma}{M^2-1}}\right)$
MQAM	$4Q\left(\sqrt{\frac{3\gamma}{M-1}}\right)$

Table 4: Approximate symbol error rate for different modulations. Source: [24]

larger M as it utilizes more of the available constellation space compared to both MPSK and MPAM. For this reason, many modern communication systems, especially those focused on high throughput, utilize MQAM, such as WiFi, as specified by the IEEE 802.11 standard [4].⁸ As such, MQAM will be used in this project going forward.

⁸Many of the standards also support BPSK and QPSK, but they all support MQAM, from 64QAM in 802.11a all the way up to 4096QAM in 802.11be, utilizing adaptive modulation depending on the channel.

4.1.2 Implementation of MQAM

The modulation used in this project is square based MQAM, meaning M must be a power of 4, specifically implemented for $M_{\text{QAM}} \in \{16, 64, 256, 1024, 4096\}$, allowing a wide range of constellation densities depending on the channel conditions. The bitstream is converted to two components; in-phase and quadrature. These are grouped into 2 groups of $\log_2(\sqrt{M_{\text{QAM}}})$ bits each, which are then interpreted as gray code to improve robustness against symbol errors by ensuring that neighboring constellation points only differ by one digit. The gray code is then converted to binary, and used to index the look-up table associated with the chosen value of M. Each look-up table consists of the $\sqrt{M_{\text{QAM}}}$ amplitude levels corresponding to the constellation points along one axis, stored as Q2.22 values normalized by the average energy per symbol, which is given by $E_{\text{avg}} = \frac{2}{3}(M_{\text{QAM}} - 1)$ for square MQAM, so each value is scaled down by the square root of the average energy, and scaled to Q2.22 format.

$$\text{Levels} = \text{round} \left(\left\{ \frac{L}{\sqrt{\frac{2}{3}(M_{\text{QAM}} - 1)}} \right\} \cdot (2^{22} - 1) \right), \quad (7)$$

Where L ranges from $-(\sqrt{M_{\text{QAM}}} - 1)$ to $(\sqrt{M_{\text{QAM}}} - 1)$ with a step size of two. These values are then stored as 24 bit signed numbers in quartus in their respective look-up tables. Q2.22 was utilized as the largest values of the normalized levels exceed 1, and thus an integer bit is necessary. 24 bits were used to store the values in order to retain sufficient precision over 16 bits, while both maintaining efficient FPGA resource usage, and avoiding the risk of overflow and underflow when interfacing with the FFT core, as it has an internal data width of 32 bit, leaving sufficient headroom.

4.2 Single Carrier

The first version of the SDR transmitter is a simple single-carrier signal chain, utilizing a Root-Raised Cosine (RRC) Filter for pulse shaping, 16QAM for the symbol shaping, as well as a configurable frequency shifter to manage the high-pass nature of the DAC. Most of the parameters of the SDR are configurable, but the instance used in this project is based on $R_s = 5$ MSym/s, being upsampled to 50MHz using a polyphase implementation of the RRC filter. This yields a data rate of

$$5 \cdot \frac{\text{MSymbols}}{\text{s}} \cdot 4 \cdot \frac{\text{bit}}{\text{symbol}} = 20 \cdot \frac{\text{MBit}}{\text{s}} \quad (8)$$

As will be shown below, this leads to about 6.5MHz bandwidth, and so a shift of about 5MHz was utilized, such that the minimum occupied frequency is about $5\text{MHz} - 6.5\text{MHz}/2 = 1.75\text{MHz}$, which is above the minimum safe frequency of 1MHz as determined in chapter 3.3.

4.2.1 Pulse Shaping

In order to minimize Intersymbol Interference (ISI), a pulse shaping filter is generally utilized [25]. In this project, the Root-Raised Cosine (RRC) filter was chosen as it is a very commonly used one, since it easily lends itself to matched filtering, as the combined filter response of the transmitter and receiver forms a full Raised-Cosine filter.

An RRC filter has one tunable parameter, β , referred to as the roll-off factor. The bandwidth associated with a root-raised cosine filter is given by

$$B_{\text{RRC}} = (1 + \beta)R_s \quad (9)$$

With a $\beta = 0.35$, this gives an occupied bandwidth of:

$$B_{\text{RRC}} = (1 + 0.35) \cdot 5 \text{ MHz} = 6.75 \text{ MHz} \quad (10)$$

Note that the above equation is for an ideal IIR RRC filter, but in this project it has been truncated to a FIR filter, which can lead to a slightly larger bandwidth.

The pulse shaper is also used to upsample the symbol stream, from the base 5MSym/s, to the 50MHz rate of the FPGA, which results in an upsampling factor of 10. To do this, 9 zeros are inserted after each symbol, and then run this through the RRC filter. To accomplish this in a resource efficient manner, a technique called polyphase filtering was utilized, which allows skipping all computations involving the zeros, vastly decreasing the necessary FPGA resources [26].

In this project, a span of 6 symbols, so 3 on either side of the current symbol, was used. The RRC filter equations were sampled from -3 to 3 at intervals of 1/10 (samples per symbol), then converted to Q1.15. Additionally, the filter was padded with zeros on the right to make it fill 70 samples, such that a full 7 upsampled symbols fit. It was also normalized to the total energy of the filter. The Q1.15 filter coefficients can be seen in Figure 5. The last 9 filter coefficients are the padded zeros, such that the length of the filter coefficients matches the 7 upscaled samples. Normally when you filter in time, you would use a convolution between your signal and your

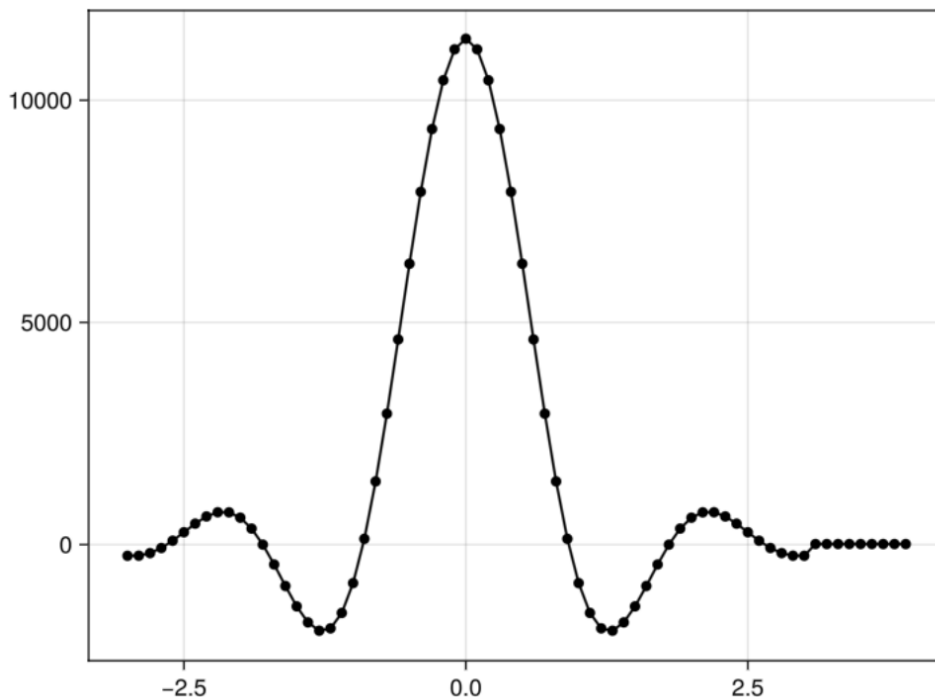


Figure 5: The used RRC filter

filter, and while this is doable, it is very resource inefficient, as mentioned above, due to most of the samples being zeros. Instead, a technique called polyphase filtering can be utilized, which allows for the bypassing of the multiplications with zeros from the upsampling. The algorithm goes like this:

Keep a running phase counter ϕ ranging from zero to the upscaling coefficient minus 1, so in this case 9, as well as an array of the current low-frequency symbols (in this case the 7 symbols considered). If the counter is zero, shift the symbols over once, and fill the now empty slot with the next symbol. On every clock the output of the filter is calculated as:

$$Y = \sum_{k=0}^{S-1} \text{RRC}(\phi + 10k) \cdot \text{Symbols}(k) \quad (11)$$

Where S is the total number of symbols in the filter (7 in this project), and ϕ is the phase counter. This cuts the number of computations down by a factor of 10, which is a very necessary optimization on an FPGA.

4.2.2 Frequency Shifting

To circumvent the high-pass nature of the DAC, it is necessary to move the active frequency region away from zero. As shown in the previous chapter, the occupied bandwidth is about 6.75MHz. To ensure that the signal remains outside the unsafe region, a frequency shift of 5MHz was implemented, placing the passband around 1.625MHz to 8.375MHz, which is comfortably above the requirement of 1MHz.

To accomplish the single-sided frequency shift as desired, we multiply by $e^{2\pi i f_c t}$, where f_c is the frequency to shift by. However, as the in-phase and quadrature components are considered separately, the complex multiplication must be separated into two real multiplications [27]:

$$\begin{aligned} x(t) &= s(t)e^{2\pi i f_c t} \\ &= (I + iQ)(\cos(2\pi f_c t) + i\sin(2\pi f_c t)) \\ &= (I \cos(2\pi f_c t) - Q \sin(2\pi f_c t)) + i(I \sin(2\pi f_c t) + Q \cos(2\pi f_c t)) \end{aligned} \quad (12)$$

For sine and cosine, a lookup table containing 4096 Q1.15 samples of sine and cosine values is utilized, since it allows for a configurable frequency shift. Internally, the frequency is calculated from a step-value which determines the stride through the LUT, and thus determines the frequency:

$$f_{\text{IF}} = \text{step} \cdot \frac{F_s}{N_{\text{LUT}}} = 410 \frac{50\text{MHz}}{4096} \approx 5.005\text{MHz} \quad (13)$$

As such, a step size of 410 was utilized in this implementation, but it is easily changeable.

4.2.3 Testing and Verification

With every component of the single-carrier digital transmission path accounted for, it must now be tested and verified to ensure it operates correctly and meets the resource and timing constraints of the FPGA.

Due to the modular design of each component, it is necessary to create a central controller that routes data between the modules, while ensuring the timing of every part matches expectations. In the case of the single-carrier SDR, the controller is very simple. On every 10 clock cycles the controller asserts the valid in signal of the data source, which corresponds to the desired symbol rate. On the next clock cycle, the source will then propagate the data and valid signal to the next block, and so on until the output, as shown on the block diagram in Figure 6.



Figure 6: Block diagram of SC SDR

The output of the DAC is then sampled by the Analog Discovery 2 (AD2), with a sample rate of 50MHz to match the DAC output. Additionally, a trigger output has been added from the FPGA to the AD2, allowing the decoder to know when the start of a symbol was, decreasing the complexity of the decoder, which has been implemented in Julia. The first 1024 samples after the trigger are shown on Figure 7.

The decoding is conceptually simple, as it is done digitally. Firstly, the DC offset between the DAC and AD2 is subtracted off the data. This DC is calculated as the average value of the signal prior to the first symbol. Then the signal gets shifted back to baseband by multiplication with a complex exponential with the negative carrier frequency. There is the problem of phase offset of the frequency shifter, but since the initial phase in the FPGA is zero, and the trigger occurs at the start of the first symbol, the phase should be zero. However, in general, it would be necessary to implement some form of phase alignment. Next, the matched RRC filter, which is just the original RRC filter in reverse, is applied to the samples, starting at the first sample + the group delay of the RRC filter, which is about half of its width, so 35 samples. After the matched filter the signal is scaled down by the average energy of the samples, and then scaled

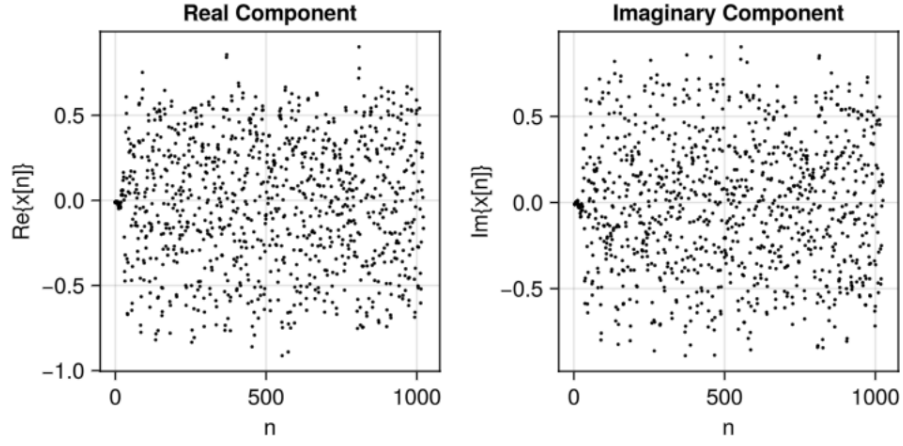


Figure 7: First 1024 samples of single-carrier capture

up by the average energy of the MQAM symbols, as to achieve unity gain. This allows direct comparison between the decoded samples and the decision levels based on the MQAM symbols.

There is however the issue of phase synchronization and downsampling. The symbol rate is 5MSym/s, whereas the output has been upsampled to 50MHz, and so it is necessary to decimate the signal in the decoder. To accomplish this, every phase offset, 0 to 9, was tested, and the one with the minimum MSE is picked.

However, one challenge that can arise in the decoding is a rotation of the MQAM symbols that occur during transmission. Since there is no channel estimation in this single-carrier chain, the rotation can be estimated by trying several phase off-sets. Combining this with the decimation phase search, the decoder chooses the decimation phase and rotation offset that yields the minimum MSE.

The decoded output in ASCII is:

”Lorem ipsum dolor sit amet, consectetur adipiscing elit. Maecenas dolor risus, luctus ut lorem non, pharetra blandit velit. Duis consequat t”

Which matches perfectly with the first 140 bytes of the transmitted data. It managed to transmit every character correctly, resulting in an error rate of zero for this payload test. The constellation of the decoded 16QAM symbols can be seen in Figure 8. As can be seen in the constellation diagram of the decoded measurements, all the points belong to well defined clusters, though there are some significant deviations from the ideal levels, however none of these are large enough to cause any erroneous decodings, which is consistent with the measured error rate of zero. The deviations from the ideal levels can be expressed quantitatively using the Error Vector Magnitude, EVM, which is defined as the RMS of the difference between the received signal and the ideal signal, normalized by the energy of the ideal signal [28]:

$$\text{EVM}(\text{dB}) = 20 \log_{10} \left(\sqrt{\frac{\sum_{n=0}^{N-1} |X_n - \hat{X}_n|^2}{\sum_{n=0}^{N-1} |X_n|^2}} \right) = 0.065 = -23.8 \text{ dB} \quad (14)$$

For later comparison with OFDM and SC-FDMA, the Peak-to-Average-Power-Ratio (PAPR) for each component and the full complex signal of the single-carrier transmission is computed:

$$\begin{aligned} \text{PAPR}_R &= \frac{\max(\mathcal{R}\{x[n]\}^2)}{\mathbb{E}[\mathcal{R}\{x[n]\}^2]} = 6.02 \approx 7.80 \text{ dB} \\ \text{PAPR}_I &= \frac{\max(\mathcal{I}\{x[n]\}^2)}{\mathbb{E}[\mathcal{I}\{x[n]\}^2]} = 5.70 \approx 7.56 \text{ dB} \\ \text{PAPR} &= \frac{\max(|x[n]|^2)}{\mathbb{E}[|x[n]|^2]} = 3.27 \approx 5.15 \text{ dB} \end{aligned} \quad (15)$$

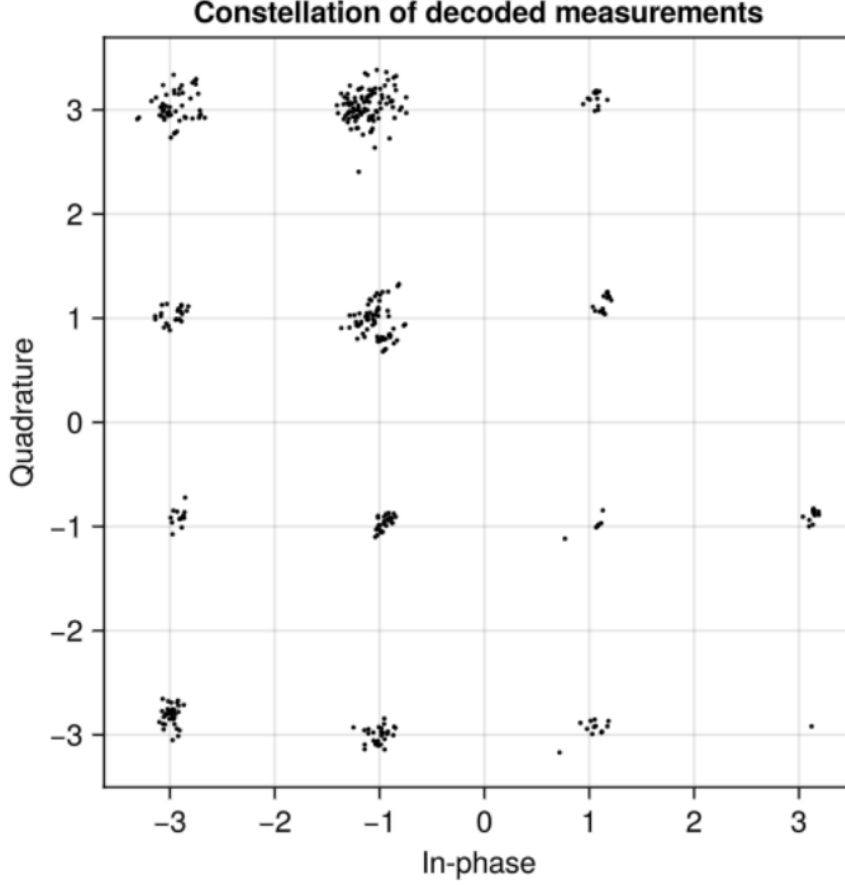


Figure 8: Constellation of the Single Carrier Decoded 16QAM

As for the resource and timing requirements, these must be read out from Quartus. Once a full compilation is complete, Quartus shows the resource utilization of the code on the specified FPGA. In the case of the DE25 and Single-Carrier digital transmission code, the final resource utilization comes out to:

- 1,475 ALMs out of a total of 46,800 (3%)
- 0 RAM blocks
- 0 Memory bits
- 27 DSP blocks out of a total of 188 (14%)

Which fits very comfortably in the FPGA. As for the timing requirements, Timing Analyzer, the Quartus tool for timing requirements, reports a worst-case slack of 8.249 ns, and with a clock period of 20 ns, this means the worst-case timing path is about 11.75 ns, giving an $F_{max} = \frac{1}{11.75ns} \approx 85\text{MHz}$, which is comfortably above the requirement of 50 MHz. As can be seen, the single-carrier SDR is not heavy on the resource utilization.

However, it is generally not viable for high throughput wireless communication due to multi-path components. For normal office buildings and housing, the general worst-case delay spread is around 100 ns [29], and, using the rule of thumb that a symbol period 10 times that of the worst-case delay spread, yields a maximum bandwidth of

$$B < \frac{1}{10 \cdot \sigma_{T_m}} = \frac{1}{10 \cdot 100ns} = 1 \text{ MHz} \quad (16)$$

Which is significantly less than the bandwidth of the Single-Carrier SDR, which was around 6.75MHz. This means it experiences a frequency-selective channel, which must be dealt with to avoid severe intersymbol interference, such as by equalization [30].

The spectral efficiency is given by the data rate divided by the occupied bandwidth:

$$\frac{R_{s_{16QAM}}}{B} \approx \frac{20 \text{ Mbit/s}}{6.75 \text{ MHz}} \approx 2.96 \text{ bit/s/Hz} \quad (17)$$

4.3 Frequency Multiplexing - OFDM & SC-FDMA

As shown above, a single-carrier approach in a high-bandwidth wireless communication system becomes challenging because multipath components can cause significant ISI. Instead, we can turn to multicarrier systems that allow us to split the spectrum up into several smaller bins, where each bin has a much smaller bandwidth. The central idea is to divide the data stream into a set of lower-rate parallel symbols, each modulated onto their own carrier;

$$x = \sum_{n=0}^{N-1} s[n]e^{2\pi i f_n t} \quad (18)$$

Equation (18) can be rewritten slightly to turn it into the Inverse Discrete Fourier Transform:

$$x[k] = \frac{1}{N} \sum_{n=0}^{N-1} S[n]e^{2\pi i \frac{k}{N}n} \quad (19)$$

In modern systems the Fast Fourier Transform (FFT) and its inverse (IFFT) are almost always used, as it takes the computation from $O(N^2)$ to $O(N \log N)$.

In this chapter, two such frequency multiplexed systems will be discussed and implemented, Orthogonal Frequency Division Multiplexing, OFDM, and Single-Carrier Frequency Division Multiple Access, SC-FDMA.

As stated, modern systems almost exclusively utilize FFT/IFFT, and as such an efficient FPGA FFT/IFFT implementation is required OFDM and SC-FDMA can be utilized.

4.3.1 FFT Implementation on the FPGA

For a real-time communication system utilizing multi-carrier modulation via FFT or IFFT, the implementation must be streaming, meaning one input and output per cycle. A large part of this project was focused on implementing such a streaming FFT/IFFT core which fulfills the requirements on resource utilization and timing, which involves several complicated optimizations, such as RAM inference and queues. The entire module is developed from first principles, without the use of external libraries⁹. The specific implementation is:

- Real-time streaming, with a delay,
- Able to do both FFT and IFFT with a mode-select signal,
- Radix-2, allowing N up to 4096,
- Agnostic to other modules,
- Decimation-in-input, direct ordered output,
- Resource efficient, allowing its use in larger systems.

It performs an N-point FFT/IFFT, and will pause if a full N samples is not supplied to it. As such, it is important that any users of the module ensure to always provide a full set of samples, or toggle the reset before a new transform is started.

⁹Aside from the `std_logic` and `numeric_std` packages, which are essentially necessary for any VHDL development[17]

The implemented FFT module is inspired by an algorithm called Single-Path Delay Feedback (SDF) FFT, which is a pipelined algorithm, lending itself naturally to a streaming implementation [31]. The idea is perform a decimation-in-input, meaning storing the input stream in bit-reversed order and then perform $\log_2 N$ sequential stages of an operation called a butterfly;

$$Y_0 = A + WB \quad (20)$$

$$Y_1 = A - WB \quad (21)$$

Where A and B are the inputs, and W is called the twiddle factor, which is a complex exponential dependent on the stage and sample.

Figure 9 shows an example of decimation-in-input for $N = 8$. While this ordering looks complicated, it is easy to implement in practice. Count the inputs as they stream in, and store them in the index corresponding to the bit-reversed count, padded with zeros to $\log N$ bits. Then, once the full N samples have been stored, the now-full bit-reversed array is streamed into the FFT/IFFT core in natural order. This decimation causes the outputs to be in natural order, which means there is no need to store them before they're output.

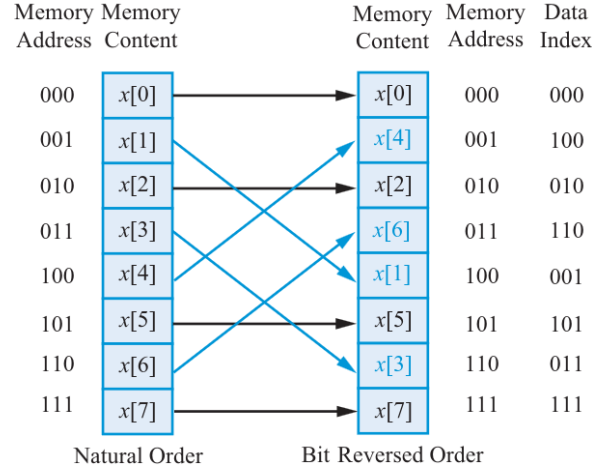


Figure 9: Example of bit-reversed order. Source: [18], p. 444

The challenge arises from aligning the two inputs. For a given stage $0 \leq s < \log N$, with $M = 2^{s+1}$, streaming input $X[k]$, and introducing a delay variable $d = M/2 = 2^s$, Equation (21) can be rewritten as:

$$Y_0 = X[k - d] + WX[k] \quad (22)$$

$$Y_1 = X[k - d] - WX[k] \quad (23)$$

To achieve this alignment in practice, a delay FIFO queue is implemented and utilized. For stage s , and introducing a counter i ranging from 1 to M , the algorithm goes as such:

- When $i < d$, store the current input in the FIFO queue.
- When $i \geq d$, perform a butterfly between the current value in the queue and the current input, and output Y_0 while storing Y_1 in another FIFO queue.
- When $i < d$, output the current value in the Y_1 queue.

The output of each intermediary stage is wired directly up to the input of the next, including both valid and data signals, whereas the output of the last stage is wired up as the output of the entire module. This is the benefit of a decimation-in-input implementation, it allows the output to be directly output rather than first stored in bit-reversed order.

An example of the algorithm using $N = 4$, $s = 1$, so $M = N = 4$ and $d = 2$ can be seen in Table 5, which shows the current input and output, as well as the state of the input and output

queues. Note that, while the computation and outputting only takes 4 clock cycles, they are offset from each other, meaning there must be logic in place to handle discontinuous streaming, by for example ensuring that the output queue flushes even though no new inputs are given. For large M this can lead to complications, as a new computation may begin when the output queue is only part-way flushed. As such, it is necessary to have a robust queue implementation.

i	Current input	Current output	Delay queue	Output queue
0	$X[0]$	none	[-----,-----]	[-----,-----]
1	$X[1]$	none	[$X[0]$,-----]	[-----,-----]
2	$X[2]$	$Y_0[0]$	[$X[0]$, $X[1]$]	[$Y_1[0]$,-----]
3	$X[3]$	$Y_0[1]$	[-----, $X[1]$]	[$Y_1[0]$, $Y_1[1]$]
4	none	$Y_1[0]$	[-----,-----]	[$Y_1[0]$, $Y_1[0]$]
5	none	$Y_1[1]$	[-----,-----]	[-----, $Y_1[0]$]

Table 5: FFT stage with $N = 4$, $s = 1$, $M = 4$, $d = 2$

To actually perform the butterfly, the twiddle factor, W , must also be calculated:

$$W = e^{2\pi i \frac{k}{M}} = \cos\left(2\pi \frac{k}{M}\right) + i \sin\left(2\pi \frac{k}{M}\right) \quad (24)$$

Where k is the index within the current butterfly span. In practice the twiddle factor is calculated from a sine and cosine look-up table, which in this project consist of 4096 Q1.15 samples each, which is where the limitation of $N \leq 4096$ comes from.

The above twiddle factor is for the FFT, and to calculate the IFFT, the sign in the exponential must be flipped. Additionally, when performing the inverse transform, the output must be divided by N . In practice, to preserve precision due to quantization, it is done by dividing the output of each stage by 2, or equivalent shifting right once, since there is $\log N$ stages, giving a total scaling for $2^{-\log N} = \frac{1}{N}$. Note that the scaling only occurs for the inverse transform, and not for the transform.

The internal data width is 32 bits, which gives plenty of precision for the growing nature of the FFT. However, it is still important to check for overflow and underflow following every computation, and to ensure saturation rather than truncation happens in that case. To this end, a custom multiplication function between complex 32 bit and complex 16 bit numbers with saturation was implemented, as well as custom addition and subtraction functions between two complex 32 bit numbers with saturation were implemented.

Another important implementation detail to discuss is on types and optimizations, as mentioned in chapter 3.2.2. As the delay of the later stages grows, so does the size of the two required arrays, the delay and output arrays, and for large N it becomes infeasible to store these in logic, requiring the use of RAM. To ensure Quartus infers them as RAM blocks, it is necessary to change the types from the custom 32 bit complex numbers to pure `std_logic_vectors`, and as to accomplish this the real and complex parts, interpreted as bits, were concatenated into a 64 bit word.

As mentioned above, the SDF FFT algorithm relies on decimation-in-input, and to ensure that the FFT/IFFT core is fully streaming, this decimation must happen after the user input. For this purpose, a set of ping-pong style RAM blocks were implemented, where they swap which is writing to memory and which is reading from memory. The exact implementation details of the ping-pong RAM will not be discussed here, as it is relatively complicated. The code can be seen in the appendix.

Since it is the input that gets decimated, the output is ordered correctly, and can be streamed out of the module.

The full implementation of the FFT core for $N = 4096$ has the following resource utilization:

- 6,183/46,800 ALMs (13%)

- 1,048,320 / 7,331,840 Memory Bits (14%)
- 86 / 358 RAM blocks (24%)
- 69 / 188 DSP blocks (37%)
- 5.272ns slack, leading to $F_{max} = 67.9\text{MHz}$

The module thus fits very well within the physical constraints, and fulfills the resource and timing requirements well. As will be discussed in the following sections, both OFDM and SC-FDMA utilize $N = 1024$, and as such the FFT implementation will fit comfortably.

Before the core can be used in any further modules, it must first have its functionality verified. To accomplish this, 4 testbenches were created, the FFT of an impulse, the FFT of a sum of 3 pure tones, the IFFT of an impulse, and an FFT-IFFT round-trip test of a sum of 3 pure tones, all utilizing $N = 1024$. The results of these four tests are shown on Figure 10.

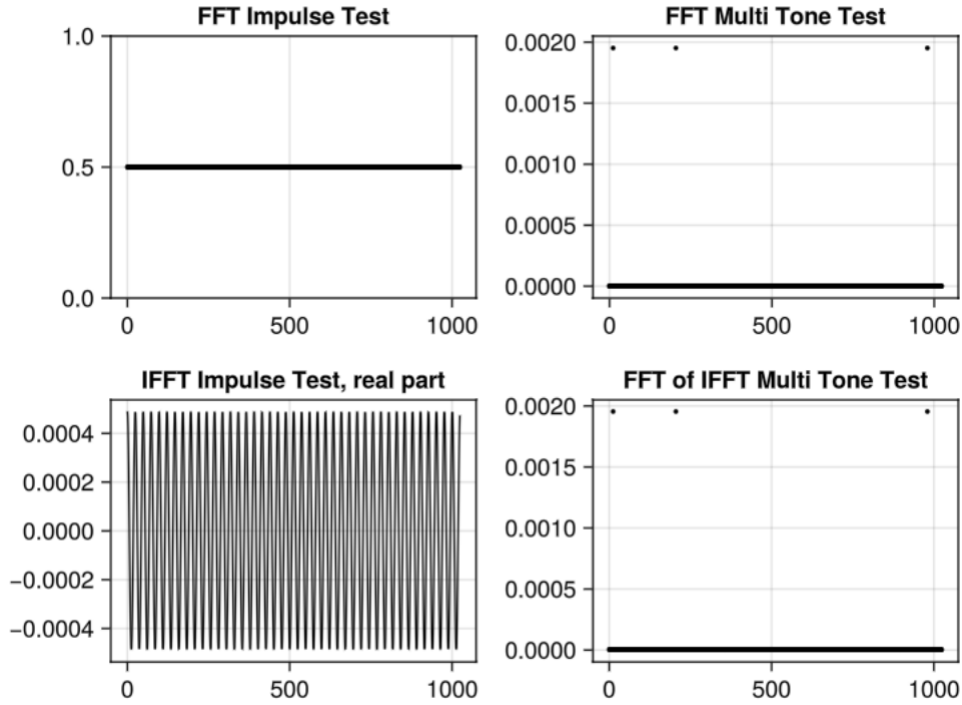


Figure 10: Series of tests of the FFT/IFFT module

In summary, the FFT core trades latency for throughput: after the pipeline has been filled, it accepts one sample per clock and produces one sample per clock, making it suitable for real-time OFDM and SC-FDMA generation at the 50 MHz system rate.

4.3.2 Orthogonal Frequency-Division Multiplexing

One of the most widely used multi-carrier schemes is OFDM, Orthogonal Frequency Division Multiplexing [32]. The idea is relatively simple; Divide the data stream into lower-rate symbols consisting of M data symbols, mapped into N frequency bins, and inverse Fourier transformed. The subcarrier spacing is:

$$B = \frac{F_s}{N} = \frac{50\text{MHz}}{1024} \approx 48.8\text{kHz} \quad (25)$$

Since each bin is orthogonal, this is the bandwidth we must compare to the coherence bandwidth from delay spread, which was calculated to be around 1MHz for normal indoor environments using Equation (16), and so this bin width will work well for this project.

There is still the concern of ISI between OFDM symbols, and to solve this you insert a cyclic prefix, i.e. inserting the last few samples of the OFDM symbol ahead of it. The length of

the cyclic prefix must be greater than the maximum delay spread, as discussed earlier is about 100 ns. The use of a cyclic prefix of sufficient length turns the linear convolution between the OFDM symbol and the channel impulse response into a circular convolution, as the symbol looks periodic within the memory of the channel. Using a cyclic prefix length of 8 gives: $T_{CP} = 8T_s = \frac{8}{F_s} = 160ns$, which just about exceeds the maximum delay spread. Further testing would be necessary to determine if a cyclic prefix length of 8 is enough, but it is sufficient for this project.

OFDM also helps solve the issue of the high-pass nature of the DAC, as it naturally allows you to decide which frequencies contain data, and so by choosing the lowest occupied frequency around 1MHz, the issue of the DAC is circumvented.

However, OFDM is not without issues. One of the biggest ones is its high Peak-to-Average-Power-Ratio (PAPR). This means, given a finite resolution DAC, that it is necessary to introduce extra back-off to avoid clipping, reducing the effective resolution of the DAC compared to other modulations with a lower PAPR.

One of the most important design parameters of OFDM is the bin mapping. For simplicity, this project utilizes a continuous mapping, with pilot subcarriers interspersed for channel estimation and synchronization purposes. The bin mapping logic is:

Insert an edge pilot before the active region, decided by k_0 , as well as after the active region, decided by $k_0 + M + N_{\text{pilot}}$ where M is the number of data bins and N_{pilot} is the amount of internal pilots. Every other bin is a guard bin, with zero energy.

The location of the internal pilot waves is dictated by equation [26]:

$$P_{\text{bins}} = \left\lfloor k_0 - 1 + \frac{P_i \cdot (M + N_{\text{pilot}} - 1)}{N_{\text{pilot}} + 1} \right\rfloor, \quad P_i \in \{1, 2, \dots, N_{\text{pilot}}\} \quad (26)$$

This spreads them equally throughout the active region. The remaining active bins are the data bins, mapped sequentially. There are many schemes for choosing the value of the pilot waves, but for simplicity, a simple complex constant of $P_v = \frac{1}{\sqrt{2}} + i\frac{1}{\sqrt{2}}$ was chosen, as it has equal power in both the in-phase and quadrature components, and magnitude one, leading nicely into using them for channel estimation. It is encoded in Q2.22, as to match the format of the test data.

The module has been designed such that all the parameters are tunable, and this project used: $M = 256, N = 1024, N_{\text{CP}} = 8, N_{\text{pilot}} = 8, k_0 = 22$. With a width of each bin being about 48.8kHz, that makes the active region range from $(k_0 - 1) \cdot 48.8\text{kHz} = 21 \cdot 48.8\text{kHz} \approx 1.02\text{MHz}$ to $(k_0 + M + N_{\text{pilot}} + 1) \cdot 48.8\text{kHz} \approx 14.0\text{MHz}$. k_0 was chosen such that the minimum occupied bin is above 1MHz, as decided in chapter 3.3.

4.3.3 OFDM Module - FPGA Implementation and Testing

In practice, implementing OFDM requires two main components; an IFFT core, and a cyclic prefix generator. The IFFT core used in the OFDM implementation is the one discussed and implemented in section 4.3.1, and the details will thus not be discussed here.

The cyclic prefix consists of the last N_{CP} samples of the completed OFDM symbol, which gets prepended before transmission. As such, since the (I)FFT algorithm is streaming, the entire OFDM symbol must first be stored in RAM, one sample at a time, and only once all N samples have been stored can the module output the OFDM symbol, with a total of $N + N_{\text{CP}}$ samples.

For this purpose, a set of ping-pong RAMs have been used, similar to the FFT module, which switch between writing (storing the output of the IFFT), and reading (outputting from the OFDM module), with the pointer being initialized to $N - N_{\text{CP}}$, then wrapping back to zero at N , and stopping when it wraps a second time, where it switches the mode select to write.

Additionally, since the IFFT core is streaming, and there is a desired bin-mapping, it is necessary to store the inputs into the OFDM module until their bin is processed. For this, a simple FIFO queue was utilized, simply storing the inputs until the module calls for a data symbol, then reading the next available. This ensures the data bins get filled sequentially, allowing for simple decoding in the receiver.

Before the OFDM module can be used in a full digital SDR, it must first be tested, to ensure it functions as expected. For this purpose, a simple test-bench was created, which performed

OFDM on a ramp of 256 samples, with a cyclic prefix of 8, OFDM size of 1024, internal pilot count of 8, and initial bin of 22. The output was written to a file, and loaded into a Julia script, which decoded it. The result of the test is shown on Figure 11.

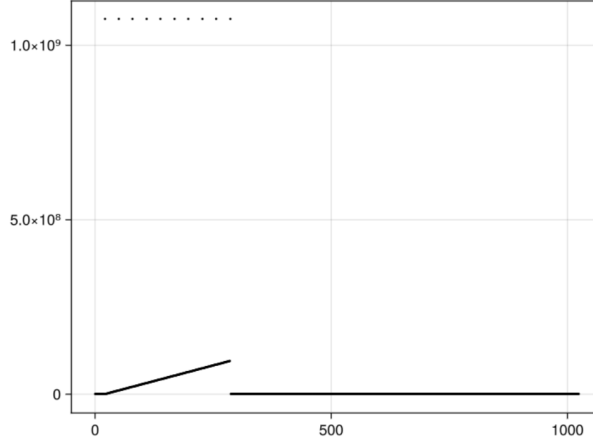


Figure 11: Decoded OFDM ramp

As can be seen, the decoded output looks like a ramp, as expected, and the dots near the top represent the pilot waves. Computing the EVM of the decoded output vs the expected for the entire OFDM symbol, the data bins and the pilot bins using Equation (14), as well as the relative energy leakage into the guard bands as RMS:

$$\text{EVM}_{\text{full}}^{(\text{dB})} = -78.4 \text{ dB} \quad (27)$$

$$\text{EVM}_{\text{data}}^{(\text{dB})} = -77.3 \text{ dB} \quad (28)$$

$$\text{EVM}_{\text{pilots}}^{(\text{dB})} = -78.6 \text{ dB} \quad (29)$$

$$\text{leakage}_{\text{guards}}^{(\text{dB})} = 20 \log_{10} \left(\sqrt{\frac{\sum_{n \in \text{guards}} |\hat{X}_n|^2}{\sum_{n \in \text{actives}} |\hat{X}_n|^2}} \right) = -95.4 \text{ dB} \quad (30)$$

These results indicate that the quantization noise is very low (on the order of -80dB), and that guard-band leakage is negligible, at least in the OFDM module.

4.3.4 OFDM SDR - Implementation and Testing

As with the single-carrier SDR test, the OFDM SDR test will be conducted using the same payload, 1024 bytes of Lorem Ipsum¹⁰, and encoded as 16QAM. However, as opposed to a simple single-carrier modulation, there is no need to explicitly perform a frequency shift to avoid the attenuation of low frequencies, as that has already been accomplished by the bin-mapping. Similarly, a pulse-shaper is not used for OFDM, as the issue of ISI was already dealt with by the orthogonal carrier modulation and cyclic prefix. Some OFDM implementations include windowing, which can reduce spectral leakage from the implicit rectangular window, but this was not implemented in this project, as it can increase the complexity significantly, and is not necessary for a proof-of-concept implementation.

The system consists of the controller, which tells the data source when to output data, by toggling a valid signal. The data then propagates through the symbol shaper, into the OFDM module, then out through the DAC, which is then sampled by the Analog Discovery 2 (AD2). Additionally, a trigger has been added which pulses high when a new OFDM symbol starts transmission, allowing for easy synchronization.

¹⁰While the entire payload gets transmitted on repeat, only a single OFDM symbol is decoded, corresponding to 128 bytes.

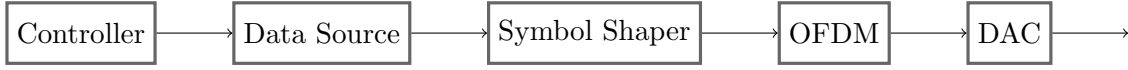


Figure 12: Block diagram of OFDM SDR

Before considering the data generated by the OFDM SDR, it is pertinent to investigate the resource utilization and timing requirements of the system. Note that this is for the whole OFDM SDR, and not just the OFDM module individually. The resource utilization, which is shown in Quartus Prime Pro, is as such:

- 3,895/46,800 ALMs (8%)
- 427,776 / 7,331,840 Memory Bits (6%)
- 54 / 358 RAM blocks (15%)
- 57 / 188 DSP blocks (30%)
- 7.030ns slack, leading to $F_{max} = 77.1\text{MHz}$

Comparing this to the utilization of the FFT core from earlier, notice that the resource utilization is significantly less. This is because the FFT core was compiled with an assumed $N = 4096$, rather than the $N = 1024$ of the OFDM module. As such, the OFDM SDR fits comfortably within the confines of the requirements of the DE25.

This project will assume perfect synchronization due to the trigger, but in real wireless systems no such help exists. Instead, a few different approaches exist, for example, in modulations utilizing a cyclic prefix, you can check for high correlation between samples separated by N samples, in order to detect the cyclic prefix.

Once the cyclic prefix has been removed, which is easy once the start of the OFDM symbol has been found, which, as mentioned before, is assumed to be the trigger, only the actual OFDM symbol remains. Firstly, before any decoding happens, it makes sense to investigate the issue of PAPR.

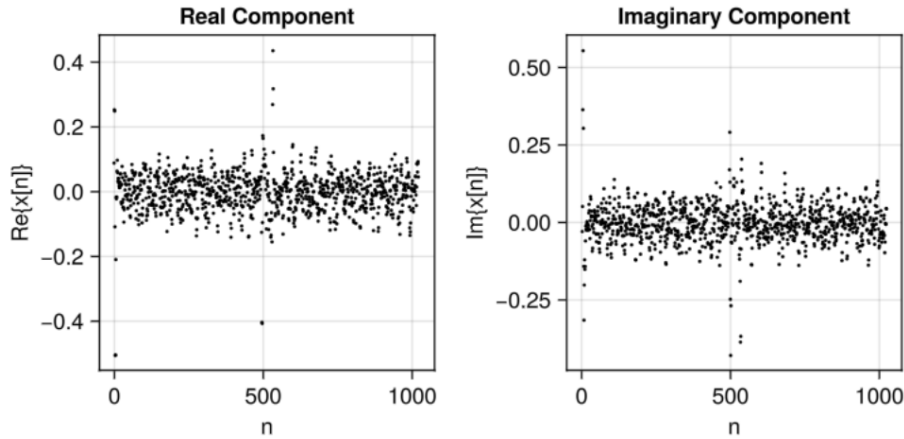


Figure 13: Real and Imaginary components of the OFDM symbol

As can be seen on Figure 13, there are large peaks compared to the mean. To quantify this, we compute the PAPR, both individually for each component, as well as for the complex signal using Equation (15):

$$\begin{aligned}
 \text{PAPR}_R &= 61.5 \approx 17.89 \text{ dB} \\
 \text{PAPR}_I &= 73.03 \approx 18.64 \text{ dB} \\
 \text{PAPR} &= 41.5 \approx 16.19 \text{ dB}
 \end{aligned} \tag{31}$$

In this project the PAPR for the real and imaginary, or in-phase and quadrature, components are the most important, as they are what influence our DAC backoff. Taking $\log_2$ of the squareroot of them yields $\log_2(\sqrt{\text{PAPR}_R}) = 2.97$, $\log_2(\sqrt{\text{PAPR}_I}) = 3.09$, and so a backoff of 3-4 bits is likely sufficient to avoid any significant clipping. The PAPR of the complex signal is needed for the analog circuitry post mixing, as this informs how much extra linearity headroom is required of the gain stage.

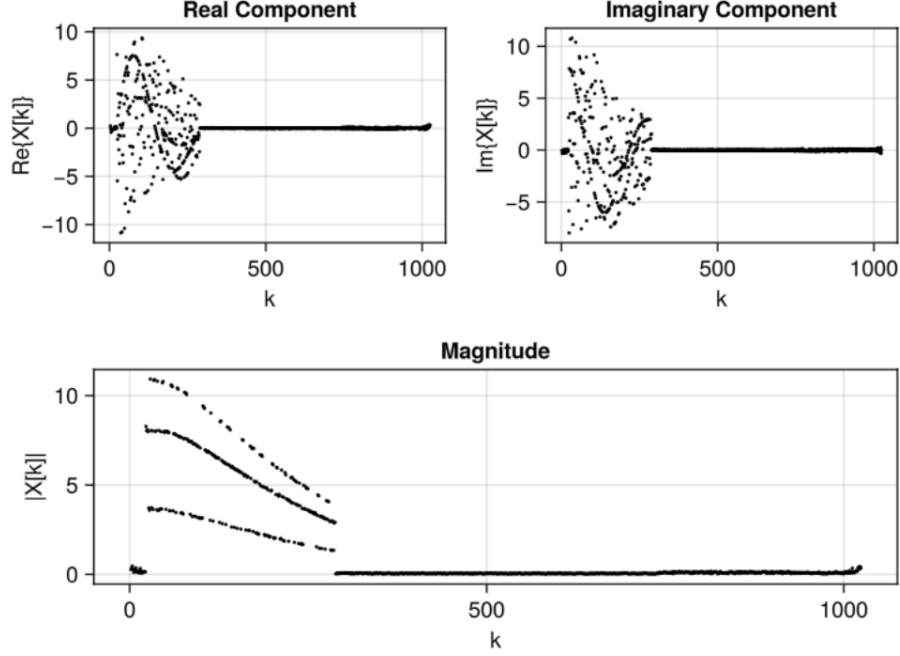


Figure 14: Real, Imaginary and magnitude of FFT of the OFDM symbol

Taking the FFT of the OFDM symbol can be seen on Figure 14. Before considering the shape of the data, first notice the small changes around the end of the symbol. This is caused by aliasing, since there is no low-pass filter between the DAC output and the AD2. However, this effect is not very pronounced, which is due to the frequency response of the channel between the SDR and AD2, which will be categorized shortly from the pilots, and which shows classic low-pass filter characteristics. This is interesting, since the DAC was also shown to fit a 1st order highpass filter in chapter 3.3, and so the channel can better be described as a band-pass filter.

The band-pass nature of the channel is also very noticeable in the magnitude plot of the FFT of the OFDM symbol in Figure 14, where the samples split into clean levels, but with a noticeable droop. To combat this, the pilot waves can be used to estimate the channel.

Since the pilots are sparse (only 10 bins spread throughout the 266 active bins), we must estimate the channel response between the pilots. In the specific case where the channel is a wire between the DAC and AD2, it might make sense to try and fit to a model, for example a low-pass filter, however in a real wireless channel, there is no such simple model in general, and as such it makes more sense to simply linearly interpolate between pilots. When linearly interpolating between two real numbers, there is no ambiguity as to how to do it, but with complex numbers, there are at least two valid approaches, interpolate between the complex numbers as a whole, or between the phase and magnitude separately. But first, the channel response at the pilots must first be estimated, which can be done since their transmitted value is known:

$$\tilde{H}[k] = \frac{X[k]}{S[k]} = \frac{X[k]}{\frac{1}{\sqrt{2}} + i\frac{1}{\sqrt{2}}}, \quad k \in \text{Pilots} \quad (32)$$

Where $X[k]$ is the received symbol, and $S[k]$ is the transmitted symbol, both in frequency domain. The channel estimation using both methods is shown on Figure 15, where the magnitude

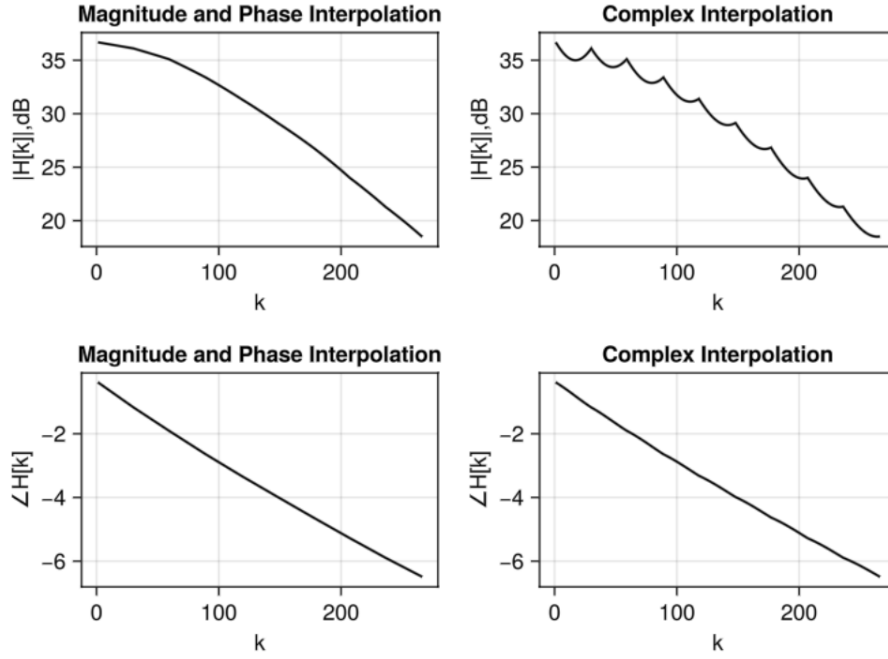


Figure 15: Channel estimation using pilots

plots are in dB. As can be seen, both plots look similar, but the one using separate magnitude and phase interpolation is a lot smoother in magnitude.

Now that a channel estimate has been found, it can be inverted using a Zero-Forcing (ZF) equalizer [30]:

$$Y[k] = \frac{X[k]}{\hat{H}[k]} \quad (33)$$

Where $Y[k]$ is the equalized symbol. Note that if the channel is very small for some frequencies, the ZF equalizer can strongly amplify the noise, as we model the transmission as AWGN. If this happens, it may be necessary to pre-equalize at the transmitter, though this requires transmitter CSI. However, this is not a problem in this project, as the OFDM symbols were never actually transmitted wirelessly. The corrected data, in frequency domain, can be seen on Figure 16. Using the (ZF) equalizer on the received signal, and scaling by the average power of the MQAM modulation, which in this case is $\sqrt{10}$, since 16QAM was used, the constellation can be seen on Figure 17. The points on the constellation diagram are very tightly clustered around the 16QAM levels, and the decoded test is

”Lorem ipsum dolor sit amet, consectetur adipiscing elit. Maecenas dolor risus, luctus ut lorem non, pharetra blandit velit. Duis”

Which corresponds to the first 128 bytes of the used Lorem Ipsum. This matches the transmitted string, and the length corresponds to 256 data samples, modulated as 16QAM, which gives 128 bytes, which matches what was decoded. This yields an error rate of zero for this data set. The EVM for the data set is

$$\text{EVM} \approx 0.015 \approx -36.2\text{dB} \quad (34)$$

This is a very good result, and it is clear from Figure 17 that larger MQAM modulations could be used. However, since this data was sampled directly from the DAC, without first being transmitted wirelessly, it is difficult to say how large MQAM could be used. Comparing the

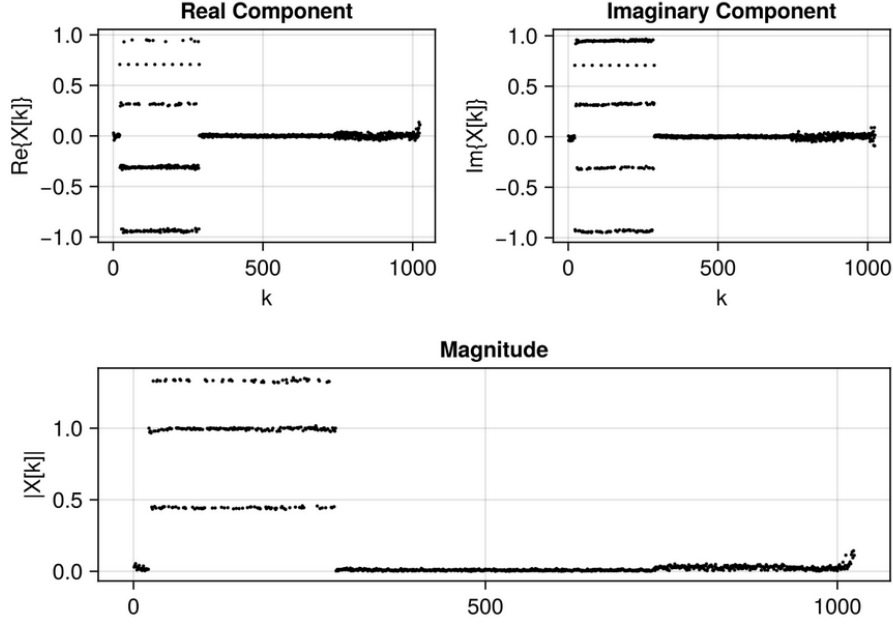


Figure 16: Real, Imaginary and magnitude of the Zero-Forced equalized OFDM symbol

data rates for different MQAM modulations:

$$R_{16QAM} = \frac{256}{1024 + 8} \cdot 4 \frac{\text{bits}}{\text{symbols}} \cdot 50 \frac{\text{Msamples}}{\text{second}} \approx 50 \frac{\text{Mbit}}{\text{s}} \quad (35)$$

$$R_{64QAM} = \frac{256}{1024 + 8} \cdot 6 \frac{\text{bits}}{\text{symbols}} \cdot 50 \frac{\text{Msamples}}{\text{second}} \approx 75 \frac{\text{Mbit}}{\text{s}} \quad (36)$$

$$R_{256QAM} = \frac{256}{1024 + 8} \cdot 8 \frac{\text{bits}}{\text{symbols}} \cdot 50 \frac{\text{Msamples}}{\text{second}} \approx 100 \frac{\text{Mbit}}{\text{s}} \quad (37)$$

Additionally, the total occupied bandwidth is

$$B = \frac{256 + 10}{1024} \cdot 50 \text{MHz} \approx 13 \text{MHz} \quad (38)$$

This gives a spectral efficiency for 16QAM of

$$\frac{R_{16QAM}}{B} \approx 3.85 \text{ bits/s/Hz} \quad (39)$$

Using the Shannon Theorem [33], the theoretical minimum required SNR for any system to obtain this spectral efficiency can be estimated:

$$\gamma_{\min} = 2^{\frac{R}{B}} - 1 \approx 13.4 \approx 11.3 \text{dB} \quad (40)$$

Note that this is the absolute minimum for a perfect system with optimal coding, modulation, and zero error from quantization can achieve. In reality, as this version of the SDR has no coding, and cannot be fully modelled as an AWGN channel, the required SNR is somewhat larger, but it is nonetheless an interesting comparison.

4.3.5 Single Carrier Frequency-Division Multiple Access

As mentioned in the prior chapters on OFDM, one of the main challenges is the high Peak-To-Average Power (PAPR). This is because, in time-domain, OFDM is the sum of many independently modulated subcarriers which, at some time instants, can add coherently, leading to a large spike in power. From a probabilistic view, the sum of many independent subcarriers will follow a complex Gaussian distribution due to the central limit theorem, leading to an exponential power distribution.

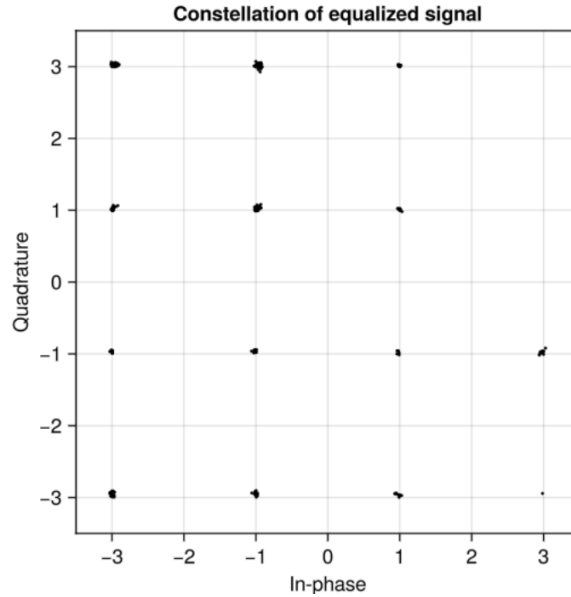


Figure 17: Constellation Diagram of OFDM symbol

SC-FDMA reduces this by applying a DFT to the MQAM symbols before the subcarrier mapping, meaning each symbol is spread across all allocated subcarriers, and for localized SC-FDMA, where the symbols are allocated sequential subcarriers, then the subcarriers can no longer be said to be independent, and thus the central limit theorem no longer leads to the signal following a complex gaussian distribution. In general, this leads to a lower PAPR [5].

The general structure of SC-FDMA is shown in Figure 18.



Figure 18: Block diagram of SC-FDMA

4.3.6 SC-FDMA Module - FPGA Implementation and Testing

Since SC-FDMA applies a DFT pre-coding, another FFT core is required. Since the FFT core in this project is radix-2 based, it means each SC-FDMA symbol must be a power of two. To make comparison with OFDM easier, the same parameter values have been chosen. In general, OFDM does not require that the amount of active data carriers fit neatly with the FFT architecture, as the symbol will be padded such that it does, but a power of 2, 256 specifically, was chosen to make the comparison possible.

With an OFDM module and FFT core already implemented, an SC-FDMA module becomes trivial to implement, as it only consists of just those parts, as shown in figure (18). However, there is one small issue that must be corrected. When performing the DFT precoding, the average energy grows by $\sqrt{M}$, which means the input into the OFDM module does not have the same average power as it would for a pure OFDM modulation. As such, it is necessary to normalize the energy, specifically if we wish to use the pilots for channel estimation. To avoid clipping, this normalization is performed before the FFT core, though this does increase quantization noise.

As with the OFDM module, it is necessary to test and verify the functionality of the module before it can be used in a larger implementation. For this purpose, a testbench using the same ramp input as for OFDM was constructed, with the output written to a file. Figure 19 shows the FFT of the output, with the cyclic prefix removed, and the DFT of the active subcarriers.

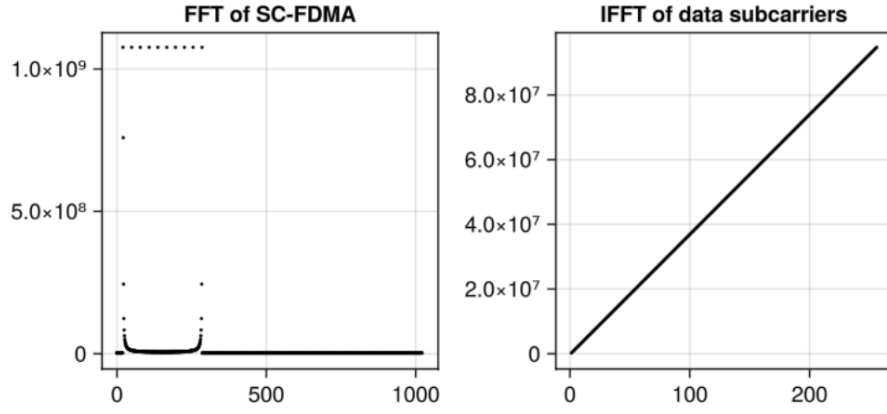


Figure 19: SC-FDMA ramp test

Using the same EVM analysis as for OFDM, in frequency domain:

$$\text{EVM}_{\text{full}}^{(\text{dB})} = -77.6 \text{ dB} \quad (41)$$

$$\text{EVM}_{\text{data}}^{(\text{dB})} = -71.9 \text{ dB} \quad (42)$$

$$\text{EVM}_{\text{pilots}}^{(\text{dB})} = -78.6 \text{ dB} \quad (43)$$

$$\text{leakage}_{\text{guards}}^{(\text{dB})} = -95.2 \text{ dB} \quad (44)$$

These results are of the same order of magnitude as for the OFDM module, which is expected as the implementation utilizes it.

4.3.7 SC-FDMA SDR - Implementation and Testing

The SC-FDMA SDR implementation and testing is almost identical to that of the OFDM SDR, and as such only the results will be discussed here, however a block diagram of the system can be seen on Figure 20.



Figure 20: Block diagram of SC-FDMA SDR

As with the OFDM SDR, the resource utilization will be investigated first, to ensure that the SC-FDMA SDR meets the resource and timing requirements. Reading utilization in Quartus Prime Pro yields:

- 6,009/46,800 ALMs (13%)
- 518,656 / 7,331,840 Memory Bits (7%)
- 88 / 358 RAM blocks (25%)
- 102 / 188 DSP blocks (54%)
- 7.158ns slack, leading to $F_{\text{max}} = 77.87\text{MHz}$

As can be seen, there is a fair increase in resource utilization over OFDM, especially in terms of the number of DSPs, which increased by almost 80 percent. This is the main trade-off between OFDM and SC-FDMA, a decrease in PAPR for an increase in computation and complexity. However, the SDR still fits comfortably within the DE25.

As mentioned, PAPR reduction is the main draw of SC-FDMA over OFDM, and quantifying it in this test using Equation (15):

$$\text{PAPR}_R = 6.96 \approx 8.43 \text{ dB} \quad (45)$$

$$\text{PAPR}_I = 8.90 \approx 9.50 \text{ dB} \quad (46)$$

$$\text{PAPR} = 5.08 \approx 7.06 \text{ dB} \quad (47)$$

Taking $\log_2$ of the square root of the real and imaginary parts give the required back-off in bits for the digital implementation, yielding around 1-2 bits of back-off for both.

A single transmitted SC-FDMA symbol, with the cyclic prefix removed, is shown in Figure 21. Compared to the plot of the transmitted OFDM symbol, shown in Figure 17, there are no massive spikes.

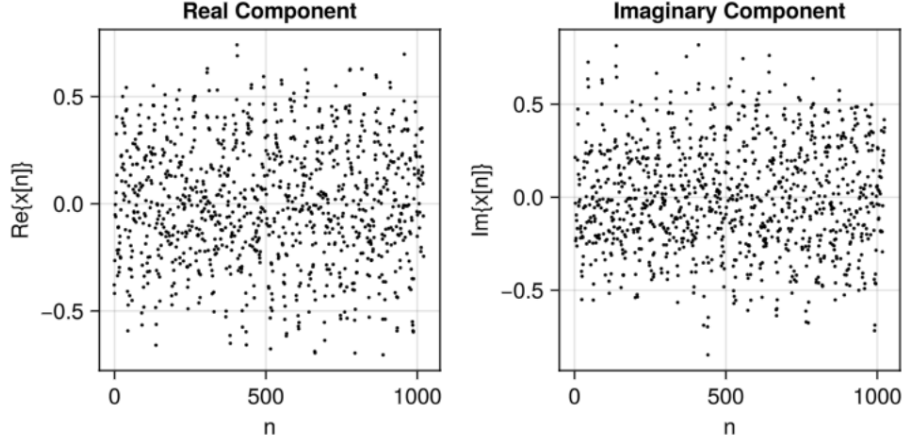


Figure 21: Transmitted SC-FDMA symbol

Taking the Fourier Transform of the data yields the spectrum, shown in Figure 22. There are two big spikes, but otherwise very little structure to the spectrum, unlike for OFDM where the clear levels stood out.

Using the pilots for channel estimation in the same way as OFDM yields the response shown in

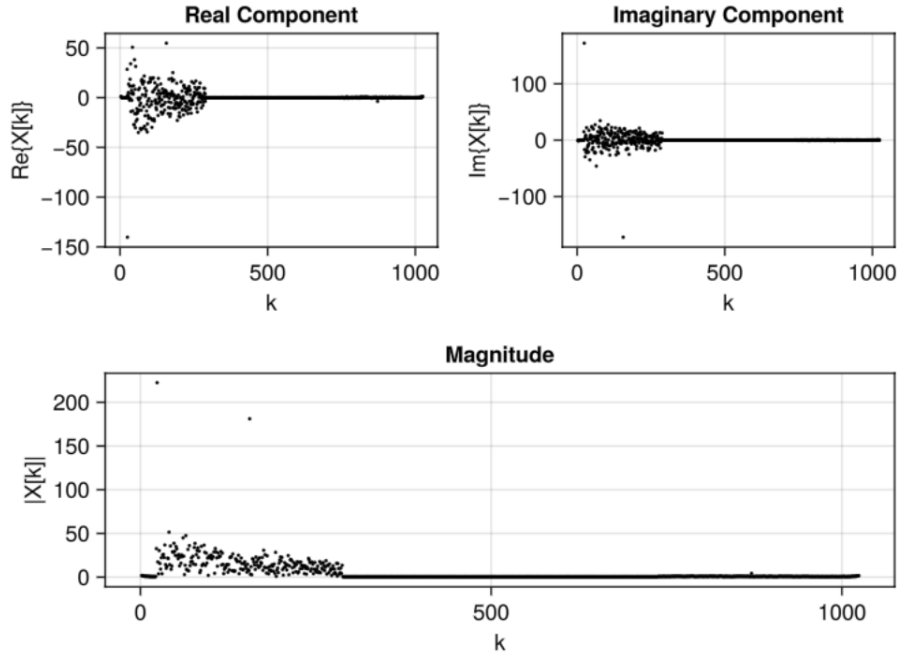


Figure 22: Fourier Transform of the SC-FDMA symbol

Figure 23. The channel estimations using both separate magnitude and phase interpolation, and

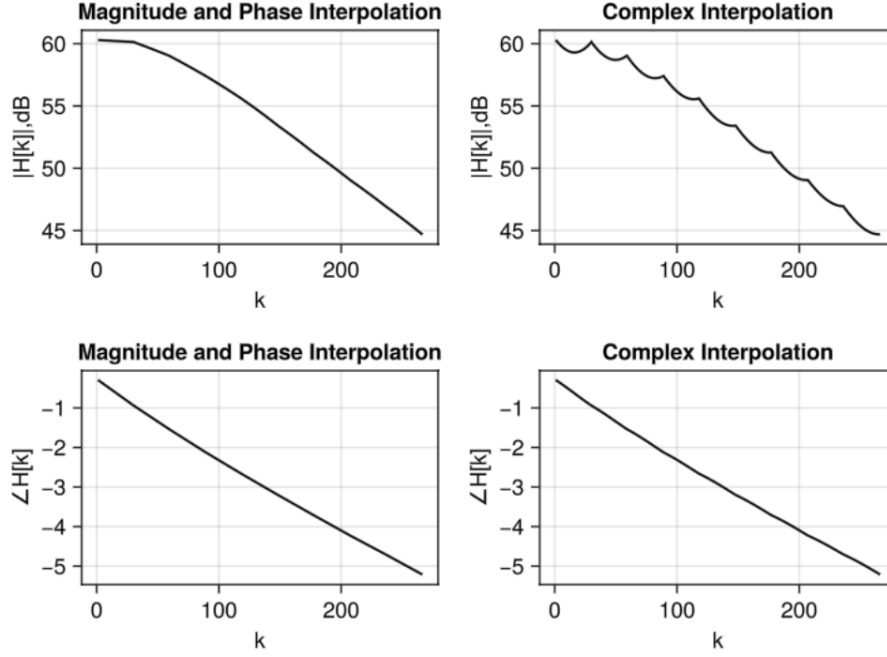


Figure 23: Channel Estimate for SC-FDMA SDR

complex value interpolation are shown on Figure 23. Again, the estimation utilizing separate magnitude and phase interpolation is chosen.

The constellation diagram for the zero-forced equalized signal is shown on Figure 24. Similar to the OFDM SDR, the constellation is incredibly tight around the levels, and a much higher order of MQAM could potentially be used, depending on the wireless channel the SDR is deployed in. But as this symbol was not transmitted wirelessly, it isn't possible to tell how large MQAM could be used.

Decoding the SC-FDMA symbol as ASCII gives "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Maecenas dolor risus, luctus ut lorem non, pharetra blandit velit. Duis", which matches the transmitted data perfectly, yielding an error rate of 0 for this test as well. Calculating the EVM yields

$$\text{EVM} \approx 0.0072 \approx -42.8 \text{ dB} \quad (48)$$

Which is about half of the OFDM test.

5 Analog Circuitry - From FPGA to Antenna

With the digital implementations having been tested and verified, it is pertinent to consider the analog front-end. An RF front-end has been designed, and several PCBs sent to manufacturing, but they remain untested as the PCBs were never delivered. As such, this chapter is focused on documenting and justifying the design, with no experimental verification of the circuitry.

5.1 Signal Chain Design

A block diagram of the RF front-end is shown in Figure 25.

The chain goes as:

1. The In-Phase and Quadrature outputs of the DAC, corresponding to the real and imaginary components, are each fed into a low-pass reconstruction filter, which is used to remove the aliases caused by the sample-and-hold behaviour of the DAC.
2. The output of the DAC is single-ended in nature, and so the reconstructed signals are turned into differential signals with a common-mode voltage of 1.5V using a Dual Fully Differential Amplifier.

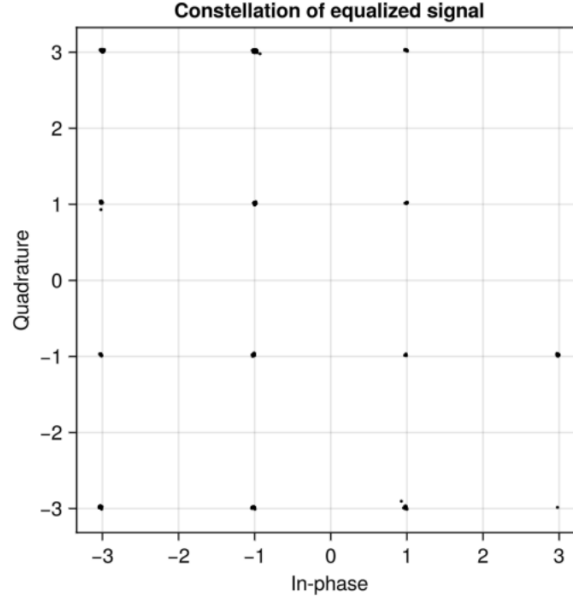


Figure 24: Constellation Diagram of the SC-FDMA symbol

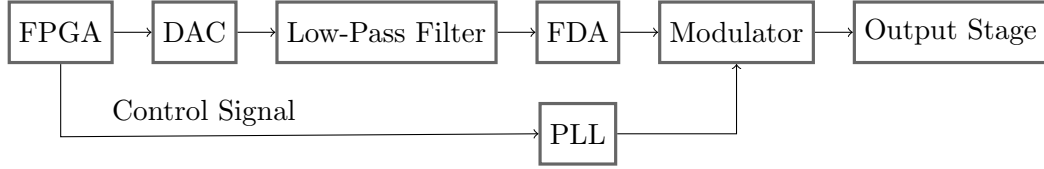


Figure 25: Block diagram of the RF Front End

3. A PLL is used as the Local Oscillator (LO), interfacing with the FPGA via a 3-wire interface to control the output frequency.
4. The differential In-Phase and Quadrature signals are modulated onto the carrier from the LO.
5. The upconverted signal is fed into the output stage, consisting of a band-pass filter, gain and impedance matching, and finally the antenna used for transmission.

5.1.1 The Low-Pass Reconstruction Filter

The digital signals being generated in the FPGA are not baseband due to the high-pass nature of the DAC output. As such, the signal is technically a band-pass signal, and it would be more correct to use a band-pass filter rather than a low-pass filter. However, as there is a band-pass filter in the output stage, it can remove any noise that might exist below the lower cutoff frequency.

In order to distort the signal as little as possible, it is necessary for the pass-band to be as flat as possible. As such, a low-pass Butterworth filter is used, as it is maximally flat [34]. A low-pass Butterworth filter is characterized by 2 main parameters;

- Cutoff frequency, defined as the frequency at which there is a 3dB attenuation.
- Order of the filter, which determines the number of components and the gain in the stop-band, which is given as -20dB/decade per order.

The cutoff frequency of the SDR depends on which modulation was used, but the OFDM and SC-FDMA architectures both have an upper cutoff given by:

$$f_{\text{upper}} = f_{\text{lower}} + B = 1 \text{ MHz} + \frac{256 + 10}{1024} \cdot 50 \text{ MHz} \approx 14 \text{ MHz} \quad (49)$$

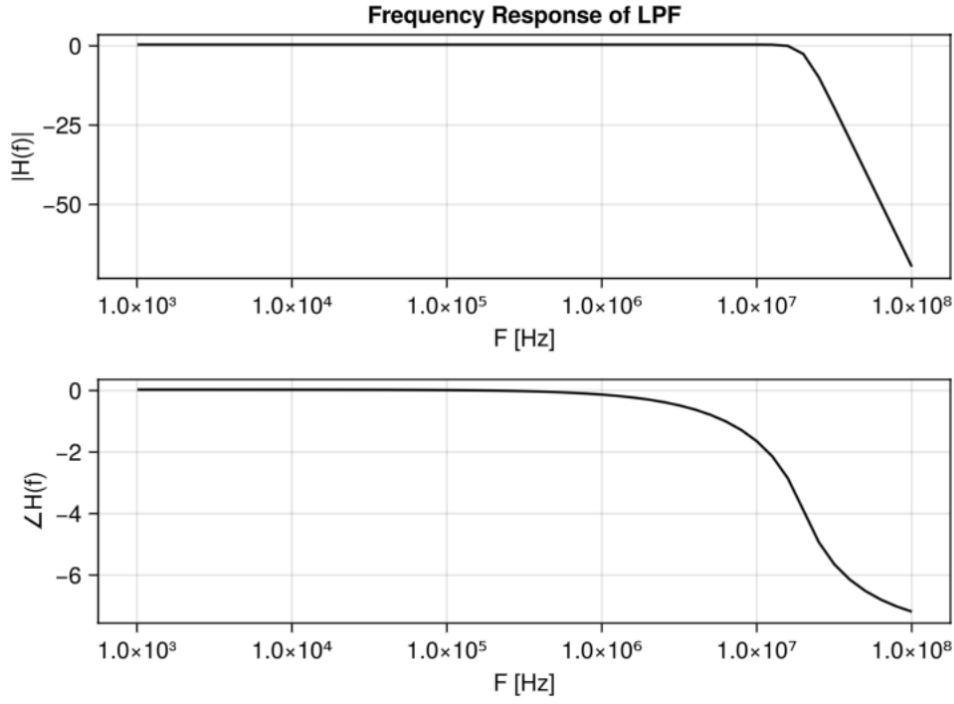


Figure 26: Simulated frequency response of the 5th order Butterworth low-pass filter

Additionally, the nearest alias starts at $50 \text{ MHz} - 4 \text{ MHz} = 36 \text{ MHz}$, and so a cutoff frequency of 20 MHz strikes a good balance between not distorting the signal, while attenuating the aliases. The Butterworth low-pass filter is of 5th order, meaning it has 100 dB/decade attenuation, and so with a cutoff frequency of 20 MHz , the attenuation at 36 MHz is

$$A = \log_{10} \left(\frac{36}{20} \right) \cdot 100 \text{ dB} = 25.5 \text{ dB} \quad (50)$$

The component values can be found by calculating the normalized coefficients, and then converting those into component values, however, in practice it is often easier to use a tool, such as [35], to find the component values, which was also done in this project. Note that a Butterworth filter requires the input and output impedance to be equal, and here it is assumed to be 50Ω . The structure of the dual Butterworth low-pass filter is shown in Appendix P, and the component values are $C_1 = C_3 = C_4 = C_6 = 98 \text{ pF}$, $C_2 = C_5 = 318 \text{ pF}$, $L_1 = L_2 = L_3 = L_4 = 643.8 \text{ nH}$, $R_1 = R_2 = 50 \Omega$. It would then be necessary to find the closest component values available and use those. The frequency response of the filter can be seen on Figure 26.

5.1.2 Fully Differential Amplifier - THS4552

The output of the DAC is single-ended, and the chosen quadrature modulator, the ADL5375, requires both the In-Phase and Quadrature components be differential with a common-mode voltage of 1.5 V . To achieve this, the THS4552 Dual Fully Differential Amplifier from Texas Instruments was chosen [36]. It has a bandwidth of 150 MHz , which is much larger than the required 14 MHz , and a tunable common-mode voltage.

The common-mode voltage of the outputs are set as the voltage on the VCOM1 and VCOM2 pins respectively. With a supply voltage of 5 V from the FPGA, a simple voltage divider was built:

$$V_{COM} = \frac{R_1}{R_1 + R_2} V_s = \frac{3 \text{ k}\Omega}{3 \text{ k}\Omega + 7 \text{ k}\Omega} \cdot 5 \text{ V} = 1.5 \text{ V} \quad (51)$$

This only works because the input impedance on the VCOM pins is much larger, and so they won't distort the voltage divider. The gain of the FDA is set to 1, with all the resistors being $1 \text{ k}\Omega$. There is also included several capacitors of $10 \mu\text{F}$ and $0.1 \mu\text{F}$ on the rails to reduce power

supply noise, especially near the voltage divider and VCOM pins. A schematic of the circuit is shown in Appendix P.

5.1.3 Quadrature Modulator - ADL5375

The upconverted signal is given by [37]:

$$x(t) = s_I(t) \cos(2\pi f_c t) - s_Q(t) \sin(2\pi f_c t) \quad (52)$$

Where $s_I(t)$ is the In-Phase component, and $s_Q(t)$ is the Quadrature component. In practice actually doing this multiplication in an analog circuit is complicated, and since the main focus of the project is on the digital implementations, a Quadrature Modulator IC was chosen, specifically the Analog Devices ADL5375 [38], which has a frequency range of 400MHz to 6GHz, which fits the desired 2.4GHz comfortably. The circuit for it is simple, with the main issue being the need for a differential input, which was solved in the previous chapter using the THS4552. Two version of the ADL5375 exists, one with a required common-mode voltage of 1.5V, and one with 0.5V. The 1.5V version was chosen, as the THS4552 can only do common-mode voltages between 0.7V-3V. The schematic of the ADL5375 circuit is shown in Appendix P.

While the ADL5375 does not require that the Local Oscillator is also a differential signal, the datasheet notes that LO leakage can be reduced by driving it differentially. As such, the output of the LO is differential.

5.1.4 Local Oscillator - ADF4351

The by far most complicated part of the RF front-end is the PLL design, which is built upon the ADF4351 by Analog Devices [39]. It is advertised as a wideband synthesizer, with a frequency range of 35MHz to 4400MHz, though it requires the use of an external oscillator, which here is a 25MHz Temperature Controlled Crystal Oscillator (TCXO).

While the frequency of the output can be controlled by a 3-wire interface with the PLL, which would interface directly with the FPGA, the behaviour of the PLL is also determined by the loop filter. The design of loop-filters is incredibly complicated, and since this project is focused on the digital implementation, inspiration for the loop-filter design is taken from the associated vendor evaluation board [40], which is a third-order passive low-pass filter with critical frequency of 35kHz, which is used to smooth out the feedback signal in the PLL.

Additionally, as mentioned in the previous section, the LO leakage from the modulator can be reduced by driving the output differentially, and so the architecture for the differential output from the evaluation board was utilized. The schematic for the ADF4351 circuitry can be seen in Appendix P.

One important detail is that the digital signal is not centered around DC, but is upconverted to a low IF to circumvent the high-pass nature of the DAC, as was described in section 3.3. Since the output frequency from the ADF4351 is programmable from the FPGA, it should be offset accordingly, such that the resulting upconverted signal is correctly centered. The frequency of the LO can be calculated by

$$f_{LO} = f_{RF} - f_{IF} \quad (53)$$

Where f_{LO} is the frequency of the LO, f_{RF} is the desired RF frequency, and f_{IF} is the intermediate frequency of the digital signal, and can be calculated as

$$f_{IF} = \frac{f_{Upper} + f_{Lower}}{2} \quad (54)$$

Which for OFDM and SC-FDMA SDRs is $\frac{14\text{ MHz} + 1\text{ MHz}}{2} = 7.5\text{ MHz}$, and for the SC SDR is $\frac{8.375\text{ MHz} + 1.625\text{ MHz}}{2} = 5\text{ MHz}$, which also matches the explicit frequency shift described in section 4.2.2

5.1.5 Proposed Output Stage

The final part of the RF front-end is the output stage, which would consist of an RF band-pass filter, gain stage, impedance matching network, and antenna interface. However, this stage was never designed in practice, as the prior analog stages were to be tested and verified before final choices on parameters such as the exact transmit frequency, desired output power and antenna were to be made.

The purpose of the band-pass filter is to ensure that the transmit power outside the allocated transmit band, including things like LO leakage, harmonics and broadband noise, is below regulatory limits. Determining the required attenuation necessitates extensive testing of the prior stages, which, as previously mentioned, were not delivered on time. The gain stage would then boost the signal to final transmit power, while the impedance matching network would ensure optimal power transfer by matching the output impedance of the RF front-end to the input impedance of the antenna.

As such the output stage is left as future work, to be designed after the output spectrum of the RF front-end has been characterized, and a final antenna has been decided on.

5.2 PCB Design

As mentioned the PCBs were not delivered on time, and as such were not experimentally verified, however there are still several design considerations worth discussing.

In this project it was decided to split the circuitry into 2 PCBs, one for the ADF4351 PLL, and one for the remaining RF chain, so low-pass reconstruction filter, fully differential amplifier and quadrature modulator, such that they could each be tested and iterated on separately. This choice was made due to the complicated nature of the ADF4351, and the higher likelihood of issues with it, meaning the RF front-end board could still be tested and iterated on separately in the event that the ADF4351 board did not work, as it could be tested using an external LO source.

The main considerations when designing the PCBs were the component size, specifically for the passive components, routing, and the number of layers. The smaller-valued or higher-frequency components, such as the capacitors for the output network for the PLL were chosen as 0402 size SMD components to minimize their parasitic effects, and the larger-valued or lower-frequency components, such as the resistors and low-pass components were chosen as 0603, as they are less sensitive to parasitic effects relative to their intended impedance, while making assembly, inspection and potential reworks easier.

Secondly, for the routing, in order to reduce parasitic effects and unwanted couplings, the more critical paths, meaning higher-frequency signals as well as the actual RF signal, should be routed shorter and straighter, while control signals can be routed longer and more convoluted if necessary. This also means keeping the more important signals on the upper plane, where the components are soldered, and allowing the less critical signals to route via the bottom plane with vias. Additionally, complementary paths, such as the path for the in-phase and quadrature components, should be as equal as possible, as any difference in them can lead to distortions.

Lastly, both PCBs use 4 layers. The top layer is where the components are soldered, as well as the most critical paths being routed. The second layer is a continuous ground plane, as several of the components, specifically the ADF4351 [39] and ADL5375 [38], benefit from short ground paths. The third layer is a continuous power plane, as it can improve performance of the components, but is generally not as critical as the ground plane. The bottom layer is where less critical paths are routed when necessary, as it is, in general, not possible to fully route a PCB with only one plane as paths cannot cross.

The main RF front-end PCB circuit consists of the individual schematics shown in the prior chapters chained together, as well as a series of test points, which can be used for measurements for testing. The schematic and PCB of the RF front-end can be seen in Appendix P, pages 4 and 5.

The PCB for the PLL is based on the same schematic as was shown in the previous chapter.

They can both be seen in Appendix P, pages 6 and 7. The connections between the two PCBs are two SMA coaxial connectors, one carrying the positive LO signal, and one the negative LO signal. This is because a differential signal requires both paths to be equivalent in terms of impedance and interference, and since an SMA cable consists of an inner core designed for carrying the signal, and an outer shielding as the return path, it is not suitable for differential signalling. As such, two were utilized, one for positive and negative, with the other shielding being wired to grounds on both sides. This is an unoptimal solution compared to using a cable designed for differential signals, but they were chosen anyways for their availability.

6 Results and Discussion

This chapter summarises all prior results, and investigate if the requirements set out in chapter 2.1 have been met, as well as compare the three digital implementations to each other on metrics of performance and resource utilization. The future work of the project is discussed, including the testing and verification of the RF front-end.

The results are shown in Table 6, and are evaluated according to the requirements defined in chapter 2.1. The comparison focuses on functional correctness, signal quality, spectral efficiency, resource utilization, and timing closure. Functional correctness is evaluated from the decoded payload and measured error rate, spectral efficiency as the ratio of data rate to bandwidth, while signal quality is evaluated using EVM and PAPR. Hardware feasibility is evaluated from the Quartus resource utilization and timing reports. Comparing the results we see that OFDM and

Measure	Single Carrier	OFDM	SC-FDMA
Data Rate	20 Mbit/s	50 Mbit/s	50 Mbit/s
Bandwidth	6.75 MHz	13 MHz	13 MHz
Spectral Efficiency	2.96 bit/s/Hz	3.85 bit/s/Hz	3.85 bit/s/Hz
Payload Error Rate	0	0	0
Payload EVM	-23.8 dB	-36.2 dB	-42.8 dB
PAPR - Complex	5.15 dB	16.19 dB	7.06 dB
ALM usage	1,475 (3%)	3,895 (8%)	6,009 (13%)
RAM block usage	0 (0%)	54 (15%)	88 (25%)
DSP usage	27 (14%)	57 (30%)	102 (54%)
Slack	8.249 ns	7.030 ns	7.158 ns
$F_{\max}$	85 MHz	77.1 MHz	77.87 MHz

Table 6: Overview of the results

SC-FDMA have a higher data rate than the single-carrier implementation, however this was a design decision, and the single-carrier data rate could easily be increased, but at the cost of increased bandwidth. The more interesting comparison here is the spectral efficiency, which is a measure of the data rate per bandwidth, where it can be seen that OFDM and SC-FDMA are somewhat more efficient. This is because the use of an RRC pulse shaping filter with $\beta = 0.35$ gives a 35% increase in bandwidth, whereas the bandwidth of OFDM and SC-FDMA is given by the number of data carriers plus pilots, which only gives an increase of $1 - \frac{256}{256+10} \approx 3.8\%$.

Another important comparison is on the suitability of the implementation for wireless transmission, which is a system requirement introduced in chapter 2.1. As discussed in chapter 4.2.3, the single-carrier implementation experiences a frequency-selective channel in most indoor environments, and so its suitability for wireless transmission is dependent upon a solution to avoid intersymbol interference (ISI). In this specific project, no such solution was implemented, and as such the single-carrier implementation does not meet this requirement. Comparing that to OFDM and SC-FDMA, which each experience narrowband fading, and only require the use of a cyclic prefix of length longer than the delay spread to avoid ISI, which were implemented, though it would be necessary to experimentally verify that the length of the cyclic prefix is

sufficient, and if it is not, the length of the cyclic prefix can be increased with ease.

Comparing the results of the actual transmission, it is seen that they all fulfill the requirement of zero errors within the payload test, though it should be noted they have very different EVM. Since the symbols were not transmitted wirelessly, the errors are purely from the digital implementation, the digital to analog conversion, the sampling using the AD2, and the decoding. Aside from the sampling with the AD2, these all occur during actual wireless transmissions, and as such give a sort of baseline error. In a wireless test, additional errors would be expected to be introduced in addition to the measured ones.

It can be seen that OFDM experiences significantly higher PAPR, which is also what the theory states [32], whereas single-carrier and SC-FDMA experience similar PAPRs, with SC-FDMA being slightly higher.

Finally, inspecting the resources utilization and timing requirements, it is seen that all three modulation schemes fit well within the FPGA, with the single-carrier implementation using significantly fewer resources, as is expected due to its simplicity. SC-FDMA has a steep increase in resource utilization over OFDM, especially in terms of DSP usage, which experiences a $102/57 - 1 = 79\%$ increase, which matches very well with expectation, as the DSP usage is dominated by the FFT stages, and OFDM consists of 10 stages, whereas SC-FDMA consists of 8 precoding FFT stages and the 10 OFDM stages, giving an increase of $18/10 - 1 = 80\%$. The slack for the single-carrier implementation is also larger than either OFDM and SC-FDMA, which are essentially the same¹¹, but they all yield a maximum frequency of more than the required 50 MHz, and as such they all meet this requirement.

The last two requirements set out in chapter 2.1, on modularity and equal evaluation, are both met, though they are not measurable. The entire architecture is based on a modular system, where each component is agnostic to all other modules (except the controller), and as such, they can both be easily replaced or extended, and tested independently, as was done with the FFT module in section 4.3.1.

In summary, all three implemented digital architectures meet the 50 MHz timing requirement, with maximum operating frequencies above 77 MHz for the OFDM and SC-FDMA designs and 85 MHz for the single-carrier design. All architectures also fit within the available resources of the DE25 FPGA. The payload test was decoded without errors for all three architectures, satisfying the functional verification requirement for the tested payload. The modularity requirement was met through the shared valid-signal-based architecture, allowing the baseband implementation to be exchanged while reusing the data source, symbol shaper, controller, and DAC interface.

As shown above, both OFDM and SC-FDMA meet all the relevant requirements, while the single-carrier implementation does not meet the requirement of suitability for wireless transmission due to ISI, which would have to be solved, for example by implementing equalization in order to make the channel appear flat [30]. However, in modern high-throughput systems, this is generally not a utilized modulation, with OFDM and SC-FDMA generally being favored, such as wifi which generally use OFDM [4], or cellular which use both OFDM and SC-FDMA [5]. The general trade-off between the two is complexity and resource utilization versus PAPR, and in systems that are battery-powered, the increased complexity of SC-FDMA is generally considered worth it, as SC-FDMA has less power consumption due to more efficient amplifier use [5]. In this project the increase of compute resources, with an 80% increase in DSP usage, is likely not worth it, as it limits what else can be executed on the FPGA alongside the modulation scheme, such as error correction.

As mentioned, the PCBs were not delivered on time. As such the RF front-end was not experimentally verified, and the discussion is limited to whether the design, as specified, satisfies the requirements set out in chapter 2.1. The two relevant requirements are on support of complex baseband signals and modularity. For the first, the RF front end was designed with a dual data path in mind, with a set of low-pass filters and fully differential amplifiers each for the in-

¹¹One may notice that SC-FDMA has a slightly larger slack, which means it actually takes less time. This may be counterintuitive, as it has a significant increase in resource utilization, but it's not entirely unexpected, as the routing process, which determines the slack, is not determinate, and so it might just have found a better local maximum.

phase and quadrature components which merge in the quadrature modulator. In the PCB design there was a focus on ensuring the in-phase and quadrature components had as equivalent a path as possible to minimize distortions. Thus this requirement, at least in the design, was fulfilled. For the requirement of modularity, it was a focus within reason, with each part designed independently. It was not realistic to make a PCB for each component, and so a compromise was struck to make the most complicated part, the PLL, its own PCB, with the rest on another, which allows for easier debugging and testing of each. For the undesigned output stage, this would likely also be its own PCB, as its development would happen after the main RF front-end has been tested and verified. As such, the design of the RF front-end as specified meets the requirement of modularity.

The measurements performed in this project test the implemented digital transmitter chains up to, and including, the DAC output, and all three implementations satisfied the verification requirements within these limits, with a payload error rate of 0 for all of them, and decent EVM performance, however this does not represent the performance of a full SDR transmitter, which also includes the RF front-end and wireless channel. Instead, the payload error rates and EVMs represent the FPGA implementation of the modulation schemes, fixed point quantization noise, digital to analog conversion, sampling and decoding. The results thus verify that the implemented digital baseband SDR on the FPGA work as expected, and fit within the confines of the resource and timing requirements.

Another limitation of the verification setup is the use of a trigger for simplified synchronization. This allows the decoder to identify exactly when the transmitted symbols start, reducing the complexity by removing the need for a synchronization step. In a real wireless SDR, the decoder would need to estimate the start of a symbol, for example by using the cyclic prefix to look for samples separated by the symbol length with high correlation. Additionally, pilots embedded in the symbols were used to estimate the channel, which requires the channel to be time-invariant over the symbol length, which is not always a given. As such, a real wireless SDR would also need to implement channel tracking as well as robust synchronization.

The project as a whole is focused on the PHY transmitter part of the SDR, but a real SDR system would also need to implement things like framing, communication protocols and adaptive modulation schemes. However, these issues were deemed out of scope, and as such were not considered for the project.

It is worth discussing the EVM measurements for OFDM and SC-FDMA more closely, as the implementations are very similar, but one yields a significantly lower EVM, 6 dB, or about half in linear terms. This likely arises from quantization noise due to the need for additional back-off in OFDM compared to SC-FDMA, which leads to less of the available DAC resolution for describing the signal. Each additional bit of back-off effectively reduces the usable DAC resolution by one bit, corresponding to approximately a 6 dB increase in quantization noise.

While there are many possible future work items, there are four that very naturally extend from the completed work;

- Testing and verification of RF front-end
- Development and implementation of the RF output stage
- Error correction
- Interfacing between the FPGA and HPS

The most obvious is to test and verify the RF front-end, as well as implementing the RF output stage, as this will allow the SDR to transmit wirelessly, turning it into an essentially fully functional SDR transmitter. The work that would need to be done in order to complete the testing and verification of the RF front-end, would be to develop a testing framework for each part, the general front-end and the PLL, and then iterate on them until they function as expected. Once they work as expected, the requirements for the RF output stage can be stated, such as the power limits both within and without the allocated frequency band, allowing the

development of an appropriate band-pass filter, which would likely be based on a Butterworth structure, similarly to the low-pass reconstructive filter, due to its maximally flat nature.

A further natural extension on the FPGA would be to implement a Forward Error Correction (FEC) algorithm, such as a convolutional code or Reed-Solomon, potentially combined with interleaving to combat deep fades [41]. Depending on the implementation details of these coding schemes, it might be necessary to either optimize the FFT implementation, or only use OFDM, as the required resources might approach the limit, especially with SC-FDMAs large DSP requirement.

Lastly, the DE25 includes a Hard Processor System, which is just a standalone CPU, which is capable of running Linux. In a full SDR transmitter, it would make sense to implement an interface between the HPS and FPGA, allowing the HPS to handle things like protocols and framing, such that the FPGA only does things it's good at, which is highly-parallelized workloads. Implementing this interface would also allow the HPS to run a server that could be connected to the internet, potentially enabling remote requests for transmission. This interface would likely consist of an amount of RAM, split into data and control signals, the data part of the RAM would then be implemented as circular RAM. The control region would contain parameters such as modulation order, start/stop flags, payload length, and selected waveform, while the circular data buffer would allow the HPS to continuously stream payload data to the FPGA without tightly coupling the software timing to the sample-rate-critical transmitter pipeline.

Together, these extensions would transform the current verified digital transmitter prototype into a practically usable SDR platform capable of configurable wireless transmission.

7 Conclusion

The project produced a functional digital SDR prototype based on a highly modular and parameterized design, which allows for easy future extensions and simple changes to parameters. Both OFDM and SC-FDMA are based on a custom FFT/IFFT core developed from scratch in VHDL, rather than an FFT IP core provided by Altera or Intel¹². This gives direct control over the data path, fixed-point formats, memory usage, and module interfaces. As such, almost any parameter of the three implemented digital baseband signal chains can be changed, including the FFT size, MQAM order, cyclic prefix length and number of pilots. This modular architecture also allowed for easy verification and debugging of individual modules, including the behavior of the DAC, which was modelled as a high-pass filter with a critical frequency of 115 kHz.

The single-carrier implementation works, and while it is not entirely suitable for wireless transmission, is a simple and light-weight solution that can be extended to work decently in wireless systems, using for example equalization [30].

The OFDM implementation works, with a very small EVM of -36.2 dB, and is suitable for frequency-selective channels, with a bin width of 48.8 kHz, allowing its use in channels with a delay spread of up to about 2 μ s using the rule of thumb that $T_s > 10\sigma_{T_m}$, which can easily be increased to 8 μ s by changing the FFT size from 1024 to 4096. The cyclic prefix would also need to change correspondingly, however this is also easily done by changing a parameter in the initialization of the module.

Similarly, the SC-FDMA implementation was equally successful, with an even smaller EVM of about -42.8 dB, and is also usable in frequency-selective channels. It improves on the issue of high peak-to-average power that plagues OFDM, however this comes with the trade-off of higher resource demands and more complicated implementation.

The requirements set out in chapter 2.1 were met by all three digital implementations with respect to timing closure, resource utilization, and zero-error decoding of the tested payload. All three designs operated above the required 50 MHz clock frequency and fit within the available resources of the DE25 FPGA. However, the RF front-end was only designed and documented, not experimentally tested, because the manufactured PCBs were not delivered in time.

¹²In FPGA development, pre-designed reusable blocks are referred to as IP cores/blocks

The main limitations of the project are therefore the missing RF front-end verification, the lack of a designed and tested output stage, the absence of error correction, and the deterministic payload of the test, which can be solved by implementing an interface between the FPGA and HPS, which would also allow easier implementation of communication protocols on the CPU.

8 References

- [1] Wireless Innovation Forum & IEEE P1900.1 group. *What is a Software Defined Radio*. URL: <https://www.wirelessinnovation.org/assets/documents/SoftwareDefinedRadio.pdf>.
- [2] *DE25-Standard_User_manual_revD*. Terasic. 2025. URL: <https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=115&No=1354&PartNo=4#contents>.
- [3] Electronic Communications Committee (ECC) within the European Conference of Postal and Telecommunications Administrations (CEPT). *THE EUROPEAN TABLE OF FREQUENCY ALLOCATIONS AND APPLICATIONS IN THE FREQUENCY RANGE 8.3 kHz to 3000 GHz (ECA TABLE)*. URL: <https://docdb.cept.org/download/4385>.
- [4] IEEE SA. *IEEE 802.11be-2024*. URL: <https://standards.ieee.org/ieee/802.11be/7516>.
- [5] Gilberto Berardinelli et al. “OFDMA vs. SC-FDMA: Performance Comparison in Local Area IMT-A Scenarios”. In: *IEEE Wireless Communications* 15.5 (2008), pp. 64–72.
- [6] Kenneth L. Short. “VHDL for Engineers”. In: 1st Edition. Pearson, 2009. Chap. 1.1.
- [7] Terasic. *DE25-Standard Development Kit product photo*. URL: https://www.terasic.com.tw/attachment/archive/1354/image/DE25-Standard_verC_45.jpg.
- [8] Randal E. Bryant and David R. O’Hallaron. “Computer Systems: A Programmer’s Perspective”. In: 3rd Edition. Pearson, 2016. Chap. 4.
- [9] Randal E. Bryant and David R. O’Hallaron. “Computer Systems: A Programmer’s Perspective”. In: 3rd Edition. Pearson, 2016. Chap. 8.2.
- [10] Taron Foxworth. *Hardware Fundamentals: what exactly is a microcontroller?* URL: <https://medium.com/free-code-camp/hardware-fundamentals-what-exactly-is-a-microcontroller-8a502a3650dc>.
- [11] Ian Smalley Josh Schneider. *What is a field programmable gate array (FPGA)?* URL: <https://www.ibm.com/think/topics/field-programmable-gate-arrays>.
- [12] Altera. *Quartus® Software Latest Version for Device Support*. URL: <https://www.altera.com/design/guidance/software/device-support>.
- [13] Digilent. *Verilog vs VHDL: Differences & FPGA*. URL: <https://digilent.com/blog/battle-over-the-fpga-vhdl-vs-verilog-who-is-the-true-champ/>.
- [14] Kenneth L. Short. “VHDL for Engineers”. In: 1st Edition. Pearson, 2009. Chap. 2.6.
- [15] Randal E. Bryant and David R. O’Hallaron. “Computer Systems: A Programmer’s Perspective”. In: 3rd Edition. Pearson, 2016. Chap. 2.2.
- [16] Randal E. Bryant and David R. O’Hallaron. “Computer Systems: A Programmer’s Perspective”. In: 3rd Edition. Pearson, 2016. Chap. 2.2.
- [17] Greg Baker. *The std_logic type*. URL: https://www2.cs.sfu.ca/~ggbaker/reference/std_logic/1164/std_logic.html.
- [18] Dimitris G. Manolakis and Vinay K. Ingle. “Applied Digital Signal Processing, THEORY AND PRACTICE”. In: 1st Edition. Cambridge University Press, 2011. Chap. 8.
- [19] Kenneth L. Short. “VHDL for Engineers”. In: 1st Edition. Pearson, 2009. Chap. 1.5.
- [20] *THDB-ADA_UserGuide*. Terasic. 2015. URL: <https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=67&No=278&PartNo=3#contents>.
- [21] *ada_v11*. Terasic. 2015. URL: <https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=67&No=278&PartNo=3#contents>.

- [22] Simon Haykin and Michael Moher. “Communication Systems”. In: 5th Edition. John Wiley & Sons, Inc., 2009. Chap. 12.5.
- [23] Andrea Goldsmith. “Wireless Communications”. In: Draft of 2nd Edition. 2020. Chap. 5.3.
- [24] Andrea Goldsmith. “Wireless Communications”. In: Draft of 2nd Edition. 2020. Chap. 6.1.
- [25] Andrea Goldsmith. “Wireless Communications”. In: Draft of 2nd Edition. 2020. Chap. 5.5, 11.3.
- [26] Dimitar Marinov. *Part 5: Polyphase FIR filters*. URL: <https://vhdlwhiz.com/part-5-polyphase-fir-filters/>.
- [27] Dimitris G. Manolakis and Vinay K. Ingle. “Applied Digital Signal Processing, THEORY AND PRACTICE”. In: 1st Edition. Cambridge University Press, 2011. Chap. 4.
- [28] Keysight. *Making and Interpreting EVM Measurements*. URL: <https://www.keysight.com/us/en/assets/7018-01305/application-notes/5989-3144.pdf>.
- [29] Richard van Nee. *Delay Spread Requirements for Wireless Networks in the 2.4 GHz and 5 GHz Bands*. URL: https://www.ieee802.org/11/Documents/DocumentArchives/1997_docs/71252.pdf.
- [30] Andrea Goldsmith. “Wireless Communications”. In: Draft of 2nd Edition. 2020. Chap. 11.4.
- [31] MIT Vladimir Stojanović. *Fast Fourier Transform: VLSI Architectures*. URL: https://ocw.mit.edu/courses/6-973-communication-system-design-spring-2006/1460f43d3993b7c956d4bb8ee03d1fb0_lecture_10.pdf.
- [32] Andrea Goldsmith. “Wireless Communications”. In: Draft of 2nd Edition. 2020. Chap. 12.4.
- [33] Andrea Goldsmith. “Wireless Communications”. In: Draft of 2nd Edition. 2020. Chap. 4.1.
- [34] Analog Devices. *CHAPTER 8 ANALOG FILTERS*. URL: <https://www.analog.com/media/en/training-seminars/design-handbooks/basic-linear-design/chapter8.pdf>.
- [35] Martin E. Meserve. *Butterworth Low/High Pass L/C Filter Calculator*. URL: https://k7mem.com/Fil_LC.html.
- [36] *THS4552: Dual-Channel, Low-Noise, Precision, 150-MHz, Fully Differential Amplifier*. THS4552. Texas Instruments. URL: <https://www.ti.com/lit/ds/symlink/ths4552.pdf>.
- [37] Andrea Goldsmith. “Wireless Communications”. In: Draft of 2nd Edition. 2020. Chap. 2.2.
- [38] *ADL5375: 400 MHz to 6 GHz Broadband Quadrature Modulator*. ADL5375. Analog Devices. URL: <https://www.analog.com/media/en/technical-documentation/data-sheets/adl5375.pdf>.
- [39] *ADF4351: Wideband Synthesizer with Integrated VCO*. ADF4351. Analog Devices. URL: <https://www.analog.com/media/en/technical-documentation/data-sheets/ADF4351.pdf>.
- [40] *EVAL-ADF4351*. Analog Devices. URL: <https://www.analog.com/en/resources/evaluation-hardware-and-software/evaluation-boards-kits/eval-adf4351.html>.
- [41] Andrea Goldsmith. “Wireless Communications”. In: Draft of 2nd Edition. 2020. Chap. 8.

Appendix A: Simple VHDL example

The DE25 has both a series of switches and small LEDs. Imagine you want to show whether two of the switches are the same. The flow would go something like this: switch1 = switch2 => LED. The below is a code snippet of this flow:

```
library ieee;
use ieee.std_logic_1164.all;

entity SW_COMPARE is
    port (
        switches : in std_logic_vector(1 downto 0);
        LED : out std_logic);
end entity;

architecture RTL of SW_COMPARE is
begin
    LED <= '1' when switches(1) = switches(0) else '0';
end RTL;
```

Let's go through this line by line. Firstly we import a library called IEEE, and use a specific package from there that includes something called std_logic, which, in this case, can be thought of as a boolean¹³. Next, in VHDL, there is no such thing as classes, instead there are entities, which can be thought of as custom logic blocks that have some input and some output. That is what we declare in the port section. We have an input named switches, which is a vector of std_logics. So we have a vector of 2 booleans, which corresponds to the two switches. Then there is an output named LED, which is a boolean. This one is tied to the LED on the board. Next there is an architecture block, which is where the actual logic lives. In this case we say "LED is 1 if switch1 = switch2 otherwise 0", so whenever either switch updates, LED will also update. This is combinational logic.

Let's recreate the same example, but as behavioural, which is synchronous. This means we only update LED on the rising edge of a clock cycle:

```
library ieee;
use ieee.std_logic_1164.all;

entity SW_COMPARE is
    port (
        CLK : in std_logic;
        switches : in std_logic_vector(1 downto 0);
        LED : out std_logic);
end entity;

architecture RTL of SW_COMPARE is
begin
    process(CLK)
    begin
        if rising_edge(CLK) then
            LED <= '1' when switches(1) = switches(0) else '0';
        end if;
    end process;
end RTL;
```

Again, we import the library, and declare our entity, except this time we have an additional input, a clock. This clock input is also just an std_logic, so a boolean. In the architecture

¹³It actually includes 9 states, but all the others are only useful in simulation and synthesis

section we have something called a process, which is, as it sounds, a process that runs upon changes to what is called its sensitivity list, which is what is in the parentheses, so in this case the clock. We then say: "If we're on the rising edge of the clock, assign LED 1 if switch1 = switch2 otherwise 0". This may sound silly, but it is an extremely powerful tool to ensure that all our logic updates only when we desire, to avoid things like race conditions. This project will be almost exclusively based on behavioural architecture as an SDR is inherently time-dependent, as we require all inputs and outputs from the system to happen at the same regular interval, which leans itself naturally into using a clocked approach. On top of this, it is essentially impossible to keep all the internal modules aligned in time in a combinational approach, and so behavioural is required.

Once you have some working VHDL code, there are a few more steps to go through, before you can actually get it onto the board. Firstly, you have to assign your inputs and outputs to actual I/O on your board. What I/O corresponds to which pins depend heavily on your specific FPGA or Development and Education board, but for the DE25, you can find the relevant pins by looking in the manual. The pin assignments for the switches, buttons and LEDs can be found on page 28, but for this example, switch 0 is PIN_BM62, switch 1 is PIN_BP62, and LEDR0 is PIN_CC71. For the synchronous example, we also need to assign the clock. The DE25 contains several clocks, most of which are 50MHz, but for this example we will use Clock0_50, which is PIN_CH128. To actually assign these pins in Quartus Prime Pro, we use the pin planner tool. Here we also need to assign the I/O standard for the pins. The manual also says what to use, for example all the switches, buttons and LEDs use 1.2V, where as the clock uses 3.3V. Later we will also see examples of pins using 3.3V LVCMOS.

Once all the pins have been assigned, we're finally ready to actually put our code onto the FPGA. How exactly this is done will depend on which exact board you have, but it will always say in the manual. For the DE25, we need to set some mode select pins to a specific configuration (111), to put the FPGA into JTAG mode. This allows us to flash the FPGA with a program. To do this, we use the programmer tool in Quartus Prime Pro. If you just want to flash it once, where it loses the program upon losing power, then it is fairly straight forward, and we can just program it directly with the compiled file. It is possible to flash it in a persistent way, by using its non-volatile program memory called QSPI flash.

Appendix B: FFT Controller - VHDL

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

use work.fft_pkg.all;
entity FFT is
  generic(
    logN : integer := 12);
  port(
    CLK : in std_logic;
    INPUT_RE : in std_logic_vector(31 downto 0);
    INPUT_IM : in std_logic_vector(31 downto 0);
    OUTPUT_RE : out std_logic_vector(31 downto 0);
    OUTPUT_IM : out std_logic_vector(31 downto 0);
    VALID_IN : in std_logic;
    VALID_OUT : out std_logic;

    RST : in std_logic;
    MODE : in std_logic); -- DECIDES FFT (1) / IFFT (0)
end FFT;
architecture RTL of FFT is

  -- Input samples are written in bit-reversed order
  -- for a radix-2 DIT FFT/IFFT implementation,
  -- so the final output can be produced in natural order.

  constant N : integer := 2**logN;
  constant STAGES : integer := logN;

  type ram_N is array (0 to N-1) of word64;

  signal RAM_A : ram_N;

  signal RAM_B : ram_N;

  attribute ramstyle : string;
  attribute ramstyle of RAM_A : signal is "M20K";
  attribute ramstyle of RAM_B : signal is "M20K";

  signal RAM_WRT_SEL : std_logic := '0'; -- 0 => A WRITE, 1 => B WRITE
  signal RAM_READ_SEL : std_logic := '0'; -- 0 => A READ, 1 => B READ

  signal READ_PTR : integer range 0 to N-1 := 0;
  signal WRT_PTR : integer range 0 to N-1 := 0;

  signal VALID_WRT : std_logic;

  type c32_array is array (0 to STAGES) of complex_S32;
  type sl_array is array (0 to STAGES) of std_logic;
```

```

signal STAGE_DATA : c32_array;
signal STAGE_VALID : sl_array := (others => '0');

begin

    assert logN <= 12
    report "FFT: N exceeds twiddle table size"
    severity failure;

    gen_stages : for i in 0 to (logN-1) generate
        stage_i : entity work.FFT_stage
            generic map (
                S => i,
                logN => logN
            )
            port map (
                CLK      => CLK,
                MODE      => MODE,
                VALID_IN  => STAGE_VALID(i),
                VALID_OUT => STAGE_VALID(i+1),
                RST       => RST,
                DIN       => STAGE_DATA(i),
                DOUT      => STAGE_DATA(i+1)
            );
        end generate;
    process(CLK)
        variable w : word64;
    begin
        if rising_edge(CLK) then
            if (RST = '1') then
                READ_PTR <= 0;
                WRT_PTR <= 0;
                RAM_WRT_SEL <= '0';
                RAM_READ_SEL <= '0';
                VALID_WRT <= '0';
                VALID_OUT <= '0';
            else
                if (VALID_IN = '1') then
                    if (RAM_WRT_SEL = '0') then
                        RAM_A(bit_reverse(WRT_PTR,logN)) <= INPUT_RE & INPUT_IM;
                    else
                        RAM_B(bit_reverse(WRT_PTR,logN)) <= INPUT_RE & INPUT_IM;
                    end if;
                    if (WRT_PTR = (N-1)) then
                        WRT_PTR <= 0;
                        VALID_WRT <= '1';
                        RAM_WRT_SEL <= not RAM_WRT_SEL;
                    else
                        WRT_PTR <= WRT_PTR + 1;
                    end if;
                end if;
            end if;
        end if;
    end process;
end;

```

```

if (VALID_WRT = '1') then
  if (RAM_READ_SEL = '0') then
    w := RAM_A(READ_PTR);
  else
    w := RAM_B(READ_PTR);
  end if;
  STAGE_DATA(0).re <= signed(w(63 downto 32));
  STAGE_DATA(0).im <= signed(w(31 downto 0));
  STAGE_VALID(0) <= '1';

  if (READ_PTR = (N-1)) then
    READ_PTR <= 0;
    RAM_READ_SEL <= not RAM_READ_SEL;
    if (WRT_PTR < (N-1)) then
      VALID_WRT <= '0';
    end if;
  else
    READ_PTR <= READ_PTR + 1;
  end if;
else
  STAGE_VALID(0) <= '0';
end if;

if (STAGE_VALID(logN) = '1') then
  VALID_OUT <= '1';
  OUTPUT_RE <= std_logic_vector(STAGE_DATA(logN).re);
  OUTPUT_IM <= std_logic_vector(STAGE_DATA(logN).im);
else
  VALID_OUT <= '0';
end if;
end if;
end process;
end RTL;

```

Appendix C: FFT Stage - VHDL

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

use work.fft_pkg.all;

entity FFT_stage is

    generic (
        S : integer;
        logN : integer);
    port (
        CLK : in std_logic;
        MODE : in std_logic; -- DECIDES FFT (1) / IFFT (0)
        VALID_IN : in std_logic;
        VALID_OUT : out std_logic;
        RST : in std_logic;
        DIN : in complex_S32;
        DOUT : out complex_S32);

end FFT_stage;

architecture RTL of FFT_stage is
    constant DELAY : integer := 2**S;
    constant N : integer := 2**logN;
    constant STRIDE : integer := N/(2*DELAY);

    constant FIFO_SIZE : integer := DELAY;

    type delay_ram_t is array (0 to DELAY-1) of word64;
    type FIFO_ram_t is array (0 to FIFO_SIZE-1) of word64;
    signal DELAY_MEM : delay_ram_t;

    signal FIFO : FIFO_ram_t;

    attribute ramstyle : string;
    attribute ramstyle of DELAY_MEM : signal is "M20K";
    attribute ramstyle of FIFO : signal is "M20K";

    signal SAMPLE_CNT : integer range 0 to (2*DELAY-1);

    signal WRT_PTR : integer range 0 to FIFO_SIZE-1 := 0;
    signal READ_PTR : integer range 0 to FIFO_SIZE-1 := 0;

    signal FIFO_CNT : integer range 0 to FIFO_SIZE := 0;

begin
    assert S < logN
    report "FFT_stage: S must be < logN"
    severity failure;

    process(CLK)
        variable A : complex_S32;
```

```

variable B : complex_S32;
variable T : complex_S32;
variable W : complex_S16;

variable Y0 : complex_S32;
variable Y1 : complex_S32;

variable FIFO_DELTA : integer range -1 to 2 := 0;

variable TW_CNT : integer range 0 to (DELAY-1) := 0;

variable bw : word64;
variable fw : word64;

begin
  if rising_edge(CLK) then
    FIFO_DELTA := 0;
    -- RESET LOGIC
    if RST = '1' then
      SAMPLE_CNT <= 0;
      VALID_OUT <= '0';
      DOUT <= (re => (others => '0'), im => (others => '0'));
      WRT_PTR <= 0;
      READ_PTR <= 0;
      FIFO_CNT <= 0;
    else
      if (VALID_IN = '1') then
        -- DELAY THE FIRST HALF OF SAMPLES
        if (SAMPLE_CNT < DELAY) then
          DELAY_MEM(SAMPLE_CNT) <= std_logic_vector(DIN.re) & std_logic_vector(DIN.im);
        else
          TW_CNT := SAMPLE_CNT - DELAY;
          bw := DELAY_MEM(TW_CNT);
          A.re := signed(bw(63 downto 32));
          A.im := signed(bw(31 downto 0));
          B := DIN;
          W := twiddle(TW_CNT * STRIDE, MODE, N);
          T := multc(B,W);

          -- SHIFT RIGHT IF IFFT TO SCALE DOWN
          if MODE = '0' then
            Y0 := shift_right_c32(add_c32(A,T),1);
            Y1 := shift_right_c32(sub_c32(A,T),1);
          else
            Y0 := add_c32(A,T);
            Y1 := sub_c32(A,T);
          end if;

          DOUT.re <= Y0.re;
          DOUT.im <= Y0.im;

          FIFO(WRT_PTR) <= std_logic_vector(Y1.re) & std_logic_vector(Y1.im);

          WRT_PTR <= (WRT_PTR + 1) mod FIFO_SIZE;
        end if;
      end if;
    end if;
  end if;
end begin;

```

```

        FIFO_DELTA := FIFO_DELTA + 1;
        VALID_OUT <= '1';
    end if;
    SAMPLE_CNT <= (SAMPLE_CNT + 1) mod (2*DELAY);
end if;

-- OUTPUT LOGIC
if (SAMPLE_CNT < DELAY or VALID_IN = '0') then
    if (FIFO_CNT > 0) then

        DOUT.re <= signed(FIFO(READ_PTR)(63 downto 32));
        DOUT.im <= signed(FIFO(READ_PTR)(31 downto 0));

        VALID_OUT <= '1';
        FIFO_DELTA := FIFO_DELTA - 1;
        READ_PTR <= (READ_PTR + 1) mod FIFO_SIZE;
    else
        VALID_OUT <= '0';
    end if;
end if;
assert FIFO_CNT + FIFO_DELTA <= FIFO_SIZE
report "FFT FIFO overflow"
severity failure;

assert FIFO_CNT + FIFO_DELTA >= 0
report "FFT FIFO underflow"
severity failure;

FIFO_CNT <= FIFO_CNT + FIFO_DELTA;
end if;
end if;
end process;
end RTL;

```

Appendix D: FFT Package - VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

package fft_pkg is

    subtype S16 is signed(15 downto 0);
    subtype S32 is signed(31 downto 0);
    subtype S48 is signed(47 downto 0);

    type complex_S16 is record
        re : S16;
        im : S16;
    end record;

    type complex_S32 is record
        re : S32;
        im : S32;
    end record;

    type complex_S48 is record
        re : S48;
        im : S48;
    end record;

    subtype word64 is std_logic_vector(63 downto 0);
    subtype word32 is std_logic_vector(31 downto 0);
    subtype word128 is std_logic_vector(127 downto 0);

    constant LUT_SIZE : integer := 4096;
    type lut_t is array(0 to LUT_SIZE-1) of std_logic_vector(15 downto 0);

    constant COS_LUT : lut_t := (..) -- 4096 samples of Q1.15 Cos
    constant SIN_LUT : lut_t := (..) -- 4096 samples of Q1.15 Sin

    attribute romstyle : string;
    attribute romstyle of COS_LUT : constant is "M20K";
    attribute romstyle of SIN_LUT : constant is "M20K";

    function twiddle(x : integer; m : std_logic; N : integer) return complex_S16;
    function bit_reverse(x : integer; logN : integer) return integer;
    function add_c32(x : complex_S32; y : complex_S32) return complex_S32;
    function sub_c32(x : complex_S32; y : complex_S32) return complex_S32;
    function multc(x : complex_S32; tw : complex_S16) return complex_S32;
    function shift_right_c32(x : complex_S32; s : integer) return complex_S32;
    function saturate_signed(x : signed; s : integer) return signed;

end package fft_pkg;
```

```

package body fft_pkg is

function twiddle(x : integer; m : std_logic; N : integer) return complex_S16 is
    variable y : complex_S16;
    variable w : integer := x*(LUT_SIZE/N);
begin
    y.re := signed(COS_LUT(w));
    if (m = '0') then
        y.im := signed(SIN_LUT(w));
    else
        y.im := -signed(SIN_LUT(w));
    end if;
    return y;
end twiddle;

function bit_reverse(x : integer; logN : integer) return integer is
    variable x_u : unsigned(logN-1 downto 0);
    variable y_u : unsigned(logN-1 downto 0);
begin
    x_u := to_unsigned(x, logN);

    for i in 0 to logN-1 loop
        y_u(i) := x_u(logN - 1 - i);
    end loop;

    return to_integer(y_u);
end bit_reverse;

function add_c32(x : complex_S32; y : complex_S32) return complex_S32 is
    variable u : complex_S32;
begin
    u.re := saturate_signed(resize(x.re,33) + resize(y.re,33),32);
    u.im := saturate_signed(resize(x.im,33) + resize(y.im,33),32);

    return u;
end add_c32;

function sub_c32(x : complex_S32; y : complex_S32) return complex_S32 is
    variable u : complex_S32;
begin
    u.re := saturate_signed(resize(x.re,33) - resize(y.re,33),32);
    u.im := saturate_signed(resize(x.im,33) - resize(y.im,33),32);
    return u;
end sub_c32;

function multc(x : complex_S32; tw : complex_S16) return complex_S32 is
    variable u_re : S48;
    variable u_im : S48;
    variable v_re : S48;
    variable v_im : S48;
    variable re_wide : signed(48 downto 0);
    variable im_wide : signed(48 downto 0);
    variable y : complex_S32;
begin

```

```

u_re := x.re * tw.re;
u_im := x.re * tw.im;
v_re := x.im * tw.im;
v_im := x.im * tw.re;

re_wide := resize(u_re, 49) - resize(v_re, 49);
im_wide := resize(u_im, 49) + resize(v_im, 49);

y.re := saturate_signed(shift_right(re_wide,15),32);
y.im := saturate_signed(shift_right(im_wide,15),32);

return y;
end multc;

function shift_right_c32(x : complex_S32; s : integer) return complex_S32 is
    variable u : complex_S32;
begin
    u.re := shift_right(x.re,s);
    u.im := shift_right(x.im,s);
    return u;
end shift_right_c32;

function saturate_signed(x : signed; s : integer) return signed is
    constant MAX : signed(s-1 downto 0) := (s-1 => '0', others => '1');
    constant MIN : signed(s-1 downto 0) := (s-1 => '1', others => '0');
begin
    if (x > resize(MAX,x'length)) then
        return MAX;
    elsif (x < resize(MIN,x'length)) then
        return MIN;
    else
        return resize(x,s);
    end if;
end saturate_signed;
end fft_pkg;

```

Appendix E: Data Source - VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity DATA is
  generic(
    DATA_WIDTH : integer := 2 -- Bits per I/Q per symbol
  );

  port(
    CLK : in std_logic;
    RST : in std_logic;

    RDY : in std_logic;

    VALID_OUT : out std_logic;

    OUTPUT_RE : out std_logic_vector(DATA_WIDTH-1 downto 0);
    OUTPUT_IM : out std_logic_vector(DATA_WIDTH-1 downto 0));
end DATA;

architecture RTL of DATA is

  constant LOREM : std_logic_vector(0 to 8191) := (...);

  constant N : integer := LOREM'length;

  constant STRIDE : integer := (DATA_WIDTH*2);

  signal INDEX : integer range 0 to N-1 := 0;

begin

  process(CLK)
    variable TMP_RE : std_logic_vector(DATA_WIDTH-1 downto 0);
    variable TMP_IM : std_logic_vector(DATA_WIDTH-1 downto 0);
  begin
    if rising_edge(CLK) then
      if (RST = '1') then
        INDEX <= 0;
        VALID_OUT <= '0';

        OUTPUT_RE <= (others => '0');
        OUTPUT_IM <= (others => '0');
      else
        if (RDY = '1') then

          for i in 0 to DATA_WIDTH-1 loop
            TMP_RE(DATA_WIDTH-1-i) := LOREM((INDEX+i) mod N);
            TMP_IM(DATA_WIDTH-1-i) := LOREM((INDEX+DATA_WIDTH+i) mod N);
          end loop;
```

```

        OUTPUT_RE <= TMP_RE;
        OUTPUT_IM <= TMP_IM;
        VALID_OUT <= '1';
        INDEX <= (INDEX + STRIDE) MOD N;
    else
        OUTPUT_RE <= (others => '0');
        OUTPUT_IM <= (others => '0');
        VALID_OUT <= '0';
    end if;
end if;
end if;
end process;
end RTL;

```

Appendix F: Symbol Shaper - VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Symbol_Shaper is
  generic(
    logQAM : integer := 4 -- Which QAM. Default 16QAM
  );
  port(
    CLK : in std_logic;
    RST : in std_logic;

    INPUT_RE : in std_logic_vector((logQAM/2)-1 downto 0);
    INPUT_IM : in std_logic_vector((logQAM/2)-1 downto 0);
    VALID_IN : in std_logic;

    OUTPUT_RE : out std_logic_vector(31 downto 0);
    OUTPUT_IM : out std_logic_vector(31 downto 0);
    VALID_OUT : out std_logic
  );
end entity;

architecture RTL of Symbol_Shaper is

  constant MQAM : integer := 2**logQAM;
  constant L : integer := 2**(logQAM/2);

  subtype Q2_22_t is signed(23 downto 0);

  type QAM is array (natural range <>) of Q2_22_t; -- Stored as Q2.22, so 1 signed bit, 1 in
  constant QAM16 : QAM(0 to 3) := (
    to_signed(-3979065, 24), to_signed(-1326355, 24), to_signed(1326355, 24), to_signed(3979065, 24)
  );

  constant QAM64 : QAM(0 to 7) := (
    to_signed(-4530365, 24), to_signed(-3235975, 24), to_signed(-1941585, 24), to_signed(-647195, 24),
    to_signed(647195, 24), to_signed(1941585, 24), to_signed(3235975, 24), to_signed(4530365, 24)
  );

  constant QAM256 : QAM(0 to 15) := (
    to_signed(-4825325, 24), to_signed(-4181949, 24), to_signed(-3538572, 24), to_signed(-2895195, 24),
    to_signed(-2251818, 24), to_signed(-1608442, 24), to_signed(-965065, 24), to_signed(-321688, 24),
    to_signed(321688, 24), to_signed(965065, 24), to_signed(1608442, 24), to_signed(2251818, 24),
    to_signed(2895195, 24), to_signed(3538572, 24), to_signed(4181949, 24), to_signed(4825325, 24)
  );

  constant QAM1024 : QAM(0 to 31) := (
    to_signed(-4978853, 24), to_signed(-4657637, 24), to_signed(-4336420, 24), to_signed(-4015204, 24),
    to_signed(-3693988, 24), to_signed(-3372771, 24), to_signed(-3051555, 24), to_signed(-2730339, 24),
    to_signed(-2409122, 24), to_signed(-2087906, 24), to_signed(-1766690, 24), to_signed(-1445473, 24),
    to_signed(-1124257, 24), to_signed(-803041, 24), to_signed(-481824, 24), to_signed(-160608, 24),
    to_signed(160608, 24), to_signed(481824, 24), to_signed(803041, 24), to_signed(1124257, 24),
    to_signed(1445473, 24), to_signed(1766690, 24), to_signed(2087906, 24), to_signed(2409122, 24),
    to_signed(2730339, 24), to_signed(3051555, 24), to_signed(3372771, 24), to_signed(3693988, 24),
    to_signed(4015204, 24), to_signed(4336420, 24), to_signed(4657637, 24), to_signed(4978853, 24)
  );
```

```

constant QAM4096 : QAM(0 to 63) := (
to_signed(-5057304, 24), to_signed(-4896754, 24), to_signed(-4736205, 24), to_signed(-4575656, 24),
to_signed(-4415106, 24), to_signed(-4254557, 24), to_signed(-4094008, 24), to_signed(-3933458, 24),
to_signed(-3772909, 24), to_signed(-3612360, 24), to_signed(-3451810, 24), to_signed(-3291261, 24),
to_signed(-3130712, 24), to_signed(-2970162, 24), to_signed(-2809613, 24), to_signed(-2649064, 24),
to_signed(-2488514, 24), to_signed(-2327965, 24), to_signed(-2167416, 24), to_signed(-2006867, 24),
to_signed(-1846317, 24), to_signed(-1685768, 24), to_signed(-1525219, 24), to_signed(-1364669, 24),
to_signed(-1204120, 24), to_signed(-1043571, 24), to_signed(-883021, 24), to_signed(-722472, 24),
to_signed(-561923, 24), to_signed(-401373, 24), to_signed(-240824, 24), to_signed(-80275, 24),
to_signed(80275, 24), to_signed(240824, 24), to_signed(401373, 24), to_signed(561923, 24),
to_signed(722472, 24), to_signed(883021, 24), to_signed(1043571, 24), to_signed(1204120, 24),
to_signed(1364669, 24), to_signed(1525219, 24), to_signed(1685768, 24), to_signed(1846317, 24),
to_signed(2006867, 24), to_signed(2167416, 24), to_signed(2327965, 24), to_signed(2488514, 24),
to_signed(2649064, 24), to_signed(2809613, 24), to_signed(2970162, 24), to_signed(3130712, 24),
to_signed(3291261, 24), to_signed(3451810, 24), to_signed(3612360, 24), to_signed(3772909, 24),
to_signed(3933458, 24), to_signed(4094008, 24), to_signed(4254557, 24), to_signed(4415106, 24),
to_signed(4575656, 24), to_signed(4736205, 24), to_signed(4896754, 24), to_signed(5057304, 24);

function gray_to_binary(g : std_logic_vector) return unsigned is
    variable b : unsigned(g'range);
begin
    b(g'high) := g(g'high);
    for i in g'high-1 downto g'low loop
        b(i) := b(i+1) xor g(i);
    end loop;
    return b;
end function;

begin

process(CLK)
    variable RE : integer range 0 to L-1 := 0;
    variable IM : integer range 0 to L-1 := 0;
begin
    if rising_edge(CLK) then
        if (RST = '1') then
            OUTPUT_RE <= (others => '0');
            OUTPUT_IM <= (others => '0');
            VALID_OUT <= '0';
        else
            if (VALID_IN = '1') then
                RE := to_integer(gray_to_binary(INPUT_RE));
                IM := to_integer(gray_to_binary(INPUT_IM));
                case MQAM is
                    when 16 =>
                        OUTPUT_RE <= std_logic_vector(resize(QAM16(RE),32));
                        OUTPUT_IM <= std_logic_vector(resize(QAM16(IM),32));
                        VALID_OUT <= '1';
                    when 64 =>
                        OUTPUT_RE <= std_logic_vector(resize(QAM64(RE),32));
                        OUTPUT_IM <= std_logic_vector(resize(QAM64(IM),32));
                        VALID_OUT <= '1';
                    when 256 =>

```

```

        OUTPUT_RE <= std_logic_vector(resize(QAM256(RE),32));
        OUTPUT_IM <= std_logic_vector(resize(QAM256(IM),32));
        VALID_OUT <= '1';
    when 1024 =>
        OUTPUT_RE <= std_logic_vector(resize(QAM1024(RE),32));
        OUTPUT_IM <= std_logic_vector(resize(QAM1024(IM),32));
        VALID_OUT <= '1';
    when 4096 =>
        OUTPUT_RE <= std_logic_vector(resize(QAM4096(RE),32));
        OUTPUT_IM <= std_logic_vector(resize(QAM4096(IM),32));
        VALID_OUT <= '1';
    when others => -- Something broke, output zeros
        OUTPUT_RE <= (others => '0');
        OUTPUT_IM <= (others => '0');
        VALID_OUT <= '0';
    end case;
else
    OUTPUT_RE <= (others => '0');
    OUTPUT_IM <= (others => '0');
    VALID_OUT <= '0';
end if;
end if;
end if;
end process;
end RTL;

```

Appendix G: Data Source - VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity DATA is
  generic(
    DATA_WIDTH : integer := 2 -- Bits per I/Q per symbol
  );

  port(
    CLK : in std_logic;
    RST : in std_logic;

    RDY : in std_logic;

    VALID_OUT : out std_logic;

    OUTPUT_RE : out std_logic_vector(DATA_WIDTH-1 downto 0);
    OUTPUT_IM : out std_logic_vector(DATA_WIDTH-1 downto 0));
end DATA;

architecture RTL of DATA is

  constant LOREM : std_logic_vector(0 to 8191) := (...);

  constant N : integer := LOREM'length;

  constant STRIDE : integer := (DATA_WIDTH*2);

  signal INDEX : integer range 0 to N-1 := 0;

begin

  process(CLK)
    variable TMP_RE : std_logic_vector(DATA_WIDTH-1 downto 0);
    variable TMP_IM : std_logic_vector(DATA_WIDTH-1 downto 0);
  begin
    if rising_edge(CLK) then
      if (RST = '1') then
        INDEX <= 0;
        VALID_OUT <= '0';

        OUTPUT_RE <= (others => '0');
        OUTPUT_IM <= (others => '0');
      else
        if (RDY = '1') then

          for i in 0 to DATA_WIDTH-1 loop
            TMP_RE(DATA_WIDTH-1-i) := LOREM((INDEX+i) mod N);
            TMP_IM(DATA_WIDTH-1-i) := LOREM((INDEX+DATA_WIDTH+i) mod N);
          end loop;
```

```

        OUTPUT_RE <= TMP_RE;
        OUTPUT_IM <= TMP_IM;
        VALID_OUT <= '1';
        INDEX <= (INDEX + STRIDE) MOD N;
    else
        OUTPUT_RE <= (others => '0');
        OUTPUT_IM <= (others => '0');
        VALID_OUT <= '0';
    end if;
end if;
end if;
end process;
end RTL;

```

Appendix H: OFDM - VHDL

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity OFDM is
  generic (
    logN : integer := 10;  -- FFT size
    M : integer := 256;    -- Active data subcarriers
    k0 : integer := 22;    -- First active bin
    P : integer := 8;      -- Amount of internal pilot waves
                          -- There will also be 2 edge pilots
    CP : integer := 8      -- Length of cyclic prefix
  );
  port (
    CLK : in std_logic;
    INPUT_RE : in std_logic_vector(31 downto 0);
    INPUT_IM : in std_logic_vector(31 downto 0);

    OUTPUT_RE : out std_logic_vector(31 downto 0);
    OUTPUT_IM : out std_logic_vector(31 downto 0);

    VALID_IN : in std_logic;
    VALID_OUT : out std_logic;

    SOF : in std_logic;
    SOF_OUT : out std_logic;

    RST : in std_logic);
end entity;

architecture RTL of OFDM is

  constant N : integer := 2**logN;

  subtype word64 is std_logic_vector(63 downto 0);
  subtype word32 is std_logic_vector(31 downto 0);

  type ram_N is array (0 to N-1) of word64;

  -- Ping pong style ram for incoming / outgoing
  signal RAM_A : ram_N;
  signal RAM_B : ram_N;

  attribute ramstyle : string;
  attribute ramstyle of RAM_A : signal is "M20K";
  attribute ramstyle of RAM_B : signal is "M20K";

  signal RAM_WRT_SEL : std_logic := '0'; -- 0 => A WRITE, 1 => B WRITE
  signal RAM_READ_SEL : std_logic := '0'; -- 0 => A READ, 1 => B READ
```

```

signal FULL_A : std_logic := '0';
signal FULL_B : std_logic := '0';

signal WRT_PTR : integer range 0 to N-1 := 0;
signal READ_PTR : integer range 0 to N-1 := N - CP;

signal WRITE_BANK_FREE : std_logic := '1';

signal READ_WORD_A : word64 := (others => '0');
signal READ_WORD_B : word64 := (others => '0');
signal READ_VALID : std_logic := '0';

signal READ_SEL_DELAYED : std_logic := '0';

signal FFT_IN_RE : std_logic_vector(31 downto 0);
signal FFT_IN_IM : std_logic_vector(31 downto 0);

signal FFT_OUT_RE : std_logic_vector(31 downto 0);
signal FFT_OUT_IM : std_logic_vector(31 downto 0);

signal FFT_VALID_IN : std_logic;
signal FFT_VALID_OUT : std_logic;

constant FFT_MODE : std_logic := '0';

signal OUT_CNT : integer range 0 to (N+CP-1);

constant FIFO_SIZE : integer := N;

signal FIFO : ram_N;
attribute ramstyle of FIFO : signal is "M20K";

signal FIFO_WRT_PTR : integer range 0 to FIFO_SIZE-1 := 0;
signal FIFO_READ_PTR : integer range 0 to FIFO_SIZE- 1 := 0;

signal FIFO_CNT : integer range 0 to FIFO_SIZE := 0;

signal INPUT_IDX : integer range 0 to N := N;

constant PILOT_VAL : std_logic_vector(31 downto 0) := x"002D413C"; -- 1/sqrt(2) in Q2.22

subtype bin_code_t is std_logic_vector(1 downto 0);
constant GUARD_C : bin_code_t := "00";
constant PILOT_C : bin_code_t := "01";
constant DATA_C : bin_code_t := "10";

type bin_lut is array(0 to N-1) of bin_code_t;

```

```

function BIN_GENERATION return bin_lut is
    constant k_first : integer := k0 - 1;
    constant k_last  : integer := k0 + M + P;      -- inclusive
    constant span    : integer := k_last - k_first; -- = M + P + 1

    variable pilot_k : integer := 0;

    variable bin_map : bin_lut;
begin

    -- Mark everything as guard bins
    for i in 0 to N-1 loop
        bin_map(i) := GUARD_C;
    end loop;

    -- Mark occupied region as all data first
    for i in k_first to k_last loop
        bin_map(i) := DATA_C;
    end loop;

    for i in 1 to P loop
        pilot_k := k_first + (i*span)/(P+1);
        if (pilot_k > k_first and pilot_k < k_last) then
            bin_map(pilot_k) := PILOT_C;
        end if;
    end loop;

    bin_map(k_first) := PILOT_C;
    bin_map(k_last)  := PILOT_C;

    return bin_map;
end BIN_GENERATION;

constant BIN_MAP : bin_lut := BIN_GENERATION;
begin

    assert k0 >= 1 and k0 + M + P <= (N-1)
    report "Occupied band out of FFT bounds"
    severity failure;

FFT : entity work.FFT
    generic map(
        logN => logN
    )
    port map(
        CLK      => CLK,
        INPUT_RE  => FFT_IN_RE,
        INPUT_IM  => FFT_IN_IM,
        OUTPUT_RE => FFT_OUT_RE,
        OUTPUT_IM => FFT_OUT_IM,
        VALID_IN  => FFT_VALID_IN,
        VALID_OUT => FFT_VALID_OUT,

```

```

RST      => RST,
MODE     => FFT_MODE
);

```

```

WRITE_BANK_FREE <= '1' when ((RAM_WRT_SEL='0' and FULL_A='0') or
                             (RAM_WRT_SEL='1' and FULL_B='0'))
                    else '0';

```

```

process(CLK)
    variable FIFO_DELTA : integer := 0;
begin
    FIFO_DELTA := 0;
    if rising_edge(CLK) then
        if (RST = '1') then
            FFT_IN_RE <= (others => '0');
            FFT_IN_IM <= (others => '0');
            OUTPUT_RE <= (others => '0');
            OUTPUT_IM <= (others => '0');
            VALID_OUT <= '0';
            FFT_VALID_IN <= '0';
            OUT_CNT <= 0;
            INPUT_IDX <= N; -- N means idle
            FIFO_CNT <= 0;
            FIFO_WRT_PTR <= 0;
            FIFO_READ_PTR <= 0;
            WRT_PTR <= 0;
            READ_PTR <= N - CP;
            RAM_WRT_SEL <= '0';
            RAM_READ_SEL <= '0';
            READ_SEL_DELAYED <= '0';
            FULL_A <= '0';
            FULL_B <= '0';
            READ_VALID <= '0';
        else
            SOF_OUT <= '0';
            assert not (SOF='1' and INPUT_IDX < N and INPUT_IDX > 0)
            report "SOF asserted mid-symbol" severity failure;
            -- Write inputs into FIFO buffer
            if (VALID_IN = '1') then
                FIFO(FIFO_WRT_PTR) <= INPUT_RE & INPUT_IM;
                FIFO_DELTA := FIFO_DELTA + 1;
                FIFO_WRT_PTR <= (FIFO_WRT_PTR + 1) mod FIFO_SIZE;
            end if;
            -- If start of frame, reset input index
            if (SOF = '1') then
                INPUT_IDX <= 0;
            end if;
            -- If INPUT_IDX < N, check what type the bin is and pass that to the IFFT core
            -- If INPUT_IDX = N, do nothing, since we dont have any data to pass to the core
            -- INPUT_IDX resets when SOF = '1'

            if (INPUT_IDX < N and WRITE_BANK_FREE = '1') then

```

```

case BIN_MAP(INPUT_IDX) is
  when GUARD_C =>
    FFT_IN_RE <= (others => '0');
    FFT_IN_IM <= (others => '0');
    FFT_VALID_IN <= '1';
    INPUT_IDX <= INPUT_IDX + 1;
  when PILOT_C =>
    FFT_IN_RE <= PILOT_VAL;
    FFT_IN_IM <= PILOT_VAL;
    FFT_VALID_IN <= '1';
    INPUT_IDX <= INPUT_IDX + 1;
  when DATA_C =>
    if (FIFO_CNT > 0) then
      FFT_IN_RE <= FIFO(FIFO_READ_PTR)(63 downto 32);
      FFT_IN_IM <= FIFO(FIFO_READ_PTR)(31 downto 0);
      FIFO_DELTA := FIFO_DELTA - 1;
      FIFO_READ_PTR <= (FIFO_READ_PTR + 1) mod FIFO_SIZE;
      FFT_VALID_IN <= '1';
      INPUT_IDX <= INPUT_IDX + 1;
    else
      FFT_VALID_IN <= '0';
    end if;
  when others =>
    FFT_IN_RE <= (others => '0');
    FFT_IN_IM <= (others => '0');
    FFT_VALID_IN <= '0';
    INPUT_IDX <= INPUT_IDX + 1;
end case;
else
  FFT_VALID_IN <= '0';
end if;
-- Update FIFO_CNT
FIFO_CNT <= FIFO_CNT + FIFO_DELTA;

if (FFT_VALID_OUT = '1') then
  if (RAM_WRT_SEL = '0') then
    RAM_A(WRT_PTR) <= FFT_OUT_RE & FFT_OUT_IM;
  else
    RAM_B(WRT_PTR) <= FFT_OUT_RE & FFT_OUT_IM;
  end if;

  if (WRT_PTR = (N-1)) then
    WRT_PTR <= 0;
    RAM_WRT_SEL <= not RAM_WRT_SEL;
    if (RAM_WRT_SEL = '0') then
      FULL_A <= '1';
    else
      FULL_B <= '1';
    end if;
  else
    WRT_PTR <= WRT_PTR + 1;
  end if;
end if;

```

```

if (RAM_READ_SEL = '0' and FULL_A = '1') then
    READ_VALID <= '1';
    if (OUT_CNT = (N + CP - 1)) then
        READ_PTR <= N - CP;
        OUT_CNT <= 0;
        RAM_READ_SEL <= not RAM_READ_SEL;
        FULL_A <= '0';
    else
        READ_PTR <= (READ_PTR + 1) mod N;
        OUT_CNT <= OUT_CNT + 1;
    end if;
    if (OUT_CNT = 1) then
        SOF_OUT <= '1';
    end if;
elsif (RAM_READ_SEL = '1' and FULL_B = '1') then
    READ_VALID <= '1';
    if (OUT_CNT = (N + CP - 1)) then
        READ_PTR <= N - CP;
        OUT_CNT <= 0;
        RAM_READ_SEL <= not RAM_READ_SEL;
        FULL_B <= '0';
    else
        READ_PTR <= (READ_PTR + 1) mod N;
        OUT_CNT <= OUT_CNT + 1;
    end if;
    if (OUT_CNT = 1) then
        SOF_OUT <= '1';
    end if;
else
    READ_VALID <= '0';
end if;
READ_WORD_A <= RAM_A(READ_PTR);
READ_WORD_B <= RAM_B(READ_PTR);
VALID_OUT <= READ_VALID;
READ_SEL_DELAYED <= RAM_READ_SEL;
if (READ_SEL_DELAYED = '0') then
    OUTPUT_RE <= READ_WORD_A(63 downto 32);
    OUTPUT_IM <= READ_WORD_A(31 downto 0 );
else
    OUTPUT_RE <= READ_WORD_B(63 downto 32);
    OUTPUT_IM <= READ_WORD_B(31 downto 0 );
end if;
end if;
end if;
end process;
end RTL;

```

Appendix I: SC-FDMA - VHDL

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity SC_FDMA is
  generic (
    logN : integer := 10;  -- FFT size
    logM : integer := 8;   -- Active data subcarriers
    k0 : integer := 22;    -- First active bin
    P : integer := 8;      -- Amount of internal pilot waves
                          -- There will also be 2 edge pilots
    CP : integer := 8      -- Length of cyclic prefix
  );
  port (
    CLK : in std_logic;
    INPUT_RE : in std_logic_vector(31 downto 0);
    INPUT_IM : in std_logic_vector(31 downto 0);

    OUTPUT_RE : out std_logic_vector(31 downto 0);
    OUTPUT_IM : out std_logic_vector(31 downto 0);

    VALID_IN : in std_logic;
    VALID_OUT : out std_logic;

    RST : in std_logic);
end entity;

architecture RTL of SC_FDMA is

  constant N : integer := 2**logN;
  constant M : integer := 2**logM;

  subtype word64 is std_logic_vector(63 downto 0);
  type FIFO_ram is array(0 to M-1) of word64;

  signal FIFO : FIFO_ram;

  attribute ramstyle : string;
  attribute ramstyle of FIFO : signal is "M20K";

  constant FIFO_SIZE : integer := M;
  signal FIFO_CNT : integer range 0 to FIFO_SIZE := 0;
  signal READ_PTR : integer range 0 to FIFO_SIZE-1 := 0;
  signal WRT_PTR : integer range 0 to FIFO_SIZE-1 := 0;

  signal DRAIN : std_logic := '0';

  signal FFT_VALID_OUT : std_logic;
  signal FFT_VALID_IN : std_logic := '0';

  signal FFT_INPUT_RE : std_logic_vector(31 downto 0);
```

```

signal FFT_INPUT_IM : std_logic_vector(31 downto 0);

signal FFT_OUTPUT_RE : std_logic_vector(31 downto 0);
signal FFT_OUTPUT_IM : std_logic_vector(31 downto 0);

signal OFDM_VALID_IN : std_logic;

signal OFDM_INPUT_RE : std_logic_vector(31 downto 0);
signal OFDM_INPUT_IM : std_logic_vector(31 downto 0);

signal SOF : std_logic := '0';

constant FFT_MODE : std_logic := '1';

constant SCALE : natural := logM/2;

begin

    DFT : entity work.FFT
    generic map(
        logN => logM
    )
    port map(
        CLK      => CLK,
        INPUT_RE  => FFT_INPUT_RE,
        INPUT_IM  => FFT_INPUT_IM,
        OUTPUT_RE => FFT_OUTPUT_RE,
        OUTPUT_IM => FFT_OUTPUT_IM,
        VALID_IN  => FFT_VALID_IN,
        VALID_OUT => FFT_VALID_OUT,
        RST       => RST,
        MODE      => FFT_MODE
    );

    OFDM : entity work.OFDM
    generic map(
        logN => logN,
        M => M,
        k0 => k0,
        P => P,
        CP => CP
    )
    port map(
        CLK      => CLK,
        INPUT_RE  => OFDM_INPUT_RE,
        INPUT_IM  => OFDM_INPUT_IM,
        OUTPUT_RE => OUTPUT_RE,
        OUTPUT_IM => OUTPUT_IM,
        VALID_IN  => OFDM_VALID_IN,
        VALID_OUT => VALID_OUT,
        RST       => RST,
        SOF       => SOF
    );

```

```

assert logN >= logM
report "SC_FDMA: logN must be >= logM for post-DFT scaling"
severity failure;

process(CLK)
    variable FIFO_DELTA : integer range -1 to 1 := 0;
begin
    if rising_edge(CLK) then
        FIFO_DELTA := 0;

        if (RST = '1') then
            FIFO_CNT <= 0;
            READ_PTR <= 0;
            WRT_PTR  <= 0;

            DRAIN <= '0';

            FFT_VALID_IN <= '0';
            FFT_INPUT_RE <= (others => '0');
            FFT_INPUT_IM <= (others => '0');

            SOF <= '0';
            OFDM_VALID_IN <= '0';

        else

            FFT_VALID_IN <= VALID_IN;
            FFT_INPUT_RE <= INPUT_RE;
            FFT_INPUT_IM <= INPUT_IM;

            -- DEFAULTS
            SOF <= '0';

            if (FFT_VALID_OUT = '1') then
                FIFO(WRT_PTR) <= std_logic_vector(shift_right(signed(FFT_OUTPUT_RE), SCALE))
                    & std_logic_vector(shift_right(signed(FFT_OUTPUT_IM), SCALE));
                FIFO_DELTA := FIFO_DELTA + 1;
                if (WRT_PTR = M-1) then
                    DRAIN <= '1';
                    WRT_PTR <= 0;
                else
                    WRT_PTR <= WRT_PTR + 1;
                end if;
            end if;

            if (DRAIN = '1') then

                OFDM_INPUT_RE <= FIFO(READ_PTR)(63 downto 32);
                OFDM_INPUT_IM <= FIFO(READ_PTR)(31 downto 0 );
                OFDM_VALID_IN <= '1';
                FIFO_DELTA := FIFO_DELTA - 1;
            end if;
        end if;
    end if;
end process;

```

```

    if (READ_PTR = M - 1) then
        READ_PTR <= 0;
        DRAIN <= '0';
    elsif (READ_PTR = 0) then
        READ_PTR <= READ_PTR + 1;
        SOF <= '1';
        report "SOF = '1'";
        report integer'image(FIFO_CNT);
    else
        READ_PTR <= READ_PTR + 1;
        SOF <= '0';
    end if;
else
    OFDM_VALID_IN <= '0';

end if;

FIFO_CNT <= FIFO_CNT + FIFO_DELTA;

assert not (FIFO_CNT = M and FIFO_DELTA = 1)
report "FIFO overflowing"
severity failure;

assert not (FIFO_CNT = 0 and FIFO_DELTA = -1)
report "FIFO underflowing"
severity failure;

assert not (FFT_VALID_OUT = '1' and DRAIN = '1')
report "FFTING while still draining"
severity failure;

assert not (VALID_IN='1' and DRAIN='1')
report "Receiving inputs while draining"
severity failure;

    end if;
end if;
end process;
end RTL;

```



```

signal SYMS_IM : sym_vec := (others => (others => '0'));

signal PHASE_CNT : integer range 0 to 9 := 0;

begin

process(CLK)
    variable ACC_RE      : signed64;
    variable ACC_IM      : signed64;
    variable SYMS_RE_V   : sym_vec;    -- variable shadow of SH
    variable SYMS_IM_V   : sym_vec;    -- variable shadow of SH
begin
    if rising_edge(CLK) then
        if (RST = '1') then
            PHASE_CNT <= 0;
            VALID_OUT <= '0';
            SYMS_RE <= (others => (others => '0'));
            SYMS_IM <= (others => (others => '0'));
            OUTPUT_RE <= (others => '0');
            OUTPUT_IM <= (others => '0');

        else
            if VALID_IN = '1' or PHASE_CNT > 0 then
                VALID_OUT <= '1';
                SYMS_RE_V := SYMS_RE;
                SYMS_IM_V := SYMS_IM;

                if PHASE_CNT = 0 then
                    for k in 6 downto 1 loop
                        SYMS_RE_V(k) := SYMS_RE_V(k-1);
                        SYMS_IM_V(k) := SYMS_IM_V(k-1);
                    end loop;

                    SYMS_RE_V(0) := signed(INPUT_RE);
                    SYMS_IM_V(0) := signed(INPUT_IM);
                end if;

                ACC_RE := (others => '0');
                ACC_IM := (others => '0');
                for k in 0 to 6 loop
                    ACC_RE := ACC_RE + (C(PHASE_CNT)(k) * SYMS_RE_V(k));
                    ACC_IM := ACC_IM + (C(PHASE_CNT)(k) * SYMS_IM_V(k));
                end loop;

                OUTPUT_RE <= std_logic_vector(resize(shift_right(ACC_RE,SCALE),32));
                OUTPUT_IM <= std_logic_vector(resize(shift_right(ACC_IM,SCALE),32));

                SYMS_RE <= SYMS_RE_V;
                SYMS_IM <= SYMS_IM_V;

                PHASE_CNT <= (PHASE_CNT + 1) mod 10;
            end if;
        end if;
    end if;
end process;

```

```
    if (PHASE_CNT = 2) then
        TRIG <= '1';
    else
        TRIG <= '0';
    end if;
else
    VALID_OUT <= '0';
end if;
end if;
end if;
end process;
end RTL;
```

Appendix K: Frequency Shifter - VHDL

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity Freq is
  generic(
    STEP : integer := 410 --  $410 * 50\text{MHz}/4096 = 5.005\text{MHz}$ 
  );
  port(

    CLK : in std_logic;
    RST : in std_logic;

    INPUT_RE : in std_logic_vector(31 downto 0);
    INPUT_IM : in std_logic_vector(31 downto 0);

    VALID_IN : in std_logic;
    VALID_OUT : out std_logic;

    OUTPUT_RE : out std_logic_vector(31 downto 0);
    OUTPUT_IM : out std_logic_vector(31 downto 0)
  );
end entity;

architecture RTL of Freq is

  constant LUT_SIZE : integer := 4096;
  type lut_t is array(0 to LUT_SIZE-1) of std_logic_vector(15 downto 0);

  constant COS_LUT : lut_t := (...) -- 4096 samples of Q1.16 Cos
  constant SIN_LUT : lut_t := (...) -- 4096 samples of Q1.16 Sin

  attribute romstyle : string;
  attribute romstyle of COS_LUT : constant is "M20K";
  attribute romstyle of SIN_LUT : constant is "M20K";

  signal INDEX : integer range 0 to LUT_SIZE-1 := 0;

begin

  process(CLK)
    variable p_ic, p_qs, p_is, p_qc : signed(47 downto 0);
    variable RE_mix, IM_mix : signed(48 downto 0);
  begin
    if rising_edge(CLK) then
      if (RST = '1') then
        INDEX <= 0;
        VALID_OUT <= '0';
        OUTPUT_RE <= (others => '0');
        OUTPUT_IM <= (others => '0');
      else
        if (VALID_IN = '1') then
          p_ic := resize(signed(INPUT_RE) * signed(COS_LUT(INDEX)), 48);
```

```

p_qs := resize(signed(INPUT_IM) * signed(SIN_LUT(INDEX)), 48);
p_is := resize(signed(INPUT_RE) * signed(SIN_LUT(INDEX)), 48);
p_qc := resize(signed(INPUT_IM) * signed(COS_LUT(INDEX)), 48);

RE_mix := resize(p_ic, 49) - resize(p_qs, 49);
IM_mix := resize(p_is, 49) + resize(p_qc, 49);

OUTPUT_RE <= std_logic_vector(resize(shift_right(RE_mix,15),32));
OUTPUT_IM <= std_logic_vector(resize(shift_right(IM_mix,15),32));

VALID_OUT <= '1';

INDEX <= (INDEX + STEP) mod LUT_SIZE;

else
    VALID_OUT <= '0';
    OUTPUT_RE <= (others => '0');
    OUTPUT_IM <= (others => '0');
end if;

end if;
end if;
end process;
end RTL;

```

Appendix L: OFDM Controller - VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity SDR is
  generic(
    logN : integer := 10;    -- Size of IFFT for OFDM
    logM : integer := 8;    -- Amount of data carriers

    logQAM : integer := 4;   -- MQAM

    k0 : integer := 22;     -- First active bin
    P : integer := 8;       -- Amount of pilot bins (excluding the two edge waves)

    CP : integer := 8;      -- Length of cyclic prefix

    DAC_WIDTH : integer := 14; -- DAC resolution

    BACK_OFF : integer := -4 -- Extra back-off headroom
  );
  port(
    CLK : in std_logic;
    RST : in std_logic;

    TRIG : out std_logic;

    DA_MODE : out std_logic;
    POWER_ON : out std_logic;

    PLL_OUT_DAC0 : out std_logic;
    PLL_OUT_DAC1 : out std_logic;

    DA_WRTA : out std_logic;
    DA_WRTB : out std_logic;

    DA_DA : out std_logic_vector(13 downto 0) := (others => '0');
    DA_DB : out std_logic_vector(13 downto 0) := (others => '0')
  );
end SDR;

architecture RTL of SDR is

  constant N : integer := 2**logN;
  constant M : integer := 2**logM;

  signal SOURCE_OUT_RE : std_logic_vector((logQAM/2)-1 downto 0);
  signal SOURCE_OUT_IM : std_logic_vector((logQAM/2)-1 downto 0);

  signal SOURCE_VALID_IN : std_logic := '0';
  signal SOURCE_VALID_OUT : std_logic;
```

```

signal SHAPER_OUT_RE : std_logic_vector(31 downto 0);
signal SHAPER_OUT_IM : std_logic_vector(31 downto 0);

signal SHAPER_VALID_OUT : std_logic;

signal OFDM_OUT_RE : std_logic_vector(31 downto 0);
signal OFDM_OUT_IM : std_logic_vector(31 downto 0);

signal OFDM_VALID_OUT : std_logic;

constant DATA_WIDTH : integer := logQAM/2;

constant DAC_SCALE : integer := 24 - DAC_WIDTH + BACK_OFF; -- 24 is the size of the data.

signal CNT : integer range 0 to (N+CP-1) := 0; -- Used to control data flow over time

signal SOF : std_logic := '0';

signal SOF_OUT : std_logic;

signal SHAPER_VALID_D : std_logic := '0'; -- Delayed Shaper_Valid_Out for SOF detection

begin

SOURCE : entity work.DATA
generic map(
DATA_WIDTH => DATA_WIDTH
)
port map(
CLK      => CLK,
RST      => RST,

RDY      => SOURCE_VALID_IN,
VALID_OUT => SOURCE_VALID_OUT,
OUTPUT_RE => SOURCE_OUT_RE,
OUTPUT_IM => SOURCE_OUT_IM
);

SHAPER : entity work.Symbol_Shaper
generic map(
logQAM => logQAM
)
port map(
CLK      => CLK,
RST      => RST,

INPUT_RE  => SOURCE_OUT_RE,
INPUT_IM  => SOURCE_OUT_IM,
VALID_IN  => SOURCE_VALID_OUT,

OUTPUT_RE => SHAPER_OUT_RE,
OUTPUT_IM => SHAPER_OUT_IM,

```

```

VALID_OUT  => SHAPER_VALID_OUT
);

OFDM : entity work.OFDM
generic map(
logN      => logN,
M         => M,
k0        => k0,
P         => P,
CP        => CP
)
port map(
CLK       => CLK,
RST       => RST,

SOF       => SOF,
SOF_OUT   => SOF_OUT,

INPUT_RE  => SHAPER_OUT_RE,
INPUT_IM  => SHAPER_OUT_IM,
VALID_IN  => SHAPER_VALID_OUT,

OUTPUT_RE => OFDM_OUT_RE,
OUTPUT_IM => OFDM_OUT_IM,
VALID_OUT => OFDM_VALID_OUT
);

DAC : entity work.DAC
generic map(
DAC_WIDTH => DAC_WIDTH
)
port map(
CLK       => CLK,
RST       => RST,

INPUT_RE  => OFDM_OUT_RE,
INPUT_IM  => OFDM_OUT_IM,
VALID_IN  => OFDM_VALID_OUT,

SCALE     => DAC_SCALE,

DA_MODE   => DA_MODE,
POWER_ON  => POWER_ON,

PLL_OUT_DAC0=> PLL_OUT_DAC0,
PLL_OUT_DAC1=> PLL_OUT_DAC1,

DA_WRTA   => DA_WRTA,
DA_WRTB   => DA_WRTB,

DA_DA     => DA_DA,
DA_DB     => DA_DB
);

```

```

SOF <= SHAPER_VALID_OUT and not SHAPER_VALID_D;

process(CLK)

begin
    if rising_edge(CLK) then
        if (RST = '1') then
            SOURCE_VALID_IN <= '0';
            SHAPER_VALID_D <= '0';
            CNT <= 0;
        else
            TRIG <= SOF_OUT;
            if (CNT < M) then
                SOURCE_VALID_IN <= '1';
            else
                SOURCE_VALID_IN <= '0';
            end if;
            CNT <= (CNT + 1) mod (N + CP);
            SHAPER_VALID_D <= SHAPER_VALID_OUT;
        end if;
    end if;
end process;
end RTL;

```

Appendix M: SC-FDMA Controller - VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity SDR is
  generic(
    logN : integer := 10;    -- Size of IFFT for OFDM
    logM : integer := 8;    -- Size of FFT for SC-FDMA

    logQAM : integer := 4;   -- MQAM

    k0 : integer := 22;      -- First data bin
    P : integer := 8;        -- Amount of pilot bins (excluding the two edge waves)

    CP : integer := 8;       -- Length of cyclic prefix

    DAC_WIDTH : integer := 14; -- DAC resolution

    BACK_OFF : integer := -6 -- Extra back-off headroom
  );
  port(
    CLK : in std_logic;
    RST : in std_logic;

    TRIG : out std_logic;

    DA_MODE : out std_logic;
    POWER_ON : out std_logic;

    PLL_OUT_DAC0 : out std_logic;
    PLL_OUT_DAC1 : out std_logic;

    DA_WRTA : out std_logic;
    DA_WRTB : out std_logic;

    DA_DA : out std_logic_vector(13 downto 0) := (others => '0');
    DA_DB : out std_logic_vector(13 downto 0) := (others => '0')
  );
end SDR;

architecture RTL of SDR is

  constant N : integer := 2**logN;
  constant M : integer := 2**logM;

  signal SOURCE_OUT_RE : std_logic_vector((logQAM/2)-1 downto 0);
  signal SOURCE_OUT_IM : std_logic_vector((logQAM/2)-1 downto 0);

  signal SOURCE_VALID_IN : std_logic := '0';
  signal SOURCE_VALID_OUT : std_logic;
```

```

signal SHAPER_OUT_RE : std_logic_vector(31 downto 0);
signal SHAPER_OUT_IM : std_logic_vector(31 downto 0);

signal SHAPER_VALID_OUT : std_logic;

signal SC_OUT_RE : std_logic_vector(31 downto 0);
signal SC_OUT_IM : std_logic_vector(31 downto 0);

signal SC_VALID_OUT : std_logic;
signal SC_VALID_OUT_D : std_logic;

constant DATA_WIDTH : integer := logQAM/2;

constant DAC_SCALE : integer := 24 - DAC_WIDTH + BACK_OFF; -- 24 is the size of the data.

signal CNT : integer range 0 to (N+CP-1) := 0; -- Used to control data flow over time

begin

SOURCE : entity work.DATA
generic map(
DATA_WIDTH => DATA_WIDTH
)
port map(
CLK      => CLK,
RST      => RST,

RDY      => SOURCE_VALID_IN,
VALID_OUT => SOURCE_VALID_OUT,
OUTPUT_RE => SOURCE_OUT_RE,
OUTPUT_IM => SOURCE_OUT_IM
);

SHAPER : entity work.Symbol_Shaper
generic map(
logQAM => logQAM
)
port map(
CLK      => CLK,
RST      => RST,

INPUT_RE  => SOURCE_OUT_RE,
INPUT_IM  => SOURCE_OUT_IM,
VALID_IN  => SOURCE_VALID_OUT,

OUTPUT_RE => SHAPER_OUT_RE,
OUTPUT_IM => SHAPER_OUT_IM,
VALID_OUT => SHAPER_VALID_OUT
);

SC : entity work.SC_FDMA
generic map(
logN      => logN,

```

```

logM      => logM,
k0        => k0,
P         => P,
CP        => CP
)
port map(
CLK       => CLK,
RST       => RST,

INPUT_RE  => SHAPER_OUT_RE,
INPUT_IM  => SHAPER_OUT_IM,
VALID_IN  => SHAPER_VALID_OUT,

OUTPUT_RE => SC_OUT_RE,
OUTPUT_IM => SC_OUT_IM,
VALID_OUT => SC_VALID_OUT
);

DAC : entity work.DAC
generic map(
DAC_WIDTH => DAC_WIDTH
)
port map(
CLK       => CLK,
RST       => RST,

INPUT_RE  => SC_OUT_RE,
INPUT_IM  => SC_OUT_IM,
VALID_IN  => SC_VALID_OUT,

SCALE     => DAC_SCALE,

DA_MODE   => DA_MODE,
POWER_ON  => POWER_ON,

PLL_OUT_DAC0=> PLL_OUT_DAC0,
PLL_OUT_DAC1=> PLL_OUT_DAC1,

DA_WRTA   => DA_WRTA,
DA_WRTB   => DA_WRTB,

DA_DA     => DA_DA,
DA_DB     => DA_DB
);

process(CLK)

begin
    if rising_edge(CLK) then
        if (RST = '1') then
            SOURCE_VALID_IN <= '0';
            CNT <= 0;
            TRIG <= '0';

```

```

        SC_VALID_OUT_D <= '0';
    else
        if (CNT < M) then
            SOURCE_VALID_IN <= '1';
        else
            SOURCE_VALID_IN <= '0';
        end if;
        CNT <= (CNT + 1) mod (N + CP);
        SC_VALID_OUT_D <= SC_VALID_OUT;
        TRIG <= SC_VALID_OUT and not SC_VALID_OUT_D;
    end if;
end if;
end process;
end RTL;

```

Appendix N: SC Controller - VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity SDR is
  generic(

    logQAM : integer := 4;          -- MQAM

    PULSE_SCALE : natural := 14;    -- Shift factor for pulse shaper

    FREQ_STEP : integer := 410;     -- Frequency shifter step size

    DAC_WIDTH : natural := 14;      -- DAC resolution

    BACK_OFF : integer := -1       -- Extra back-off headroom
  );

  port(
    CLK : in std_logic;
    RST : in std_logic;

    TRIG : out std_logic;

    DA_MODE : out std_logic;
    POWER_ON : out std_logic;

    PLL_OUT_DAC0 : out std_logic;
    PLL_OUT_DAC1 : out std_logic;

    DA_WRTA : out std_logic;
    DA_WRTB : out std_logic;

    DA_DA : out std_logic_vector(13 downto 0) := (others => '0');
    DA_DB : out std_logic_vector(13 downto 0) := (others => '0')
  );
end SDR;

architecture RTL of SDR is

  signal SOURCE_OUT_RE : std_logic_vector((logQAM/2)-1 downto 0);
  signal SOURCE_OUT_IM : std_logic_vector((logQAM/2)-1 downto 0);

  signal SOURCE_VALID_IN : std_logic := '0';
  signal SOURCE_VALID_OUT : std_logic;

  signal SHAPER_OUT_RE : std_logic_vector(31 downto 0);
  signal SHAPER_OUT_IM : std_logic_vector(31 downto 0);

  signal SHAPER_VALID_OUT : std_logic;
```

```

signal PULSE_OUT_RE : std_logic_vector(31 downto 0);
signal PULSE_OUT_IM : std_logic_vector(31 downto 0);

signal PULSE_VALID_OUT : std_logic;

signal FREQ_OUT_RE : std_logic_vector(31 downto 0);
signal FREQ_OUT_IM : std_logic_vector(31 downto 0);

signal FREQ_VALID_OUT : std_logic;

constant DATA_WIDTH : integer := logQAM/2;
constant DAC_SCALE : integer := 24 - DAC_WIDTH + BACK_OFF; -- 24 is the size of the data.

signal CNT : integer range 0 to 9 := 0;

begin

SOURCE : entity work.DATA
generic map(
DATA_WIDTH => DATA_WIDTH
)
port map(
CLK      => CLK,
RST      => RST,

RDY      => SOURCE_VALID_IN,
VALID_OUT => SOURCE_VALID_OUT,
OUTPUT_RE => SOURCE_OUT_RE,
OUTPUT_IM => SOURCE_OUT_IM
);

SHAPER : entity work.Symbol_Shaper
generic map(
logM => logQAM
)
port map(
CLK      => CLK,
RST      => RST,

INPUT_RE  => SOURCE_OUT_RE,
INPUT_IM  => SOURCE_OUT_IM,
VALID_IN  => SOURCE_VALID_OUT,

OUTPUT_RE => SHAPER_OUT_RE,
OUTPUT_IM => SHAPER_OUT_IM,
VALID_OUT => SHAPER_VALID_OUT
);

PULSE : entity work.Pulse
generic map(
SCALE    => PULSE_SCALE
)
port map(

```

```

CLK      => CLK,
RST      => RST,
TRIG     => TRIG,

INPUT_RE  => SHAPER_OUT_RE,
INPUT_IM  => SHAPER_OUT_IM,
VALID_IN  => SHAPER_VALID_OUT,

OUTPUT_RE => PULSE_OUT_RE,
OUTPUT_IM => PULSE_OUT_IM,
VALID_OUT => PULSE_VALID_OUT
);

FREQ : entity work.FREQ
generic map(
STEP => FREQ_STEP
)
port map(
CLK      => CLK,
RST      => RST,

INPUT_RE  => PULSE_OUT_RE,
INPUT_IM  => PULSE_OUT_IM,
VALID_IN  => PULSE_VALID_OUT,

OUTPUT_RE => FREQ_OUT_RE,
OUTPUT_IM => FREQ_OUT_IM,
VALID_OUT => FREQ_VALID_OUT
);

DAC : entity work.DAC
generic map(
DAC_WIDTH => DAC_WIDTH
)
port map(
CLK      => CLK,
RST      => RST,

INPUT_RE  => FREQ_OUT_RE,
INPUT_IM  => FREQ_OUT_IM,
VALID_IN  => FREQ_VALID_OUT,

SCALE     => DAC_SCALE,

DA_MODE   => DA_MODE,
POWER_ON  => POWER_ON,

PLL_OUT_DAC0=> PLL_OUT_DAC0,
PLL_OUT_DAC1=> PLL_OUT_DAC1,

DA_WRTA   => DA_WRTA,
DA_WRTB   => DA_WRTB,

DA_DA     => DA_DA,

```

```

DA_DB      => DA_DB
);

process(CLK)
begin
    if rising_edge(CLK) then
        if (RST = '1') then
            CNT <= 0;
            SOURCE_VALID_IN <= '0';
        else
            if (CNT = 0) then
                SOURCE_VALID_IN <= '1';
            else
                SOURCE_VALID_IN <= '0';
            end if;
            CNT <= (CNT + 1) mod 10;
        end if;
    end if;
end process;
end RTL;

```

Appendix O: DAC Interface - VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity DAC is
  generic (
    DAC_WIDTH : integer := 14);
  port (
    CLK : in std_logic;
    RST : in std_logic;

    INPUT_RE : in std_logic_vector(31 downto 0);
    INPUT_IM : in std_logic_vector(31 downto 0);

    VALID_IN : in std_logic;

    SCALE : in natural range 0 to 31;

    DA_MODE : out std_logic;
    POWER_ON : out std_logic;

    PLL_OUT_DAC0 : out std_logic;
    PLL_OUT_DAC1 : out std_logic;

    DA_WRTA : out std_logic;
    DA_WRTB : out std_logic;

    DA_DA : out std_logic_vector(13 downto 0) := (others => '0');
    DA_DB : out std_logic_vector(13 downto 0) := (others => '0'));
end DAC;

architecture RTL of DAC is

  subtype dac_uint_t is unsigned(DAC_WIDTH-1 downto 0);
  subtype dac_int_t is signed(DAC_WIDTH-1 downto 0);

  function signed_to_offset(x : dac_int_t) return dac_uint_t is
    variable u : signed(DAC_WIDTH downto 0);
  begin
    u := resize(x,DAC_WIDTH+1) + to_signed(2**(DAC_WIDTH-1),DAC_WIDTH+1); -- +8192
    return resize(unsigned(u),DAC_WIDTH);
  end;

  function scale_to_dac (x : signed; s : natural) return dac_int_t is
    variable u : signed(x'length-1 downto 0);
  begin
    u := shift_right(x,s);
    if u > to_signed(2**(DAC_WIDTH-1) - 1,x'length) then
      return to_signed(2**(DAC_WIDTH-1)-1,DAC_WIDTH);
    elsif (u < to_signed(-2**(DAC_WIDTH-1),x'length)) then

```

```

        return to_signed(-2**(DAC_WIDTH-1),DAC_WIDTH);
    else
        return resize(u,DAC_WIDTH);
    end if;
end;

begin

    PLL_OUT_DAC0 <= CLK;
    PLL_OUT_DAC1 <= CLK;

    DA_WRTA <= not CLK;
    DA_WRTB <= not CLK;

    DA_MODE <= '1'; -- Dual Channel Mode
    POWER_ON <= '1'; -- On/Off

    process(CLK)
        variable DATA_RE : signed(31 downto 0);
        variable DATA_IM : signed(31 downto 0);

        variable DATA_A : dac_uint_t;
        variable DATA_B : dac_uint_t;

        constant ZERO : dac_uint_t := signed_to_offset((others => '0'));

    begin
        if rising_edge(CLK) then
            if (RST = '1') then
                DATA_A := ZERO;
                DATA_B := ZERO;
            else

                DATA_RE := signed(INPUT_RE);
                DATA_IM := signed(INPUT_IM);

                if (VALID_IN = '1') then
                    DATA_A := signed_to_offset(scale_to_dac(DATA_RE,SCALE));
                    DATA_B := signed_to_offset(scale_to_dac(DATA_IM,SCALE));
                else
                    DATA_A := ZERO;
                    DATA_B := ZERO;
                end if;
            end if;
            DA_DA <= std_logic_vector(DATA_A);
            DA_DB <= std_logic_vector(DATA_B);
        end if;
    end process;
end RTL;

```

Appendix P: Schematics and PCBs

The following pages consist of the Schematics and PCBs, in the following order:

1. Low-Pass Filter Schematic
2. Fully-Differential Amplifier Schematic
3. Quadrature Modulator Schematic
4. RF Front-End Schematic
5. RF Front-End PCB
6. PLL Schematic
7. PLL PCB

